

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026

Oleksandr Aronov

Univerzita Pardubice
Fakulta elektrotechniky a informatiky

Webová platforma typu marketplace pro digitální produkty a služby
Bakalářská práce

Univerzita Pardubice
Fakulta elektrotechniky a informatiky
Akademický rok: 2025/2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Oleksandr Aronov
Osobní číslo:	I21118
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Webová platforma typu marketplace pro digitální produkty a služby
Zadávající katedra:	Katedra informačních technologií

Zásady pro vypracování

Cílem bakalářské práce je návrh a implementace webové aplikace typu marketplace, která umožní uživatelům obchodovat s digitálními produkty a službami. Aplikace poskytne uživatelům možnost registrace, správy nabídek, komunikace prostřednictvím integrovaného chatu a hodnocení prodávajících. V teoretické části bude provedena analýza existujících řešení a jejich funkcionalit. Na základě této analýzy bude navržena architektura systému s důrazem na bezpečnost, přehlednost a rozšiřitelnost. Praktická část práce se zaměří na vývoj webové aplikace využívající moderní technologie pro frontend, backend a databázovou vrstvu. Výsledná aplikace bude zahrnovat uživatelské rozhraní, backend logiku pro správu nabídek, objednávek a uživatelů a funkce pro komunikaci mezi prodávajícími a kupujícími.

Rozsah pracovní zprávy: **30-40 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná/elektronická**

Seznam doporučené literatury:

FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. ISBN 978-0321127426. KRUG, Steve. Don't Make Me Think: A Common Sense Approach to Web Usability. 3rd ed. New Riders, 2014. ISBN 978-0321965516.

Vedoucí bakalářské práce: **Ing. Monika Borkovcová, Ph.D.**
Katedra informačních technologií

Datum zadání bakalářské práce: **12. prosince 2025**
Termín odevzdání bakalářské práce: **15. května 2026**

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

LS.

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 27. února 2026

Prohlašuji:

Práci s názvem Webová platforma typu marketplace pro digitální produkty a služby jsem vypracoval samostatně a uvedl v ní všechny použité literární a informační zdroje v souladu s právními předpisy, vnitřními předpisy a vnitřními normami Univerzity Pardubice.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2025 Pravidla pro odevzdávání, zpřístupňování závěrečných prací veřejnosti a jejich formální úpravu, bude závěrečná práce zpřístupněna veřejnosti prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 12. 05. 2026

Oleksandr Aronov

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu této bakalářské práce za odborné vedení, užitečné připomínky a čas, který mi věnoval během konzultací.

Dále bych chtěl poděkovat vyučujícím Fakulty elektrotechniky a informatiky Univerzity Pardubice za znalosti a zkušenosti, které jsem mohl během studia získat a které mi pomohly při zpracování této práce.

Velké poděkování patří také mé rodině a blízkým za podporu, trpělivost a důvěru během celého studia. Jejich pomoc a motivace pro mě byly důležité zejména při dokončování této práce.

ANOTACE

Cílem této bakalářské práce je návrh a implementace webové marketplace platformy pro digitální produkty a služby. Práce se zabývá analýzou existujících řešení, návrhem architektury systému a implementací fullstack aplikace postavené na frameworku NestJS, knihovně React a databázi PostgreSQL. Systém je navržen jako modulární monolit a jeho klíčovou součástí je interní peněženka s ledgerem a escrow mechanismem pro bezpečnější zpracování obchodních operací. Aplikace dále obsahuje realtime komunikaci pomocí Socket.IO, recenze, reportování obsahu, administrátorské rozhraní a základní bezpečnostní opatření.

KLÍČOVÁ SLOVA

webová aplikace, marketplace, digitální produkty, digitální služby, React, NestJS, PostgreSQL, Prisma, Socket.IO, escrow, interní peněženka, ledger, modulární monolit, realtime komunikace, webová bezpečnost

TITLE

Marketplace Web Platform for Digital Products and Services

ANNOTATION

The aim of this bachelor thesis is to design and implement a web marketplace platform for digital products and services. The thesis focuses on the analysis of existing solutions, system architecture design, and implementation of a fullstack application built with the NestJS framework, React library, and PostgreSQL database. The system follows a modular monolith architecture and its key part is an internal wallet with a ledger and escrow mechanism for safer processing of business operations. The application also includes real-time communication using Socket.IO, reviews, content reporting, an administration interface, and basic security measures.

KEYWORDS

web application, marketplace, digital products, digital services, React, NestJS, PostgreSQL, Prisma, Socket.IO, escrow, internal wallet, ledger, modular monolith, real-time communication, web security

OBSAH

SEZNAM ILUSTRACÍ A TABULEK.....	12
SEZNAM ZKRATEK A ZNAČEK	14
TERMINOLOGIE	15
ÚVOD.....	16
1 Webové aplikace.....	18
1.1 Architektura webových aplikací	18
1.1.1 Monolitická architektura.....	18
1.1.2 Mikroslužbová architektura	19
1.1.3 Modulární monolit	19
1.1.4 Zhodnocení architektur	19
1.2 Architektonické vzory.....	20
1.2.1 MVC	20
1.2.2 MVVM.....	20
1.3 Technologie pro vývoj webových aplikací.....	20
1.3.1 Backendové technologie	21
1.3.2 Frontendové technologie.....	21
1.3.3 Databázové systémy	21
1.3.4 Transakce a konzistence dat	22
1.3.5 Realtime komunikace	22
1.3.6 Bezpečnost webových aplikací	22
1.3.7 Moderní nástroje pro vývoj.....	23
2 Analýza existujících řešení	24
2.1 FunPay	24
2.2 G2A.....	24
2.3 GGsel	25
2.4 Plati.Market.....	25
2.5 Srovnání existujících řešení	25
2.6 Dopady analýzy na návrh vlastního systému.....	26
3 Analýza požadavků.....	28
3.1 Funkční požadavky	28

3.2 Nefunkční požadavky	29
3.3 Případy užití systému.....	29
4 Návrh systému	31
4.1 Architektura systému	31
4.2 Volba technologií.....	32
4.3 Backendová část systému	34
4.4 Frontendová část systému.....	34
4.5 Databázový návrh	35
4.6 Komunikace v systému.....	36
5 Implementace.....	37
5.1 Struktura projektu	37
5.1.1 Monorepozitář.....	38
5.1.2 Backendová struktura	38
5.1.3 Frontendová struktura	39
5.1.4 Konfigurační soubory	40
5.2 Implementace backendové části	40
5.2.1 Struktura NestJS aplikace	40
5.2.2 Použité závislosti	41
5.2.3 Konfigurace aplikace	42
5.2.4 Autentizace a autorizace	42
5.2.5 Uživatelé a profily	45
5.2.6 Nabídky a katalog	46
5.2.7 Obchody a escrow logika.....	47
5.2.8 Peněženka a ledger transakce	48
5.2.9 Chat, konverzace a zprávy	49
5.2.10 Recenze a reputace uživatele	50
5.2.11 Reporty, moderace a administrace	51
5.2.12 Notifikace, achievementy a doplňkové funkce.....	52
5.2.13 Nahrávání souborů	53
5.3 Implementace frontendové části	54
5.3.1 Struktura React aplikace	54

5.3.2 Routing a chráněné stránky.....	55
5.3.3 API klient a práce s tokenem	56
5.3.4 Autentizační obrazovky	57
5.3.5 Katalog a detail nabídky	58
5.3.6 Správa nabídek.....	60
5.3.7 Obchody a peněženka	62
5.3.8 Chat a inbox	63
5.3.9 Uživatelský profil a veřejný profil prodávajícího.....	64
5.3.10 Administrátorské rozhraní	65
5.3.11 Stylování a responzivita.....	66
5.4 Implementace databázové vrstvy	67
5.4.1 Prisma schema	68
5.4.2 Hlavní entity systému	68
5.4.3 Vazby mezi entitami	69
5.4.4 Transakční data a auditovatelnost.....	70
5.4.5 Migrace a inicializace databáze	70
5.4.6 Indexy a optimalizace dotazů	71
5.5 Realtime komunikace	71
5.5.1 Socket.IO gateway	72
5.5.2 Události chatu	72
5.5.3 Online stav a inbox	72
5.5.4 Aktualizace stavu obchodu	73
5.6 Docker a spuštění aplikace	73
5.6.1 Docker Compose.....	74
5.6.2 Konfigurace prostředí	74
5.6.3 Lokální spuštění	75
6 Testování.....	76
6.1 Cíle testování	76
6.2 Backendové testování	76
6.3 Frontendové testování.....	79
6.4 Testování realtime komunikace	80
6.5 Testování bezpečnosti.....	80
6.6 Uživatelské testování	82

6.7 Shrnutí testování	82
6.8 Omezení testování.....	83
ZÁVĚR	84
POUŽITÁ LITERATURA	86
SEZNAM PŘÍLOH.....	88

SEZNAM ILUSTRACÍ A TABULEK

Obrázek 1: Diagram případů užití systému (vlastní zpracování)	30
Obrázek 2: Architektura aplikace a komunikace mezi částmi systému (vlastní zpracování)...	32
Obrázek 3: Zjednodušená struktura projektu (vlastní zpracování)	37
Obrázek 4: Struktura backendové části systému (vlastní zpracování)	39
Obrázek 5: Struktura frontendové části systému (vlastní zpracování)	39
Obrázek 6: Struktura NestJS modulů (vlastní zpracování).....	41
Obrázek 7: Příklad environment proměnných v souboru .env.example (vlastní zpracování) ..	42
Obrázek 8: Zjednodušený průběh autentizace uživatele v systému (vlastní zpracování).....	43
Obrázek 9: Implementace JWT strategie (vlastní zpracování)	44
Obrázek 10: Validací DTO pro úpravu uživatelského profilu (vlastní zpracování).....	45
Obrázek 11: Načtení minimální ceny nabídky za posledních 30 dní (vlastní zpracování).....	46
Obrázek 12: Životní cyklus obchodu s využitím escrow mechanismu (vlastní zpracování)....	47
Obrázek 13: Databázové transakce při dokončení obchodu (vlastní zpracování)	48
Obrázek 14: Práce s ledger transakcemi v modulu peněženky (vlastní zpracování).....	49
Obrázek 15: Realtime komunikace pomocí Socket.IO gateway (vlastní zpracování)	50
Obrázek 16: Validace recenzí a aktualizace reputace uživatele (vlastní zpracování)	51
Obrázek 17: Zpracování reportu v administrátorském systému (vlastní zpracování)	52
Obrázek 18: Kontrola administrátorských oprávnění pomocí AdminGuard (vlastní zpracování).....	52
Obrázek 19: Validace nahrávaných souborů při uploadu médií (vlastní zpracování).....	53
Obrázek 20: Struktura frontendové části React aplikace (vlastní zpracování)	55
Obrázek 21: Implementace routingu a chráněných stránek ve frontendové části (vlastní zpracování).....	56
Obrázek 22: Přihlašovací obrazovka aplikace (vlastní zpracování)	57
Obrázek 23: Registrační obrazovka aplikace (vlastní zpracování).....	58
Obrázek 24: Katalog nabídek marketplace aplikace (vlastní zpracování).....	59
Obrázek 25: Detail nabídky s informacemi o produktu a prodávajícím (vlastní zpracování) ..	60
Obrázek 26: Formulář pro vytvoření nebo úpravu nabídky (vlastní zpracování).....	61
Obrázek 27: Detail obchodu a escrow procesu (vlastní zpracování).....	62
Obrázek 28: Rozhraní interní peněženky uživatele (vlastní zpracování)	63
Obrázek 29: Inbox a realtime chat mezi uživateli (vlastní zpracování)	64
Obrázek 30: Veřejný profil prodávajícího (vlastní zpracování)	65
Obrázek 31: Administrátorské rozhraní marketplace aplikace (vlastní zpracování)	66
Obrázek 32: Responzivní zobrazení frontendové části aplikace (vlastní zpracování)	67
Obrázek 33: ER diagram hlavních entit systému (vlastní zpracování).....	69
Obrázek 34: Princip realtime komunikace mezi frontendem a backendem (vlastní zpracování)	71
Obrázek 35: Realtime aktualizace stavu obchodu pomocí Socket.IO (vlastní zpracování)	73
Obrázek 36: Kontejnerizovaná architektura aplikace pomocí Docker Compose (vlastní zpracování).....	74
Obrázek 37: Unit test escrow operace ve wallet service pomocí frameworku Jest (vlastní zpracování).....	77

Obrázek 38: End-to-end test autentizačního endpointu backendového API (vlastní zpracování)	78
Obrázek 39: Odepření přístupu k chráněnému endpointu bez autentizace (vlastní zpracování)	81
Obrázek 40: Ochrana administrátorského endpointu pomocí role-based autorizace (vlastní zpracování)	81
Obrázek 41: Automatizované spuštění testů a build procesu pomocí GitHub Actions (vlastní zpracování)	83
Tabulka 1: Srovnání architektur webových aplikací	19
Tabulka 2: Srovnání existujících marketplace platforem	25
Tabulka 3: Funkční požadavky systému	28
Tabulka 4: Nefunkční požadavky systému	29
Tabulka 5: Srovnání zvolených technologií a alternativ	33
Tabulka 6: Hlavní backendové knihovny a jejich využití	41
Tabulka 7: Přehled vybraných testovacích scénářů backendové části	78

SEZNAM ZKRATEK A ZNAČEK

API (Application Programming Interface) – Rozhraní pro programování aplikací, které umožňuje komunikaci mezi frontendovou a backendovou částí systému.

ACID (Atomicity, Consistency, Isolation, Durability) – Sada vlastností databázových transakcí zaručující spolehlivé zpracování dat.

CORS (Cross-Origin Resource Sharing) – Mechanismus pro zabezpečení požadavků mezi různými doménami.

CSS (Cascading Style Sheets) – Kaskádové styly používané pro definici vzhledu uživatelského rozhraní.

DOM (Document Object Model) – Objektový model dokumentu, který reprezentuje strukturu webové stránky.

DTO (Data Transfer Object) – Objekt sloužící k přenosu a validaci dat mezi klientem a serverem.

HTTP/HTTPS (Hypertext Transfer Protocol / Secure) – Protokol pro přenos webových stránek a dat, v zabezpečené verzi využívající šifrování.

JSON (JavaScript Object Notation) – Lehký formát pro výměnu dat, využívaný pro komunikaci v rámci REST API.

JWT (JSON Web Token) – Standard pro bezpečné předávání informací mezi stranami jako objekt JSON, využíváný pro bezstavovou autentizaci.

MVC (Model-View-Controller) – Architektonický vzor rozdělující aplikaci na datový model, uživatelské rozhraní a řídicí logiku.

MVVM (Model-View-ViewModel) – Architektonický vzor optimalizovaný pro moderní frontendové frameworky.

ORM (Object-Relational Mapping) – Technika umožňující práci s relační databází pomocí objektově orientovaného programování (v práci nástroj Prisma).

REST (Representational State Transfer) – Architektonický styl pro navrhování síťových aplikací využívající HTTP metody.

SPA (Single Page Application) – Webová aplikace, která se načte jednou a dynamicky aktualizuje svůj obsah bez obnovování celé stránky.

SQL (Structured Query Language) – Standardizovaný dotazovací jazyk pro práci s relačními databázemi.

UI (User Interface) – Uživatelské rozhraní, část aplikace, se kterou uživatel přímo interaguje.

UX (User Experience) – Uživatelský zážitek, zaměřený na celkovou přívětivost a použitelnost aplikace.

TERMINOLOGIE

Backend – Serverová část aplikace, která implementuje business logiku, zajišťuje autentizaci a komunikuje s databází.

Escrow – Finanční mechanismus zajišťující bezpečnost transakce tím, že prostředky jsou dočasně blokovány systémem a uvolněny prodejci až po potvrzení doručení produktu kupujícím.

Frontend – Klientská část aplikace běžící v prohlížeči uživatele, zodpovědná za prezentaci dat a interakci.

Fullstack – Přístup k vývoji, který zahrnuje implementaci klientské i serverové části aplikace současně.

Ledger – Hlavní účetní kniha (historie transakcí), která zaznamenává každou změnu zůstatku v peněženke pro zajištění auditovatelnosti systému.

Marketplace – Webová platforma sloužící jako zprostředkovatel obchodů mezi nezávislými prodejci a kupujícími.

Modulární monolit – Architektura, kde je aplikace nasazena jako jeden celek, ale vnitřně je logicky rozdělena do samostatných doménových modulů.

PostgreSQL – Objektově-relační databázový systém použitý pro perzistentní ukládání dat v aplikaci.

Prisma – Moderní ORM nástroj pro Node.js, který zajišťuje typově bezpečný přístup k databázi.

Realtime komunikace – Okamžitý přenos dat mezi serverem a klientem bez nutnosti obnovování stránky, v práci realizovaný pomocí technologie WebSockets a knihovny Socket.IO.

Vnitřní peněženka (Wallet) – Modul pro správu virtuálního finančního zůstatku uživatele v rámci platformy.

WebSocket – Komunikační protokol umožňující obousměrnou realtime komunikaci přes jedno TCP spojení.

ÚVOD

Digitální produkty a služby tvoří významnou část současného online obchodování. Na rozdíl od fyzického zboží nejde vždy o předmět, který lze jednoduše odeslat běžnou dopravou. Uživatel často kupuje přístupové údaje, digitální soubor, službu, herní položku, software nebo jiný nehmotný výstup. U tohoto typu obchodů proto nestačí pouze zobrazit katalog nabídek. Platforma musí řešit také důvěru mezi kupujícím a prodávajícím, komunikaci, ochranu transakce a evidenci finančních operací.

Téma práce bylo zvoleno z důvodu rostoucího významu webových platforem, které zprostředkovávají obchod mezi uživateli. Marketplace aplikace pro digitální produkty představuje systém, ve kterém se propojuje návrh uživatelského rozhraní, backendová business logika, databázový model, autentizace, realtime komunikace a bezpečné zpracování navazujících operací. Právě propojení těchto částí činí téma vhodným pro prakticky zaměřenou bakalářskou práci.

Cílem této bakalářské práce je navrhnout a implementovat webovou marketplace platformu pro digitální produkty a služby. Aplikace má pokrývat celý průběh obchodu mezi kupujícím a prodávajícím, tedy registraci a přihlášení uživatele, správu nabídek, procházení katalogu, komunikaci mezi uživateli, vytvoření obchodu, práci s interní peněženkou, escrow mechanismus, recenze, reportování obsahu a administrátorské rozhraní.

Při zpracování práce je nejprve provedena analýza existujících marketplace platforem. Na jejím základě jsou definovány funkční a nefunkční požadavky a případy užití vlastní aplikace. Následně je navržena architektura systému, zvolen technologický stack, popsána databázová vrstva a komunikace mezi jednotlivými částmi aplikace. Praktická část práce se věnuje implementaci backendu, frontendu, databázové vrstvy, realtime komunikace, kontejnerizovaného spuštění a ověření hlavních funkcí systému.

Hlavní část obchodního procesu je založena na escrow principu. Při vytvoření obchodu jsou prostředky kupujícího nejprve zablokovány v interní peněžence a prodávajícímu se uvolní až po dokončení transakce. Každá změna zůstatku je evidována pomocí ledger záznamu. Práce se nezaměřuje na napojení skutečné platební brány; finanční část je navržena jako demonstrační model interních operací.

Aplikace je navržena jako modulární monolit. Backendová část je implementována ve frameworku NestJS, frontendová část využívá React a TypeScript, databázová vrstva je

postavena na PostgreSQL a Prisma ORM. Pro realtime komunikaci je použita knihovna Socket.IO.

První kapitola shrnuje základní principy webových aplikací a použitých technologií. Druhá kapitola se věnuje analýze existujících marketplace platforem. Třetí kapitola definuje požadavky na systém a případy užití. Čtvrtá kapitola popisuje návrh systému. Pátá kapitola se zabývá implementací jednotlivých částí aplikace a šestá kapitola popisuje testování vytvořeného řešení.

1 Webové aplikace

Webová aplikace je softwarový systém dostupný prostřednictvím webového prohlížeče. Uživatel nemusí instalovat samostatnou aplikaci, protože klientská část je načítána v prohlížeči a serverová část zajišťuje zpracování požadavků, aplikační logiku a práci s daty.

Základním principem webových aplikací je model klient-server. Klient odesílá požadavky pomocí protokolu HTTP nebo HTTPS a server na ně odpovídá po provedení potřebných operací [13]. V moderních aplikacích se data mezi klientem a serverem často přenášejí ve formátu JSON, který je vhodný pro komunikaci mezi frontendovou a backendovou částí systému [14].

Současné webové aplikace často využívají přístup Single Page Application. Při tomto modelu je aplikace načtena jednou a další změny uživatelského rozhraní probíhají bez úplného obnovení stránky [9]. Díky tomu je práce s aplikací plynulejší, což je vhodné zejména pro systémy s větším množstvím interakcí, například administrace, chatovací aplikace nebo marketplace platformy.

Výhodou webových aplikací je dostupnost z různých zařízení, centralizovaná správa dat a jednodušší aktualizace systému. Změny se nasazují na serverovou část systému, takže uživatel nemusí ručně instalovat nové verze aplikace. Nevýhodou může být závislost na internetovém připojení, nutnost řešit bezpečnost komunikace a vyšší nároky na návrh serverové části [4].

Pro marketplace platformu je webová aplikace vhodným řešením, protože umožňuje propojit veřejný katalog, uživatelské účty, komunikaci mezi uživateli a transakční logiku v jednom prostředí. V rámci této práce je tento princip využit při návrhu aplikace, která kombinuje REST API pro běžné operace a realtime komunikaci pro chat, inbox a změny stavu obchodů [4], [11], [15].

1.1 Architektura webových aplikací

Architektura webové aplikace určuje strukturu systému, rozdělení jednotlivých částí a způsob jejich komunikace. Návrh architektury ovlivňuje udržitelnost systému, možnosti rozšiřování, výkon aplikace i složitost vývoje [1].

V praxi se používá několik přístupů k organizaci backendové části systému. Nejčastější jsou monolitická architektura, mikroslužby a modulární monolit [1], [3]. Jednotlivé přístupy se liší především způsobem rozdělení aplikační logiky, komunikací mezi částmi systému a náročností infrastruktury.

Volba vhodné architektury závisí na velikosti projektu, požadavcích na škálovatelnost a složitosti obchodní logiky. Správné oddělení odpovědností je důležité pro dlouhodobou udržitelnost systému, jak uvádí Fowler ve své práci zaměřené na enterprise architekturu [1].

1.1.1 Monolitická architektura

Monolitická architektura představuje přístup, při kterém je celá aplikace provozována jako jeden celek. Všechny části systému sdílejí jednu kódovou bázi a běží v rámci jedné služby [1].

Komunikace mezi jednotlivými vrstvami probíhá interně bez síťové komunikace, což zjednodušuje vývoj i nasazení aplikace. Monolitická architektura je proto vhodná zejména pro menší projekty nebo první verze systému.

Nevýhodou je postupné zvyšování složitosti při růstu aplikace. S rostoucím množstvím funkcí bývá obtížnější oddělovat jednotlivé části systému a změny v jedné oblasti mohou ovlivnit ostatní části aplikace [1].

1.1.2 Mikroslužbová architektura

Mikroslužbová architektura rozděluje aplikaci na více samostatných služeb, které řeší konkrétní část systému. Každá služba může být vyvíjena, nasazována a škálována nezávisle na ostatních [3].

Jednotlivé služby spolu komunikují přes síť, nejčastěji pomocí REST API nebo asynchronních zpráv [3], [4]. Mikroslužby se proto uplatňují hlavně u rozsáhlých systémů, které potřebují samostatně škálovat jednotlivé části.

Nevýhodou je vyšší infrastrukturní a vývojová složitost. Je nutné řešit komunikaci mezi službami, monitoring, správu nasazení a konzistenci dat mezi jednotlivými částmi systému [3].

1.1.3 Modulární monolit

Modulární monolit kombinuje jednoduchost monolitické architektury s logickým rozdělením systému do samostatných modulů [1], [5]. Aplikace je nasazena jako jeden celek, ale její vnitřní struktura je rozdělena podle doménových oblastí.

Každý modul má vlastní odpovědnost a obsahuje oddělenou aplikační logiku. Komunikace mezi moduly probíhá interně bez potřeby síťové komunikace mezi službami.

Výhodou tohoto přístupu je jednodušší nasazení a současně lepší přehlednost kódu oproti klasickému monolitu. Modulární monolit zároveň umožňuje využívat databázové transakce napříč více částmi systému, což je důležité například při implementaci obchodní a finanční logiky marketplace platformy [6], [7].

1.1.4 Zhodnocení architektur

Volba architektury závisí na charakteru projektu a požadavcích na provoz systému. Monolitická architektura je vhodná pro menší aplikace a rychlý vývoj. Mikroslužby se využívají především u rozsáhlých systémů s vysokými nároky na škálování a nezávislý vývoj jednotlivých částí [3].

Modulární monolit představuje kompromis mezi těmito přístupy. Zachovává jednodušší infrastrukturu než mikroslužby a současně poskytuje lepší organizaci systému než klasický monolit [1], [5]. Tento přístup je proto vhodný pro aplikace středního rozsahu s komplexnější business logikou.

Tabulka 1: Srovnání architektur webových aplikací

Kritérium	Monolitická architektura	Mikroslužbová architektura	Modulární monolit
Struktura	Jedna aplikace	Více služeb	Moduly v jednom celku
Složitost	Nízká	Vysoká	Střední
Škálovatelnost	Omezená	Vysoká	Střední

Nasazení	Jednoduché	Komplexní	Jednodušší
Komunikace	Interní	Síťová	Interní
Odolnost	Nízká	Vysoká	Střední
Vhodnost	Menší projekty	Velké systémy	Střední projekty

Zdroj: (vlastní zpracování podle [1], [3])

1.2 Architektonické vzory

Architektonické vzory určují způsob organizace aplikační logiky a rozdělení odpovědností mezi jednotlivé části systému [1], [5]. Cílem těchto vzorů je oddělit práci s daty, business logiku a uživatelské rozhraní tak, aby byl systém přehlednější a snadněji udržovatelný.

Ve webových aplikacích se často využívají vzory MVC a MVVM. Oba přístupy řeší oddělení jednotlivých vrstev aplikace, ale liší se způsobem práce s uživatelským rozhraním a stavem aplikace.

1.2.1 MVC

Model-View-Controller (MVC) rozděluje aplikaci na tři základní části: model, view a controller [1].

Model představuje datovou a aplikační vrstvu systému. Zajišťuje práci s daty, validaci a implementaci business logiky. View slouží k prezentaci dat uživateli a controller přijímá požadavky, zpracovává vstupy a komunikuje s modelem.

Oddělení jednotlivých vrstev zjednodušuje údržbu aplikace a umožňuje lépe organizovat zdrojový kód. MVC je proto často využíváno v backendových frameworkech, které pracují s HTTP požadavky a API vrstvou [1], [4].

1.2.2 MVVM

Model-View-ViewModel (MVVM) rozšiřuje principy MVC o vrstvu ViewModel, která zajišťuje propojení mezi daty a uživatelským rozhraním.

ViewModel připravuje data pro zobrazení a reaguje na změny stavu aplikace. Díky tomu může být uživatelské rozhraní automaticky aktualizováno bez přímé manipulace s jednotlivými prvky stránky.

Tento přístup je vhodný zejména pro frontendové aplikace s větším množstvím interaktivních prvků a dynamických dat. Moderní frontendové frameworky často využívají komponentový přístup, při kterém je rozhraní rozděleno na samostatné znovupoužitelné části [9], [21], [22].

1.3 Technologie pro vývoj webových aplikací

Vývoj webových aplikací využívá kombinaci technologií tvořících frontendovou, backendovou a databázovou vrstvu systému. Jednotlivé části spolu komunikují prostřednictvím definovaných rozhraní, nejčastěji pomocí REST API nebo realtime komunikace [4], [11].

Zvolený technologický stack ovlivňuje způsob vývoje aplikace, možnosti rozšiřování systému i náročnost následné údržby. Moderní webové systémy proto často využívají frameworky a

nástroje, které zjednodušují správu aplikační logiky, práci s daty a komunikaci mezi jednotlivými částmi systému.

1.3.1 Backendové technologie

Backend představuje serverovou část aplikace, která zajišťuje zpracování požadavků, implementaci business logiky a komunikaci s databází. Zároveň odpovídá za autentizaci uživatelů, autorizaci přístupu, validaci vstupních dat a správu aplikační logiky [30], [31].

Komunikace mezi frontendem a backendem probíhá nejčastěji pomocí REST API, které využívá HTTP metody pro práci se zdroji systému [4], [13]. Data jsou obvykle přenášena ve formátu JSON [14].

Součástí backendové vrstvy bývá také práce s databázovými transakcemi, zpracování oprávnění uživatelů nebo realtime komunikace pomocí WebSocket technologií [6], [11], [15].

Pro implementaci backendu existuje více frameworků a technologií, například Node.js, Spring Boot, Django nebo Laravel [18], [19], [20]. Tyto nástroje poskytují prostředky pro tvorbu API, správu aplikační logiky a komunikaci s databází.

1.3.2 Frontendové technologie

Frontend představuje klientskou část aplikace běžící v prohlížeči uživatele. Zajišťuje zobrazení dat, práci s uživatelským rozhraním a komunikaci se serverovou částí systému.

Moderní frontendové aplikace často využívají frameworky založené na komponentové architektuře [9], [21], [22]. Uživatelské rozhraní je rozděleno na menší samostatné části, které lze opakovaně využívat napříč aplikací.

Současné frontendové frameworky zároveň umožňují efektivní správu stavu aplikace a dynamickou aktualizaci rozhraní bez nutnosti úplného obnovení stránky. Tento přístup je důležitý zejména u interaktivních systémů pracujících s realtime daty nebo větším množstvím uživatelských akcí [9].

Frontend komunikuje s backendem pomocí API a zajišťuje synchronizaci dat mezi uživatelským rozhraním a serverovou částí aplikace. Při návrhu frontendových aplikací je důležitá také přehlednost rozhraní a jednoduchost ovládání, což odpovídá principům použitelnosti popsaným Krugem [2].

1.3.3 Databázové systémy

Databázové systémy slouží k ukládání a správě dat aplikace. Umožňují efektivní práci s daty, jejich vyhledávání, úpravu a zachování konzistence systému [6].

Ve webových aplikacích se používají především relační a NoSQL databáze [6]. Relační databáze ukládají data do tabulek propojených pomocí vztahů a jsou vhodné pro systémy, kde je důležitá konzistence dat a jasně definovaná struktura [7], [23].

NoSQL databáze ukládají data flexibilnějším způsobem, například jako dokumenty nebo dvojice klíč-hodnota [24]. Tento přístup je vhodný zejména pro systémy s velmi vysokým objemem dat nebo proměnlivou strukturou dat.

Pro aplikace pracující s uživatelskými účty, transakcemi a vztahy mezi entitami bývají často vhodnější relační databáze [6], [7]. Návrh databázové vrstvy stojí především na vhodně zvoleném datovém modelu, který určuje strukturu tabulek a vztahy mezi nimi [5].

Při práci s databází se často využívá ORM (Object-Relational Mapping), které umožňuje pracovat s databázovými daty pomocí objektového modelu místo přímého psaní SQL dotazů [10], [25].

1.3.4 Transakce a konzistence dat

Při práci s databází je důležité zajistit konzistenci dat i při současném zpracování více požadavků. K tomu slouží databázové transakce, které umožňují provést více operací jako jeden celek [6], [7].

Pokud během transakce dojde k chybě, změny se neuloží a databáze se vrátí do původního stavu. Transakce tak pomáhají předcházet nekonzistentním stavům systému.

Databázové transakce splňují vlastnosti označované jako ACID (Atomicity, Consistency, Isolation, Durability), které zajišťují spolehlivé zpracování dat i při souběžném přístupu více uživatelů [6], [7].

Tyto principy jsou důležité zejména u systémů pracujících s finančními operacemi nebo více navazujícími kroky. V marketplace aplikacích mohou být transakce využity například při práci s interní peněženkou, escrow mechanismem nebo změnou stavu obchodů.

1.3.5 Realtime komunikace

Realtime komunikace slouží k okamžité synchronizaci dat mezi klientem a serverem bez nutnosti opakovaného odesílání HTTP požadavků [15].

Tento způsob komunikace bývá realizován pomocí technologie WebSocket nebo knihoven, které nad ní poskytují vyšší úroveň abstrakce, například Socket.IO [11], [15]. Server tak může aktivně odesílat informace klientovi okamžitě po změně stavu systému.

Realtime komunikace se využívá zejména u chatovacích systémů, notifikací nebo synchronizace změn mezi více uživateli. V marketplace platformách může sloužit například pro realtime chat, aktualizaci inboxu nebo změny stavu obchodů.

Nevýhodou je vyšší složitost implementace a správy aktivních spojení. Z tohoto důvodu se realtime komunikace často kombinuje s REST API, které zajišťuje běžnou práci s daty [4], [11].

1.3.6 Bezpečnost webových aplikací

Bezpečnost webových aplikací zajišťuje ochranu dat, správnou správu uživatelských oprávnění a bezpečnou komunikaci mezi klientem a serverem [30].

Základními prvky jsou autentizace a autorizace uživatelů. Autentizace ověřuje identitu uživatele a autorizace určuje, k jakým částem systému má uživatel přístup [30], [31].

Vstupní data je nutné validovat ještě před jejich zpracováním. Validace pomáhá předcházet zpracování chybných nebo škodlivých požadavků [30]. Pro zabezpečení komunikace se běžně využívá protokol HTTPS, který zajišťuje šifrování přenášených dat [13].

Součástí bezpečnostních opatření je také bezpečné ukládání hesel pomocí hashovacích algoritmů a kontrola přístupu k datům jednotlivých uživatelů [30], [31], [32].

1.3.7 Moderní nástroje pro vývoj

Součástí moderního vývoje webových aplikací jsou také nástroje usnadňující správu projektu, testování a nasazení systému.

Verzovací systémy, například Git, umožňují sledovat změny v kódu a spolupráci více vývojářů. Pro zajištění konzistentního prostředí se často využívá kontejnerizace pomocí nástrojů jako Docker [12].

Vývoj moderních aplikací bývá doplněn také o automatizované testování nebo CI/CD procesy, které pomáhají zjednodušit nasazení a omezit riziko chyb při vývoji systému.

2 Analýza existujících řešení

Před návrhem vlastního systému byla provedena analýza existujících marketplace platform zaměřených na digitální produkty a služby. Cílem analýzy nebylo pouze popsat jejich funkce, ale především určit, jakým způsobem tyto platformy řeší obchodní proces mezi kupujícím a prodávajícím, důvěryhodnost prodejců, ochranu transakcí, komunikaci mezi uživateli a přehlednost uživatelského rozhraní [1], [2].

Pro srovnání byly zvoleny platformy FunPay, G2A, GGsel a Plati.Market [26], [27], [28], [29]. Tyto systémy pracují s podobným typem digitálního obsahu, například herními účty, digitálními klíči, softwarem, předplatnými, herní měnou nebo službami. Zároveň se liší rozsahem katalogu, cílovou skupinou, mírou zapojení platformy do obchodního procesu a způsobem ochrany kupujícího i prodávajícího.

Výsledky analýzy byly následně využity při návrhu vlastní aplikace. Z existujících řešení byly převzaty především obecné principy, například katalog nabídek, hodnocení prodejců, přímá komunikace a ochrana transakce. Současně však bylo cílem navrhnout jednodušší a přehlednější systém, který tyto prvky propojuje v rámci jedné fullstack aplikace.

2.1 FunPay

FunPay je marketplace zaměřený především na hráče a digitální herní obsah [26]. Systém podporuje prodej herní měny, účtů, předmětů, skinů a herních služeb. Významnou součástí systému je přímé propojení kupujícího a prodávajícího v rámci jedné platformy. FunPay zároveň využívá princip bezpečné transakce, kdy platba není uvolněna prodávajícímu, dokud kupující nepotvrdí přijetí produktu nebo služby.

Z hlediska návrhu vlastního systému je FunPay důležitým příkladem propojení katalogu nabídek, přímé komunikace a ochrany transakce. Takový přístup je vhodný zejména pro služby, u nichž není možné vždy zajistit automatické doručení produktu a je nutná komunikace mezi oběma stranami [6].

Nevýhodou může být výrazné zaměření na herní segment, které omezuje univerzálnost platformy pro jiné typy digitálních produktů. Uživatelské rozhraní je navrženo především pro rychlé vyhledání konkrétní herní kategorie.

2.2 G2A

G2A představuje rozsáhlý globální marketplace zaměřený na digitální produkty [27]. Nabídka zahrnuje digitální klíče, hry, DLC, dárkové karty, software, předplatná nebo e-learningové produkty. Platforma je zaměřena na velký rozsah katalogu, mezinárodní působnost a spolupráci s ověřenými prodejci.

Výhodou systému je profesionální zpracování, rozsáhlý katalog a rozvinuté nástroje pro prodejce. G2A podporuje prodej digitálních produktů ve velkém rozsahu a je vhodná zejména pro uživatele hledající rychlý nákup digitálních klíčů nebo kódů. Součástí systému jsou také recenze, hodnocení prodejců a zákaznická podpora, které pomáhají budovat důvěryhodnost platformy [2].

Nevýhodou z pohledu menší marketplace aplikace je vysoká komplexita systému. Přímá komunikace mezi kupujícím a prodávajícím zde není hlavním prvkem obchodního procesu. Pro

návrh vlastního řešení je proto vhodné převzít důraz na bezpečnost a správu katalogu, ale zachovat jednodušší a přehlednější průběh nákupu.

2.3 GGsel

GGsel je marketplace zaměřený na digitální herní produkty, například herní klíče, účty, software, předplatná nebo herní položky [28]. Platforma pracuje s nabídkami od jednotlivých prodejců a zobrazuje jejich hodnocení, počet prodejů i cenu jednotlivých produktů.

GGsel kombinuje katalog nabídek, hodnocení prodejců a ochranu kupujícího pomocí rezervace platby. Důvěryhodnost systému je založena nejen na samotné platformě, ale také na historii prodejce a uživatelských recenzích.

Nevýhodou může být menší důraz na moderní uživatelské rozhraní a celkovou přehlednost systému. Pro návrh vlastní aplikace je proto vhodné zachovat princip hodnocení a bezpečné transakce, ale zjednodušit práci s katalogem, správou objednávek a komunikací mezi uživateli [2].

2.4 Plati.Market

Plati.Market je marketplace zaměřený na prodej digitálních produktů [29]. Platforma podporuje prodej her, klíčů, účtů, předplatných, softwaru i herní měny. Marketplace zároveň podporuje publikování různých typů digitálního obsahu od jednotlivých prodejců.

Výhodou systému je široká nabídka digitálního obsahu a dlouhodobě zavedený marketplace model. Platforma poskytuje prodej mnoha typů digitálních položek, což je důležité také pro návrh univerzálnější marketplace aplikace.

Nevýhodou je méně moderní uživatelské rozhraní a slabší důraz na komfort používání. Uživatel musí pracovat s velkým množstvím kategorií a nabídek, což může zhoršovat orientaci v systému. Z hlediska návrhu vlastního řešení je proto důležité zaměřit se nejen na funkčnost, ale také na jednoduchost a přehlednost uživatelského rozhraní [2].

2.5 Srovnání existujících řešení

Srovnávané platformy mají společný základ v podobě prodeje digitálního obsahu mezi uživateli nebo ověřenými prodejci. Liší se však cílovou skupinou, rozsahem katalogu, důrazem na komunikaci a způsobem ochrany transakcí.

Tabulka 2: Srovnání existujících marketplace platform

Kritérium	FunPay	G2A	GGsel	Plati.Market
Hlavní zaměření	Herní služby a obsah	Digitální klíče a zboží	Herní digitální zboží	Digitální produkty
Typické produkty	Účty, měna, služby	Klíče, DLC, software	Klíče, účty, software	Klíče, účty, software
Přímá komunikace	Výrazná	Spíše omezená	Částečná	Částečná

Ochrana transakce	Escrow princip	Ověření prodejců	Rezervace platby	Ochrana kupujícího
Rozhraní	Jednoduché	Komplexní	Spíše jednoduché	Méně moderní
Vhodné prvky pro návrh	Chat, escrow	Katalog, důvěryhodnost	Hodnocení, escrow	Široká nabídka
Hlavní omezení	Herní zaměření	Vysoká komplexita	Slabší UX	Slabší UX

Zdroj: (vlastní zpracování na základě [26], [27], [28], [29])

Z porovnání vyplývá, že analyzované platformy řeší podobný typ problému, ale každá z nich klade důraz na jinou oblast. FunPay je důležitý zejména propojením katalogu, přímé komunikace a bezpečnějšího průběhu transakce. Tento přístup je vhodný pro digitální služby, u kterých nelze vždy zajistit plně automatické doručení a kde je nutná dohoda mezi kupujícím a prodávajícím.

G2A ukazuje význam rozsáhlého katalogu, profesionální prezentace produktů a důvěryhodnosti prodejců. Pro návrh vlastní aplikace je zde důležitý především princip práce s velkým množstvím nabídek, kategorií a prodejců. Nevýhodou podobně rozsáhlých platforem však může být vyšší komplexita rozhraní, která zvyšuje nároky na orientaci uživatele.

GGsel a Plati.Market představují příklady platforem zaměřených na digitální zboží, kde je důležitá rychlost nákupu, hodnocení prodejců a ochrana kupujícího. Z pohledu návrhu vlastní aplikace jsou tyto systémy přínosné zejména tím, že ukazují význam reputace, historie obchodů a jednoduchého průchodu nákupním procesem.

Z analýzy vyplývá, že marketplace pro digitální produkty nemůže fungovat pouze jako katalog nabídek. Musí pracovat také s důvěrou mezi uživateli, bezpečnějším průběhem transakce, přímou komunikací a moderací problematického obsahu.

2.6 Dopady analýzy na návrh vlastního systému

Výsledky analýzy se promítly do několika částí návrhu vlastní aplikace. Nejprve šlo o katalog nabídek. Srovnávané platformy ukazují, že u digitálních produktů je důležité umožnit uživateli rychlé vyhledání konkrétního typu obsahu, práci s kategoriemi, filtrování a zobrazení důležitých informací o produktu i prodávajícím. Z tohoto důvodu vlastní aplikace obsahuje katalog nabídek, detail nabídky, filtrování, řazení a veřejný profil prodávajícího.

Druhou oblastí byla ochrana obchodního procesu. U digitálních produktů a služeb často neexistuje fyzické doručení zboží a mezi kupujícím a prodávajícím může vzniknout spor o splnění domluvené služby. Z tohoto důvodu byl ve vlastní aplikaci navržen escrow mechanismus, při kterém jsou prostředky kupujícího nejprve zablokovány a prodávajícímu jsou uvolněny až po dokončení obchodu. Tento princip byl v projektu doplněn o interní peněženku a ledger záznamy, aby bylo možné zpětně dohledat historii finančních operací.

Třetí oblastí byla komunikace mezi uživateli. U služeb, herních účtů nebo jiného digitálního obsahu je často nutné, aby se kupující a prodávající mohli před dokončením obchodu domluvit na detailech. Proto byla do aplikace zahrnuta realtime komunikace pomocí chatu, inboxu a

websocketových událostí. Realtime komunikace je využita také pro aktualizaci stavu obchodů a systémové notifikace.

Čtvrtou oblastí byla důvěryhodnost a moderace. Analyzované platformy ukazují, že u marketplace systému nestačí pouze umožnit publikování nabídek. Je nutné řešit také reputaci prodejců, recenze, reportování problematického obsahu a administrátorské zásahy. Vlastní aplikace proto obsahuje hodnocení po dokončení obchodu, veřejný profil prodávajícího, reporty, administrátorské rozhraní, systém varování a auditní záznamy důležitých administrátorských akcí.

Na základě analýzy bylo rozhodnuto nevytvářet pouze jednoduchý katalog digitálních produktů, ale navrhnout systém, který propojuje nabídky, komunikaci, obchodní workflow, escrow logiku, interní finanční evidenci a moderaci. Tím se vlastní řešení odlišuje od jednodušších katalogových aplikací a lépe odpovídá požadavkům marketplace platformy pro digitální produkty a služby.

3 Analýza požadavků

Cílem této práce je návrh a implementace webové platformy typu marketplace pro digitální produkty a služby. Marketplace platforma kombinuje prvky katalogového systému, komunikaci mezi uživateli a zpracování interních transakcí [1], [6]. Z tohoto důvodu bylo při návrhu požadavků nutné zohlednit nejen běžné operace nad obsahem, ale také bezpečnost systému, realtime komunikaci a konzistenci dat při zpracování obchodů [6], [30].

Na základě analýzy existujících řešení a požadavků na podobné systémy byly definovány funkční a nefunkční požadavky, které musí navržená aplikace splňovat.

Funkční požadavky popisují konkrétní vlastnosti a chování systému. Nefunkční požadavky se zaměřují na kvalitu systému, například výkon, bezpečnost nebo škálovatelnost [1], [6]. Na základě těchto požadavků byl zvolen technologický stack a architektura systému.

3.1 Funkční požadavky

Funkční požadavky popisují hlavní operace, které mají být v aplikaci dostupné pro návštěvníka, přihlášeného uživatele, prodávajícího a administrátora [1].

Tabulka 3: Funkční požadavky systému

ID	Požadavek	Popis
F1	Registrace uživatele	Uživatel se může zaregistrovat pomocí e-mailu a hesla
F2	Autentizace uživatele	Uživatel se může přihlásit do systému
F3	Správa profilu	Uživatel může upravovat své údaje
F4	Vytváření nabídek	Uživatel může vytvářet nabídky produktů nebo služeb
F5	Správa nabídek	Uživatel může upravovat nebo mazat své nabídky
F6	Vyhledávání	Uživatel může vyhledávat a filtrovat nabídky
F7	Zobrazení detailu nabídky	Systém zobrazí detail produktu nebo služby
F8	Správa objednávek	Uživatel může vytvářet a spravovat objednávky
F9	Interní transakce	Systém zpracuje interní transakci mezi uživateli pomocí peněženky a escrow mechanismu
F10	Realtime komunikace	Uživatelé mohou komunikovat v reálném čase
F11	Správa stavu objednávky	Systém eviduje stav objednávky
F12	Hodnocení	Uživatel může hodnotit transakci
F13	Ověření e-mailu	Systém umožňuje ověření účtu pomocí e-mailového odkazu
F14	Obnova hesla	Uživatel může požádat o reset hesla
F15	Nahrávání médií	Uživatel může nahrávat obrázky a přílohy

F16	Reportování obsahu	Uživatel může nahlásit nabídku, zprávu nebo uživatele
F17	Administrace	Administrátor může moderovat obsah a uživatele
F18	Systémové notifikace	Systém zobrazuje systémové notifikace a umožňuje jejich realtime aktualizaci
F19	Motivační systém	Systém podporuje motivační a reputační prvky pro uživatele

Zdroj: (vlastní zpracování)

3.2 Nefunkční požadavky

Nefunkční požadavky určují vlastnosti systému, které nejsou přímo viditelné pro uživatele, ale ovlivňují jeho kvalitu a použitelnost [1], [6].

Tabulka 4: Nefunkční požadavky systému

ID	Požadavek	Popis
N1	Výkon	Systém musí rychle reagovat na požadavky uživatele
N2	Škálovatelnost	Systém musí zvládnout rostoucí počet uživatelů
N3	Bezpečnost	Ochrana dat a bezpečné zpracování transakcí
N4	Stabilita a dostupnost	Systém by měl být dostupný bez výpadků
N5	Použitelnost	Rozhraní musí být jednoduché a přehledné
N6	Responzivita	Aplikace musí fungovat na různých zařízeních
N7	Udržitelnost	Kód musí být přehledný a snadno rozšiřitelný
N8	Modularita systému	Systém musí být rozdělen na logické části

Zdroj: (vlastní zpracování)

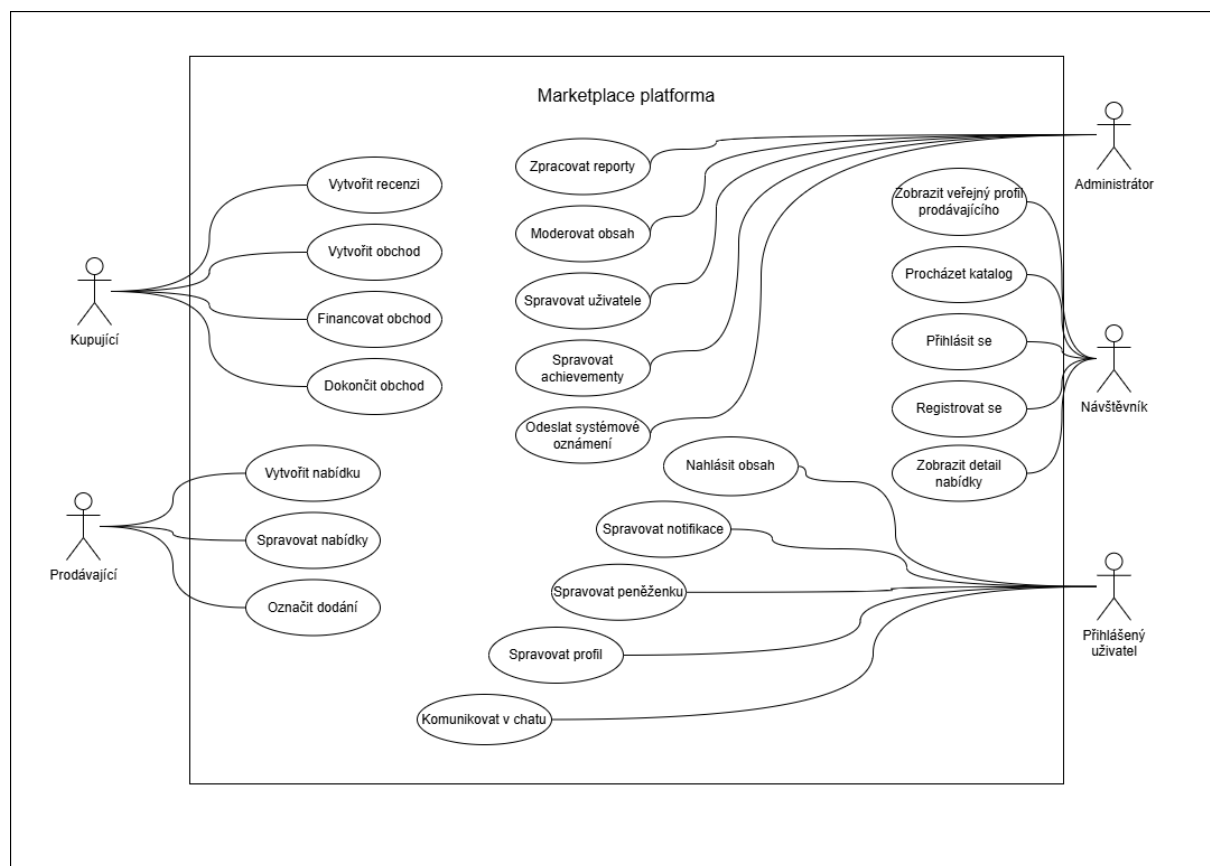
Definované požadavky sloužily jako základ pro návrh architektury systému, výběr technologií a implementaci jednotlivých modulů aplikace. Důraz byl kladen především na bezpečnost transakcí, realtime komunikaci a modularitu systému [3], [6], [30].

3.3 Případy užití systému

Případy užití popisují hlavní interakce mezi uživatelskými rolemi a navrženou marketplace platformou. Systém rozlišuje návštěvníka, přihlášeného uživatele a administrátora. Role kupujícího a prodávajícího nejsou v systému chápány jako pevně oddělené typy účtů. Jeden přihlášený uživatel může v jednom obchodě vystupovat jako kupující a v jiném jako prodávající. Administrátor je také přihlášený uživatel, který má navíc oprávnění pro správu a moderaci systému. V UML diagramu jsou kupující a prodávající znázorněni jako samostatní aktéři, protože představují role uživatele v konkrétním scénáři použití systému.

Návštěvník může procházet veřejný katalog, zobrazit detail nabídky, veřejný profil prodávajícího a provést registraci nebo přihlášení. Po přihlášení získává uživatel přístup k vlastnímu profilu, peněžence, konverzacím, notifikacím a funkcím spojeným s obchodním procesem. V roli kupujícího může vytvořit obchod, financovat jej pomocí interní peněženky, dokončit obchod a vytvořit recenzi. V roli prodávajícího může vytvářet a spravovat vlastní nabídky a označit dodání produktu nebo služby. Administrátor zajišťuje zpracování reportů, moderaci obsahu, správu uživatelů, achievementů a systémových oznámení.

Hlavní případy užití a jejich vazby na jednotlivé aktéry jsou znázorněny na obrázku 1.



Obrázek 1: Diagram případů užití systému (vlastní zpracování)

Diagram znázorňuje rozdělení funkcí do veřejné, uživatelské, obchodní a administrátorské části systému. Veřejná část je dostupná i bez přihlášení, zatímco uživatelské a obchodní funkce vyžadují přihlášeného uživatele. Administrátorské funkce jsou dostupné pouze uživateli s administrátorským oprávněním.

4 Návrh systému

Na základě definovaných funkčních a nefunkčních požadavků a výsledků analýzy existujících řešení byl navržen systém webové marketplace platformy pro digitální produkty a služby. Návrh se zaměřuje na propojení katalogu nabídek, obchodního workflow, interní peněženky, escrow mechanismu, realtime komunikace a administrátorské moderace.

Cílem návrhu bylo vytvořit systém, který není pouze katalogem digitálních produktů, ale podporuje celý průběh obchodu mezi kupujícím a prodávajícím. Z tohoto důvodu bylo nutné navrhnout architekturu, která umožňuje oddělit jednotlivé doménové části systému a zároveň zachovat konzistenci dat při obchodních a finančních operacích [1], [5], [6].

4.1 Architektura systému

Navržený systém je tvořen frontendovou aplikací, backendovým API, realtime komunikační vrstvou a relační databází. Frontendová část slouží jako uživatelské rozhraní pro kupující, prodávající a administrátory. Backendová část zpracovává business logiku aplikace, zajišťuje autentizaci, autorizaci, práci s nabídkami, obchody, interní peněženkou, recenzemi, reporty a administrátorskými funkcemi. Databázová vrstva slouží k perzistentnímu ukládání uživatelských dat, nabídek, obchodů, zpráv, finančních transakcí a auditních záznamů. Návrh vychází z principu oddělení odpovědností mezi jednotlivými částmi systému a z architektury klient-server [1], [4].

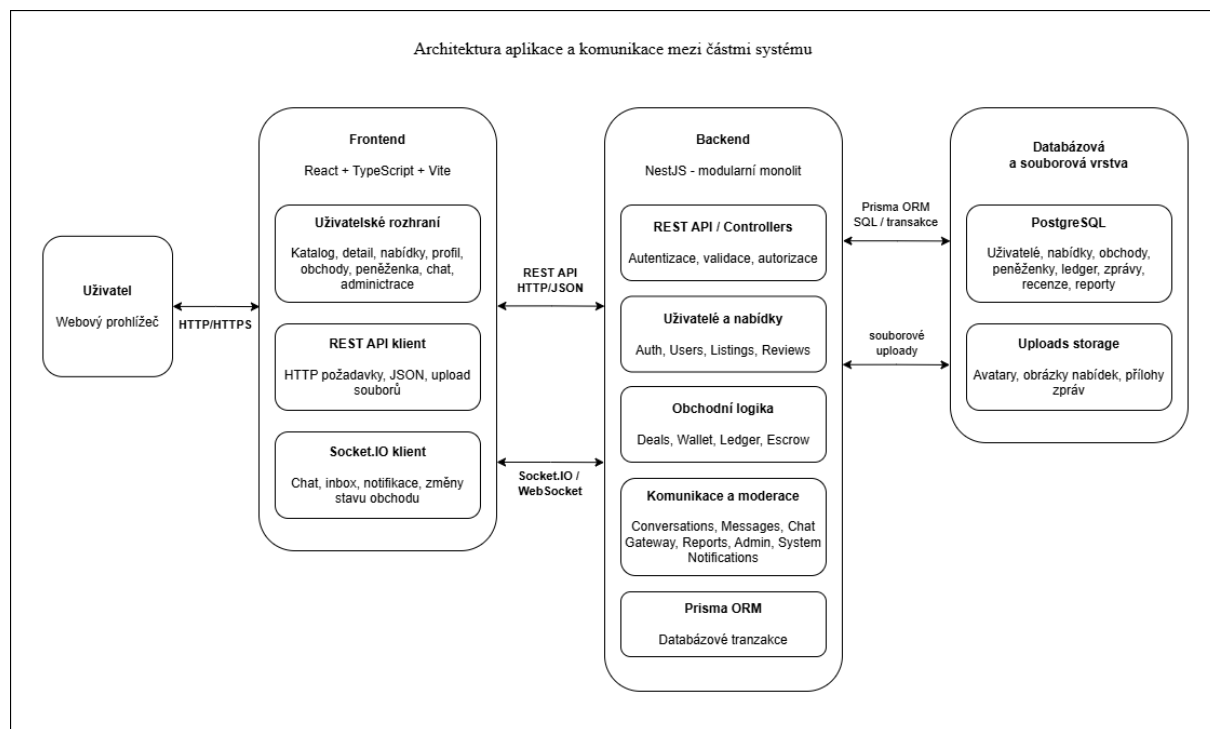
Aplikace je navržena jako modulární monolit. To znamená, že systém je nasazen jako jeden backendový celek, ale jeho vnitřní struktura je rozdělena do samostatných doménových modulů. Tato architektura odpovídá charakteru aplikace, protože jednotlivé části business logiky na sebe přímo navazují a musí pracovat nad konzistentními daty. Jedná se zejména o moduly nabídek, obchodů, peněženky, chatu, recenzí, reportů a administrace. Logické členění systému podle doménových oblastí odpovídá principům oddělení odpovědností a doménově orientovaného návrhu [1], [5].

Pro marketplace platformu je tento návrh vhodný zejména kvůli escrow mechanismu. Při vytvoření nebo dokončení obchodu nestačí změnit pouze jeden záznam v databázi. Systém musí současně změnit stav obchodu, upravit zůstatek v peněžence, vytvořit ledger záznam a případně aktualizovat dostupnost nabídky. Díky modulárnímu monolitu mohou tyto operace probíhat v rámci jedné aplikace a jedné databázové transakce, což zjednodušuje zachování konzistence dat. Význam transakcí a konzistence dat je v systémech pracujících s navazujícími operacemi zásadní [6], [7].

Alternativou by bylo rozdělení aplikace do více mikroslužeb, například samostatná služba pro uživatele, obchody, platby a komunikaci. Pro rozsah této práce by však takové řešení přineslo zbytečně vysokou infrastrukturní složitost. Bylo by nutné řešit síťovou komunikaci mezi službami, distribuované transakce, monitoring a složitější nasazení. Modulární monolit proto představuje vhodný kompromis mezi přehledností, rozšiřitelností a jednoduchostí provozu [1], [3], [6].

Komunikace mezi frontendem a backendem probíhá dvěma způsoby. Běžné operace, například registrace, přihlášení, načítání nabídek, správa profilu nebo práce s obchody, jsou realizovány

pomocí REST API. Události, které mají být předány uživateli okamžitě, jsou řešeny pomocí realtime vrstvy založené na Socket.IO. Socket.IO je v návrhu použito hlavně pro chat, inbox, online stav uživatelů, systémové notifikace a aktualizace stavu obchodů. Kombinace HTTP rozhraní a realtime komunikace odpovídá potřebám interaktivních webových aplikací [4], [11], [15].



Obrázek 2: Architektura aplikace a komunikace mezi částmi systému (vlastní zpracování)

Obrázek 2 znázorňuje základní komunikaci mezi částmi systému. Frontendová aplikace komunikuje s backendem pomocí HTTP požadavků a websocketového spojení. Backend následně zpracovává aplikační logiku a komunikuje s databází PostgreSQL prostřednictvím ORM nástroje Prisma. Uživatelské soubory jsou ukládány do lokálního adresáře uploads, který backend zpřístupňuje přes statickou cestu. Celé řešení je připraveno pro spuštění v kontejnerizovaném prostředí pomocí Docker Compose [7], [10], [12], [15].

4.2 Volba technologií

Volba technologií vycházela z požadavků na vytvoření fullstack aplikace s jasně oddělenou frontendovou a backendovou částí, relační databází, realtime komunikací a možností lokálního spuštění v konzistentním prostředí. Důležitým kritériem byla také vhodnost technologií pro implementaci business logiky marketplace platformy, zejména escrow mechanismu, interní peněženky a auditovatelných ledger transakcí [6], [7].

Pro backend byl zvolen framework NestJS. Hlavním důvodem byla jeho modulární struktura, podpora dependency injection a dobrá organizace zdrojového kódu. Tyto vlastnosti odpovídají návrhu systému jako modulárního monolitu, protože každý doménový celek může být implementován jako samostatný modul se svými controllery, services a DTO objekty. NestJS zároveň poskytuje vhodné prostředky pro autentizaci, validaci vstupních dat, práci s guardy a integraci websocketové komunikace [8].

Frontendová část byla implementována pomocí React, TypeScriptu a Vite. React byl zvolen kvůli komponentovému přístupu, který umožňuje rozdělit uživatelské rozhraní na menší opakovaně použitelné části. To je vhodné například pro katalog nabídek, formuláře, karty obchodů, chatovací komponenty, profil uživatele a administrátorské rozhraní. TypeScript pomáhá omezovat chyby při práci s daty z API a zvyšuje přehlednost kódu. Vite byl použit jako moderní build nástroj, který zjednodušuje vývoj a zrychluje spouštění frontendové části [9], [16].

Databázová vrstva je postavena na PostgreSQL. Relační databáze byla zvolena z důvodu jasné definovaných vztahů mezi entitami a potřeby transakčního zpracování dat. Marketplace platforma pracuje s uživateli, nabídkami, obchody, zprávami, recenzemi, reporty a finančními transakcemi. Tyto části jsou vzájemně propojené a u obchodní logiky je důležité zachovat konzistenci dat. PostgreSQL je proto vhodnější volbou než dokumentová databáze, protože systém vyžaduje relační vazby a databázové transakce [6], [7], [24].

Pro práci s databází byl použit nástroj Prisma ORM. Prisma umožňuje definovat databázový model v jednom schématu, generovat typově bezpečného klienta a provádět databázové migrace. V rámci této práce bylo důležité především typově bezpečné napojení backendu na databázovou vrstvu a možnost využívat transakce při operacích, které mění stav obchodu, zůstatek peněženky a ledger záznamy [10].

Realtime komunikace je realizována pomocí Socket.IO. Oproti běžným HTTP požadavkům umožňuje serveru aktivně odesílat události klientovi v okamžiku, kdy dojde ke změně stavu systému. V aplikaci se tato komunikace používá pro chat mezi kupujícím a prodávajícím, aktualizaci inboxu, online presence, systémové notifikace a změny stavu obchodů. REST API je proto v systému doplněno websocketovou vrstvou, čímž vzniká hybridní komunikační model [4], [11], [15].

Pro lokální spuštění a sjednocení vývojového prostředí byl použit Docker Compose. Docker Compose umožňuje spustit databázi, backend a frontend jako samostatné kontejnery. Díky tomu není nutné ručně instalovat všechny části prostředí a aplikaci lze jednodušeji spustit na jiném zařízení. Takové uspořádání zároveň odpovídá běžné praxi při vývoji moderních webových aplikací [12].

Tabulka 5: Srovnání zvolených technologií a alternativ

Oblast	Zvolená technologie	Alternativy	Důvod výběru
Backend	NestJS (Node.js)	Spring Boot, Django	Modulární architektura a přehledná struktura
Frontend	React + TypeScript	Angular, Vue	Komponentový přístup a flexibilita
Databáze	PostgreSQL	MySQL, MongoDB	Podpora relací a databázových transakcí
Realtime	WebSocket (Socket.IO)	Polling, SSE	Obousměrná komunikace a nízká latence

ORM	Prisma	TypeORM	Typová bezpečnost a jednodušší práce s databází
Infrastructure	Docker	Lokální instalace	Konzistentní prostředí
Authentication	JWT	Session-based authentication	Bezstavová autentizace

Zdroj: (vlastní zpracování podle [7], [8], [9], [10], [11], [12], [16], [18], [19], [21], [22], [23], [24], [25], [31])

4.3 Backendová část systému

Backendová část je rozdělena do doménových modulů podle hlavních oblastí systému. Modul autentizace řeší registraci, přihlášení, odhlášení, ověření e-mailu a práci s aktuálně přihlášeným uživatelem. Modul uživatelů spravuje profily, veřejné profily prodávajících, reputaci a uživatelská nastavení. Modul nabídek zajišťuje katalog, filtrování, detail nabídky, správu vlastních nabídek a práci s cenovou historií. Takové členění odpovídá modulární organizaci backendové aplikace a principu oddělení odpovědností [1], [5], [8].

Nejdůležitější část business logiky je soustředěna v modulech obchodů a peněženky. Modul obchodů spravuje životní cyklus transakce mezi kupujícím a prodávajícím, zatímco modul peněženky zajišťuje změny zůstatků a vytváření ledger záznamů. Tyto moduly spolu úzce spolupracují při escrow operacích, například při blokaci prostředků, dokončení obchodu nebo refundaci. U těchto operací je důležité zachovat konzistenci dat pomocí databázových transakcí [6], [7].

Komunikační část backendu je rozdělena mezi konverzace, zprávy a websocketovou gateway. Konverzace a zprávy zajišťují perzistentní uložení komunikace, zatímco realtime vrstva doručuje nové zprávy, aktualizace inboxu a změny stavu obchodů připojeným klientům. Administrátorský modul potom poskytuje nástroje pro moderaci uživatelů, nabídek, reportů, recenzí a systémových oznámení [11], [15].

4.4 Frontendová část systému

Frontendová část byla navržena jako single page application. Uživatelské rozhraní je rozděleno do samostatných stránek a komponent, které odpovídají hlavním částem marketplace platformy. Veřejná část aplikace podporuje procházení katalogu, zobrazení detailu nabídky, registraci a přihlášení. Přihlášený uživatel má přístup k profilu, správě vlastních nabídek, obchodům, peněžence, inboxu a nastavení účtu. Tento přístup odpovídá principům moderních komponentově orientovaných frontendových aplikací [9], [16].

Rozhraní aplikace se přizpůsobuje roli a kontextu uživatele. Kupující může vyhledávat nabídky, vytvářet obchody, komunikovat s prodávajícím a po dokončení obchodu vložit recenzi. Proávající může vytvářet a upravovat nabídky, sledovat příchozí obchody, komunikovat se zákazníkem a označovat obchod jako doručení. Administrátor má k dispozici samostatné rozhraní pro moderaci systému. Rozhraní je proto rozděleno podle hlavních rolí a scénářů

použití tak, aby uživatel rychle našel funkce odpovídající jeho aktuální činnosti. Tento návrh odpovídá principům použitelnosti uživatelských rozhraní [2].

Frontend komunikuje s backendem pomocí vlastního API klienta založeného na knihovně Axios. Autentizace je řešena pomocí httpOnly cookie, proto frontend neukládá JWT token do localStorage. Pro realtime komunikaci je využit socket klient, který zajišťuje připojení k websocketové vrstvě backendu a zpracování událostí, jako jsou nové zprávy, změny online stavu nebo aktualizace obchodu [11], [15].

4.5 Databázový návrh

Databázová vrstva systému je navržena nad relační databází PostgreSQL a přístup k datům je realizován pomocí ORM nástroje Prisma [7], [10]. Datový model vychází z hlavních domén marketplace systému a je navržen s důrazem na konzistenci dat, přehlednost vztahů a podporu transakční logiky [5], [6].

Základ databázového modelu tvoří uživatelé, nabídky, obchody, peněženky, ledger transakce, konverzace, zprávy, recenze, reporty a administrátorské záznamy. Tyto entity jsou navrženy tak, aby bylo možné zachytit celý průběh marketplace transakce od vytvoření nabídky až po dokončení obchodu a uložení finanční historie.

Zvláštní pozornost byla věnována finanční části systému. Aktuální zůstatek uživatele je uložen v peněžence, ale každá změna zůstatku je zároveň zaznamenána jako samostatná ledger transakce. Díky ledgeru lze zpětně dohledat, proč se zůstatek změnil, zda šlo o dobítí, výběr, blokaci prostředků, uvolnění escrow částky nebo refundaci. Ledger tedy neslouží pouze pro zobrazení historie uživateli, ale také jako auditovatelná vrstva finančních operací. U systému pracujících s navazujícími změnami dat je taková auditovatelnost a konzistence důležitá [6], [7].

Databázový návrh podporuje také komunikační a moderační část systému. Konverzace a zprávy umožňují uchovávat historii komunikace mezi kupujícím a prodávajícím. Reporty, administrátorské zásahy a auditní záznamy pomáhají řešit problematický obsah a zpětně dohledávat důležité akce v systému. Logické rozdělení těchto částí odpovídá doménovému členění aplikace [5].

Základní entitou systému je uživatel, který může vystupovat v roli kupujícího, prodávajícího nebo administrátora. Návrh počítá s vazbami uživatele na nabídky, obchody, zprávy, hodnocení a interní peněženku.

Entita nabídky reprezentuje digitální produkt nebo službu. Návrh zahrnuje informace o názvu, popisu, kategorii, ceně, typu produktu, stavu nabídky a případné slevě. Součástí datového modelu je také historie cen, lze pomocí něj evidovat změny ceny v čase.

Obchod propojuje kupujícího, prodávajícího a konkrétní nabídku. Datový model obsahuje stav obchodu, množství produktu a cenový snapshot uložený v okamžiku vytvoření objednávky. Tím lze zachovat konzistenci dat i v případě pozdější změny ceny nabídky [6].

Součástí návrhu je také systém interní peněženky. Peněženka slouží k evidenci aktuálního zůstatku uživatele a je propojena se samostatnou entitou finančních transakcí. Návrh počítá s ukládáním jednotlivých operací, například dobítí prostředků, escrow rezervace, uvolnění platby

nebo vrácení financí. Tento přístup podporuje auditovatelnost systému a kontrolu změn zůstatku [6].

Komunikační část systému je tvořena entitami konverzací a zpráv. Konverzace je navázána na konkrétní nabídku a dvojici uživatelů. Návrh počítá s chronologickým ukládáním zpráv a podporou textového obsahu i metadat médií.

Datový model dále zahrnuje recenze, achievementy, systémové notifikace, reporty obsahu a auditní záznamy administrátorských akcí. Recenze jsou navázány na dokončené obchody a slouží jako základ reputačního systému platformy.

Moderace systému je v databázovém návrhu podpořena entitami pro reporty, uživatelská upozornění, systémové notifikace a auditní záznamy administrátorských akcí.

Databázový návrh je vytvořen s důrazem na normalizaci dat, minimalizaci redundance a zachování konzistence při zpracování obchodních a finančních operací [6], [7].

4.6 Komunikace v systému

Komunikace v systému je rozdělena na synchronní komunikaci pomocí REST API a realtime komunikaci pomocí Socket.IO. REST API je využito pro operace, u kterých klient odešle požadavek a očekává konkrétní odpověď serveru. Patří sem například registrace, přihlášení, načítání katalogu, vytvoření nabídky, založení obchodu, práce s peněženkou nebo administrátorské akce. Tento způsob komunikace odpovídá principům REST architektury a využití HTTP rozhraní pro práci se zdroji systému [4], [13].

Realtime komunikace je využita tam, kde je potřeba okamžitě informovat uživatele o změně stavu bez ručního obnovení stránky. Jedná se především o chat mezi kupujícím a prodávajícím, aktualizaci inboxu, online presence, systémové notifikace a změny stavu obchodů. Díky tomu může například kupující okamžitě vidět, že prodávající označil obchod jako doručený, nebo prodávající obdrží novou zprávu bez nutnosti znovu načítat stránku. Pro tyto scénáře jsou vhodné technologie umožňující obousměrnou komunikaci mezi klientem a serverem [11], [15].

Tento hybridní komunikační model byl zvolen proto, že marketplace platforma kombinuje běžné CRUD operace s interaktivními scénáři. REST API poskytuje jednoduché a přehledné rozhraní pro většinu operací, zatímco websocketová vrstva doplňuje systém o okamžitou synchronizaci událostí mezi uživateli [4], [11], [15].

5 Implementace

Praktická část práce navazuje na návrh systému z předchozí kapitoly a popisuje realizaci fullstack marketplace aplikace pro digitální produkty a služby. Výsledkem implementace je aplikace propojující katalog nabídek, uživatelské účty, obchodní workflow s escrow logikou, interní peněženku, realtime komunikaci a administrátorskou správu.

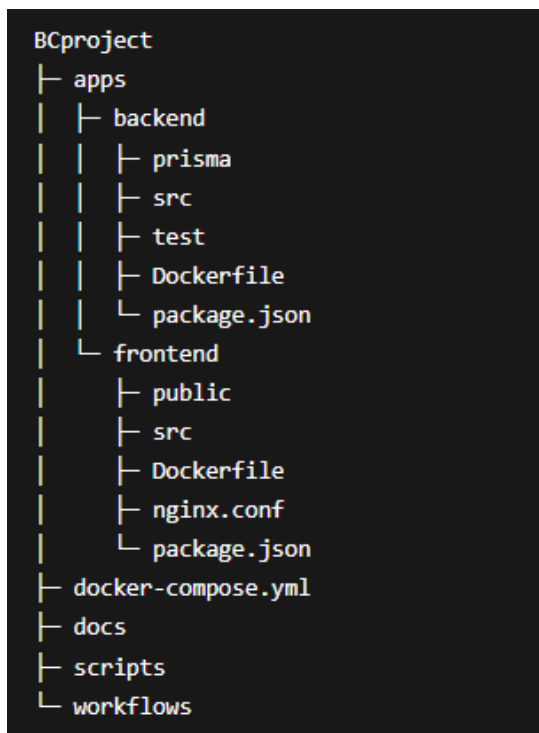
Aplikace je rozdělena na backendovou, frontendovou a databázovou vrstvu. Backend je vytvořen ve frameworku NestJS a poskytuje REST API i realtime komunikaci přes Socket.IO. Frontend využívá React, TypeScript a Vite. Perzistentní data jsou uložena v databázi PostgreSQL, ke které backend přistupuje pomocí Prisma ORM.

Implementace vychází z principu modulárního monolitu. Backend běží jako jedna aplikace, ale zdrojový kód je rozdělen podle doménových oblastí, například autentizace, nabídky, obchody, peněženka, chat nebo administrace. Frontend je členěn na stránky, komponenty, API vrstvu, autentizační logiku a pomocné utility.

5.1 Struktura projektu

Projekt je uložen v monorepozitáři, který obsahuje backendovou i frontendovou část aplikace. Hlavní zdrojové soubory jsou umístěny v adresářích apps/backend a apps/frontend. Backendová část obsahuje zdrojový kód NestJS aplikace, Prisma schéma, databázové migrace, testy a soubory pro spuštění v kontejneru. Frontendová část obsahuje React aplikaci, stránky, komponenty, API klienty, konfiguraci Vite a soubor nginx.conf pro produkční běh aplikace.

V kořenové části projektu se nachází zejména docker-compose.yml, dokumentace, podpůrné skripty a konfigurační soubory pro automatizované procesy. Docker Compose propojuje databázi PostgreSQL, backend a frontend do jednoho lokálně spustitelného prostředí.



Obrázek 3: Zjednodušená struktura projektu (vlastní zpracování)

Rozdělení projektu odpovídá zvolené architektuře. Backend je provozován jako jeden celek, ale jeho vnitřní části jsou odděleny do samostatných modulů. Frontend je členěn podle uživatelských obrazovek a znovupoužitelných komponent. Díky tomu lze jednotlivé části aplikace rozšiřovat bez zásadních zásahů do zbytku projektu.

5.1.1 Monorepozitář

Monorepozitář sdružuje všechny části aplikace na jednom místě. Obsahuje backend, frontend, Docker konfiguraci, podpůrné skripty a dokumentaci. Pro tento projekt je takové uspořádání praktické, protože všechny vrstvy aplikace spolu úzce souvisejí a spouštějí se společně pomocí Docker Compose.

Výhodou monorepozitáře je jednodušší orientace v projektu a jednotná správa konfiguračních souborů. Při vývoji není nutné přecházet mezi několika samostatnými repozitáři, protože aplikační kód i infrastruktura jsou uloženy společně.

Kořenová struktura projektu obsahuje adresář apps pro aplikační části, soubor docker-compose.yml pro spuštění kontejnerů, adresář docs pro dokumentaci, adresář scripts pro podpůrné skripty a adresář workflows pro automatizované procesy.

5.1.2 Backendová struktura

Serverová část je implementována ve frameworku NestJS a organizována jako modulární monolit. Zdrojové soubory se nacházejí v adresáři apps/backend/src, kde jsou rozděleny do samostatných modulů podle odpovědnosti.

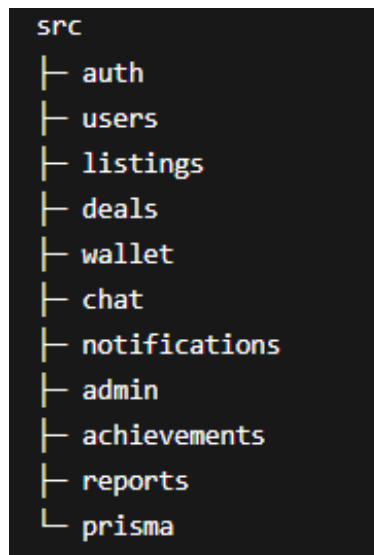
Jednotlivé moduly obsahují controllery, služby, DTO objekty, entity a podle potřeby také guardy nebo websocket gateway. Controller přijímá požadavky z klientské části, servisní vrstva zpracovává business logiku a DTO objekty určují strukturu vstupních dat. Toto rozdělení pomáhá udržet hranice mezi doménovými oblastmi aplikace.

Mezi hlavní backendové moduly patří autentizace, uživatelé, nabídky, obchody, peněženka, konverzace, zprávy, notifikace, reporty a administrace. Samostatně jsou řešeny také achievements, auditní záznamy administrátorských akcí a realtime komunikace.

Autentizační modul pokrývá registraci, přihlášení, práci s JWT tokeny, obnovu hesla a ověření e-mailové adresy. Modul uživatelů pracuje s profily, avatary, reputací a veřejnými údaji prodávajících. Nabídky tvoří katalog digitálních produktů a služeb, zatímco modul obchodů řeší životní cyklus objednávky a escrow mechanismus.

Finanční operace jsou soustředěny do modulu peněženky. Ten pracuje se zůstatky uživatelů, ledger transakcemi a operacemi typu escrow lock, release nebo refund. Změny zůstatků probíhají v databázových transakcích, aby nedocházelo k nekonzistentním stavům při zpracování obchodů.

Komunikační část kombinuje REST API a realtime gateway založenou na Socket.IO. REST API slouží pro běžné načítání a ukládání dat, zatímco websocketová vrstva zajišťuje okamžité doručování zpráv, aktualizaci inboxu, online stav uživatelů a změny stavu obchodů.



Obrázek 4: Struktura backendové části systému (vlastní zpracování)

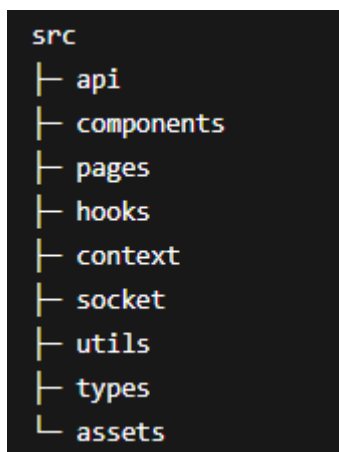
5.1.3 Frontendová struktura

Klientská část je vytvořena pomocí Reactu, TypeScriptu a nástroje Vite. Zdrojové soubory jsou umístěny v adresáři apps/frontend/src a rozděleny podle role v uživatelském rozhraní.

Adresář pages obsahuje hlavní stránky aplikace, například katalog nabídek, detail nabídky, profil uživatele, peněženku, inbox nebo administrátorskou část. Znovupoužitelné prvky rozhraní jsou umístěny v adresáři components. Patří sem navigace, formuláře, modální okna, tabulky, karty nabídek a další sdílené části UI.

Komunikace s backendem je oddělena do samostatné API vrstvy. HTTP požadavky využívají knihovnu Axios, realtime komunikace je řešena přes Socket.IO klienta. Frontend tak kombinuje běžné REST požadavky s okamžitou synchronizací dat tam, kde je potřeba rychlá reakce aplikace.

Autentizační vrstva spravuje uživatelskou relaci, načítání aktuálně přihlášeného uživatele a ochranu neveřejných stránek. Aplikace je navržena jako Single Page Application, takže navigace mezi hlavními částmi probíhá bez úplného obnovení stránky.



Obrázek 5: Struktura frontendové části systému (vlastní zpracování)

5.1.4 Konfigurační soubory

Projekt obsahuje konfigurační soubory pro vývoj, testování, build a spuštění aplikace. Backendová i frontendová část mají vlastní package.json, ve kterém jsou definovány závislosti a skripty pro lokální běh, sestavení a testování.

Soubory .env.example slouží jako vzor pro nastavení environment proměnných. Obsahují hodnoty potřebné pro připojení k databázi, nastavení API, CORS, JWT nebo dalších částí aplikace.

Backend obsahuje konfiguraci TypeScriptu, ESLintu, Prettieru, NestJS a Dockeru. Frontend pracuje s konfigurací Vite, TypeScriptu, Tailwind CSS, PostCSS, ESLintu a Nginxu. Soubor nginx.conf je použit při produkčním běhu sestavené frontendové aplikace.

V kořenové části projektu je umístěn docker-compose.yml, který propojuje backend, frontend a databázi PostgreSQL. Díky tomu lze aplikaci spustit v konzistentním prostředí bez manuální konfigurace jednotlivých služeb.

5.2 Implementace backendové části

Backend aplikace je implementován ve frameworku NestJS. Zajišťuje REST API, realtime komunikaci, autentizaci uživatelů, práci s databází a hlavní business logiku marketplace platformy. V rámci projektu backend zpracovává registraci a přihlášení, správu nabídek, obchodní workflow, escrow operace, interní peněženku, chat, recenze, reporty a administrátorské zásahy.

Z hlediska struktury je backend navržen jako modulární monolit. Aplikace se spouští jako jeden celek, ale zdrojový kód je rozdělen podle doménových oblastí. Každý modul řeší vlastní část systému a obsahuje controllery, servisní vrstvu, DTO objekty a případně guardy nebo gateway pro realtime komunikaci. Díky tomu zůstává aplikace jednodušší na spuštění než systém založený na mikroslužbách, ale zároveň je kód rozdělen přehledně podle odpovědností.

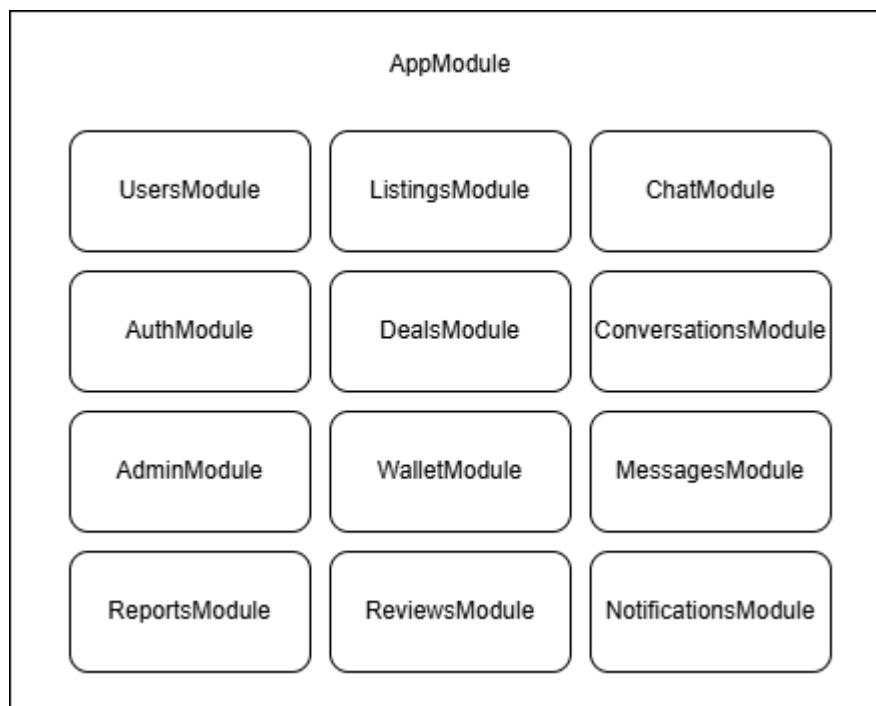
5.2.1 Struktura NestJS aplikace

Vstupním bodem backendu je soubor main.ts, ve kterém se inicializuje server, globální validace vstupních dat, CORS politika, bezpečnostní middleware a další základní nastavení aplikace. Hlavní propojení doménových částí je definováno v souboru app.module.ts.

Aplikace je rozdělena do modulů podle hlavních oblastí systému. Mezi nejdůležitější patří moduly pro autentizaci, uživatele, nabídky, obchody, peněženku, konverzace, zprávy, recenze, reporty, systémové notifikace a administraci. Samostatně je řešena také realtime komunikace pomocí Socket.IO gateway.

Controller v jednotlivých modulech přijímá HTTP požadavky a předává je servisní vrstvě. Service obsahuje aplikační logiku, pracuje s databází prostřednictvím Prisma ORM a provádí kontroly oprávnění nebo stavů entit. DTO objekty určují strukturu vstupních dat a slouží k validaci požadavků ještě před jejich zpracováním.

Toto rozdělení omezuje míchání transportní vrstvy s business logikou. Například modul listings řeší katalog nabídek, modul deals pracuje se stavem obchodů a escrow procesem, zatímco modul wallet zpracovává operace nad interní peněženkou a ledgerem.



Obrázek 6: Struktura NestJS modulů (vlastní zpracování)

5.2.2 Použité závislosti

Backendová Backend využívá několik knihoven, které pokrývají hlavní technické potřeby aplikace. Základem je framework NestJS, který poskytuje modulární strukturu, dependency injection a nástroje pro tvorbu REST API.

Pro práci s databází je použito Prisma ORM. Datový model je definován v souboru schema.prisma a aplikace následně komunikuje s databází pomocí typově bezpečného Prisma Clientu. Prisma je využita nejen pro běžné CRUD operace, ale také pro databázové transakce v částech, kde je nutné zachovat konzistenci finančních operací.

Autentizace je postavena na JWT tokenech a Passport strategii. Knihovny `@nestjs/jwt` a Passport slouží k ověřování identity uživatele a ochraně neveřejných endpointů. Hesla jsou ukládána pouze v hashované podobě pomocí knihovny `bcrypt`.

Validace vstupních dat je realizována pomocí `class-validator` a `class-transformer`. Tyto knihovny kontrolují strukturu příchozích požadavků na úrovni DTO objektů. Backend tak odmítá neúplná nebo chybně typovaná data dříve, než se dostanou do servisní logiky.

Realtime vrstva využívá `Socket.IO`. V aplikaci slouží hlavně pro chat, aktualizaci inboxu, online stav uživatelů, systémové notifikace a změny stavu obchodů. Upload souborů je řešen přes multipart požadavky a ukládání do serverového adresáře `uploads`.

Tabulka 6: Hlavní backendové knihovny a jejich využití

Knihovna	Účel
NestJS	Backend framework
Prisma	ORM

Passport	Auth
JWT	Token auth
bcrypt	Hashování hesel
Socket.IO	Realtime
class-validator	Validace
Multer	Upload souborů

Zdroj: (vlastní zpracování)

5.2.3 Konfigurace aplikace

Konfigurace backendu je řešena pomocí environment proměnných. Citlivé nebo prostředím závislé hodnoty tak nejsou uloženy přímo ve zdrojovém kódu. Vzorová konfigurace je připravena v souboru `.env.example`.

Mezi hlavní konfigurační hodnoty patří připojení k databázi, tajný klíč pro podepisování JWT tokenů, port backendu, adresa frontendové aplikace, nastavení CORS a údaje pro e-mailovou komunikaci. Při spuštění aplikace se tyto hodnoty načtou a použijí při inicializaci jednotlivých modulů.

```

1  DATABASE_URL=postgresql://marketplace:marketplace@localhost:5432/marketplace?schema=public
2  JWT_SECRET=change_me_to_a_long_random_secret
3  JWT_EXPIRES_IN=1h
4  CORS_ORIGIN=http://localhost:5173
5  PORT=3000
6  NODE_ENV=development
7  APP_BASE_URL=http://localhost:5173
8  SMTP_HOST=smtp.gmail.com
9  SMTP_PORT=587
10 SMTP_SECURE=false
11 SMTP_USER=no-reply@example.com
12 SMTP_PASS=xxxx xxxx xxxx xxxx
13 SMTP_FROM=TradeMarket <no-reply@example.com>

```

Obrázek 7: Příklad environment proměnných v souboru `.env.example` (vlastní zpracování)

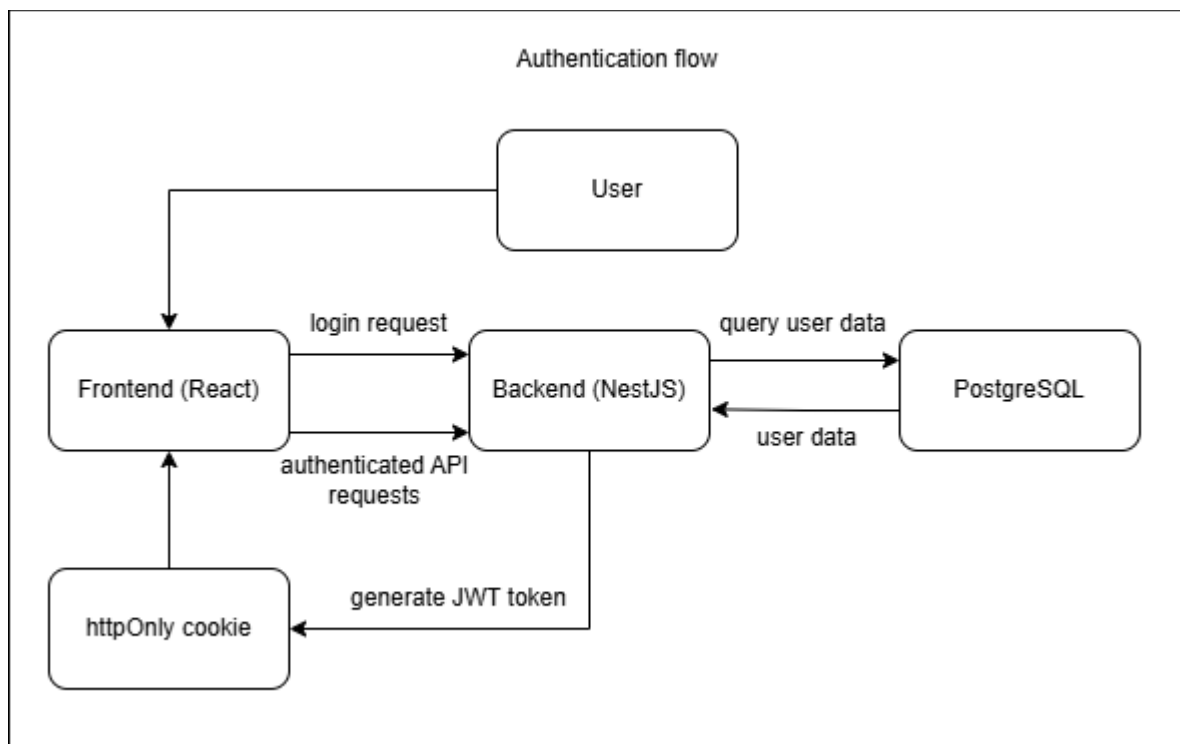
V souboru `main.ts` je nastavena globální validace vstupních dat pomocí `ValidationPipe`. Požadavky jsou kontrolovány před předáním do controllerů a servisní vrstvy. Neplatná data jsou odmítnuta automaticky, což snižuje riziko, že se do aplikační logiky dostanou neočekávané hodnoty.

Základní bezpečnostní konfiguraci doplňuje CORS politika, middleware `Helmet` a omezení počtu požadavků pomocí `throttlingu`. Tato pravidla se aplikují na celé API, ne pouze na vybrané endpointy. Centralizované nastavení zároveň usnadňuje spuštění aplikace v Docker prostředí, kde backend komunikuje s databází a frontendovou částí přes hodnoty definované v kontejnerové konfiguraci.

5.2.4 Autentizace a autorizace

Autentizace je založena na JWT tokenech. Po úspěšném přihlášení backend vytvoří token obsahující identifikátor uživatele a jeho roli. Token je uložen do `httpOnly` cookie, aby s ním

frontend nemusel pracovat přímo v JavaScriptu. Klientská část následně odesílá požadavky s nastavením `withCredentials`, díky čemuž se cookie přikládá automaticky.



Obrázek 8: Zjednodušený průběh autentizace uživatele v systému (vlastní zpracování)

Registrace začíná ověřením vstupních dat pomocí DTO objektů. Heslo je před uložením zahashováno pomocí knihovny `bcrypt`. Backend zároveň vytvoří ověřovací token pro e-mailovou adresu a odešle uživateli aktivační odkaz. Tím se omezuje vznik neplatných účtů a aplikace může rozlišovat mezi registrovaným a ověřeným uživatelem.

Při přihlášení backend porovná zadané údaje s daty v databázi. Pokud jsou správné a účet je aktivní, vytvoří JWT token a uloží jej do cookie. Ověřování tokenu probíhá pomocí Passport strategie v souboru `jwt.strategy.ts`. Strategie umí token načíst z cookie i z Authorization headeru, což je užitečné například při testování API.

```

13  @Injectable()
14  export class JwtStrategy extends PassportStrategy(Strategy) {
15      private static readonly ACCESS_COOKIE = 'access_token';
16
17      private static fromCookie(req: Request) {
18          const cookieHeader = req?.headers?.cookie;
19          if (!cookieHeader) return null;
20
21          const pair = cookieHeader
22              .split(';')
23              .map((chunk) => chunk.trim())
24              .find((chunk) => chunk.startsWith(`${JwtStrategy.ACCESS_COOKIE}=`));
25
26          if (!pair) return null;
27          const value = pair.substring(`${JwtStrategy.ACCESS_COOKIE}=`.length);
28          return value ? decodeURIComponent(value) : null;
29      }
30
31      constructor(private readonly prisma: PrismaService) {
32          const secret = process.env.JWT_SECRET;
33          if (!secret) throw new Error('JWT_SECRET is missing');
34
35          super({
36              jwtFromRequest: ExtractJwt.fromExtractors([
37                  (req: Request) => JwtStrategy.fromCookie(req),
38                  ExtractJwt.fromAuthHeaderAsBearerToken(),
39              ]),
40              ignoreExpiration: false,
41              secretOrKey: secret,
42          });
43      }

```

Obrázek 9: Implementace JWT strategie (vlastní zpracování)

Chráněné endpointy používají JwtAuthGuard. Po úspěšném ověření je aktuální uživatel dostupný v request objektu a servisní vrstva s ním může dále pracovat. Autentizační strategie nekontroluje pouze platnost tokenu, ale také aktuální stav účtu v databázi. Pokud byl uživatel mezitím zablokován administrátorem, požadavek je odmítnut ještě před zpracováním business logikou.

Autorizace je řešena kombinací rolí a vlastnických kontrol. Role určuje přístup k obecným částem systému, například k administraci. U konkrétních dat se navíc ověřuje vlastnictví nebo účast v daném procesu. Backend proto kontroluje, zda uživatel vlastní nabídku, je účastníkem obchodu nebo patří do konkrétní konverzace.

Tato kontrola je vynucena na serverové straně, protože frontend nelze považovat za bezpečnostní hranici. Skrytí tlačítka v rozhraní zlepšuje použitelnost, ale nezabrání přímému volání API. Kritická pravidla proto zůstávají v backendu.

Administrátorské endpointy jsou chráněny samostatnou kontrolou oprávnění. Pro práci s rolemi se využívají vlastní dekorátory a guardy, například při moderaci obsahu, správě reportů nebo změně rolí uživatelů. Autentizační modul zároveň obsahuje obnovu hesla pomocí časově omezeného tokenu zasláného na e-mail uživatele.

5.2.5 Uživatelé a profily

Modul uživatelů pracuje s uživatelskými účty, veřejnými profily a nastavením účtu. Obsahuje logiku pro úpravu profilu, změnu hesla, správu avataru, platebních údajů, achievementů a profilových odznaků.

Uživatel patří mezi hlavní entity aplikace. Účet obsahuje autentizační údaje, veřejně zobrazované informace, reputační data a vazby na ostatní části systému. Podle role může uživatel vystupovat jako kupující, prodávající nebo administrátor.

Přihlášený uživatel může upravovat vlastní profilové údaje, například zobrazované jméno, avatar nebo vybrané veřejné informace. Backend při těchto operacích ověřuje identitu uživatele a neumožňuje měnit cizí profil.

```
12 export class UpdateMeDto {
13   @ApiPropertyOptional({ example: 'New Name' })
14   @IsOptional()
15   @IsString()
16   @MinLength(2)
17   @MaxLength(50)
18   displayName?: string;
19
20   @ApiPropertyOptional({ example: 'new@example.com' })
21   @IsOptional()
22   @IsEmail()
23   @MaxLength(100)
24   email?: string;
25
26   @ApiPropertyOptional({ example: 'https://example.com/avatar.png' })
27   @IsOptional()
28   @Transform(({ value }: { value: unknown }) => {
29     if (typeof value !== 'string') return value;
30     const trimmed = value.trim();
31     return trimmed.length > 0 ? trimmed : null;
32   })
33   @IsUrl({ require_protocol: true })
34   @MaxLength(500)
35   avatarUrl?: string | null;
36 }
```

Obrázek 10: Validací DTO pro úpravu uživatelského profilu (vlastní zpracování)

Změna hesla vyžaduje zadání původního hesla. Backend jej porovná s uloženým hashem a teprve poté uloží nové heslo, opět pouze v hashované podobě. Tento postup chrání účet před neoprávněnou změnou přihlašovacích údajů.

Veřejný profil prodávajícího obsahuje údaje důležité pro důvěryhodnost v marketplace prostředí. Zobrazuje hodnocení, počet dokončených obchodů, aktivní nabídky, achievementy a profilové odznaky. Kupující tak může před vytvořením obchodu posoudit zkušenost a aktivitu prodávajícího.

Modul rozlišuje veřejná a neveřejná data. Veřejné endpointy vracejí pouze informace určené ostatním uživatelům. Citlivější údaje, například platební nastavení nebo správa účtu, jsou dostupné pouze vlastníkovému účtu nebo administrátorovi. Uživatelská entita je zároveň propojena s nabídkami, obchody, recenzemi, peněženkou, reporty a notifikacemi.

5.2.6 Nabídky a katalog

Modul nabídek implementuje práci s digitálními produkty a službami publikovanými v marketplace platformě. Nachází se v adresáři listings a obsahuje logiku pro vytvoření, úpravu, archivaci, obnovení, filtrování a správu historie cen.

Nabídka obsahuje název, popis, cenu, kategorii, typ položky, dostupné množství, obrázek, tagy a případnou slevu. Každá nabídka je spojena s konkrétním prodávajícím.

Při vytvoření nebo úpravě nabídky backend kontroluje vstupní data pomocí DTO objektů. Ověřuje povinná pole, formát ceny, typ nabídky, dostupné množství a pravidla pro slevy. Proávající může měnit pouze vlastní nabídky, což backend kontroluje při úpravě, archivaci i obnovení.

Veřejný katalog podporuje filtrování podle kategorie, typu položky, cenového rozsahu, hodnocení prodávajícího a dalších parametrů. Implementováno je také textové vyhledávání a řazení výsledků. Díky tomu katalog neslouží pouze jako výpis položek, ale jako hlavní vstupní bod do obchodního procesu.

Detail nabídky propojuje katalog s dalšími částmi aplikace. Uživatel zde vidí popis, cenu, dostupnost, informace o prodávajícím, hodnocení a možnosti navázání kontaktu nebo vytvoření obchodu.

Backend ukládá také historii cen do tabulky ListingPriceHistory. Při změně ceny vzniká nový záznam, který umožňuje sledovat vývoj ceny v čase. Tato data jsou využita i při kontrole flash sale pravidel, například při ověřování minimální ceny za posledních 30 dní.

```
61 private async getMinBasePrice30d(  
62     listingId: string,  
63     currentBasePrice: number,  
64     client: PrismaService | Prisma.TransactionClient = this.prisma,  
65 ) {  
66     const rows = await client.listingPriceHistory.findMany({  
67         where: {  
68             listingId,  
69             isSale: false,  
70             createdAt: { gte: this.getHistoryWindowStart() },  
71         },  
72         select: { price: true },  
73     });  
74  
75     const prices = rows.map((row) => row.price);  
76     prices.push(currentBasePrice);  
77  
78     return Math.min(...prices);  
79 }
```

Obrázek 11: Načtení minimální ceny nabídky za posledních 30 dní (vlastní zpracování)

Nabídky se při odstranění nevymazávají fyzicky, ale archivují se. Archivace skryje položku z veřejného katalogu, ale zachová návazná data, například historii obchodů, zprávy nebo cenové záznamy. Tento způsob je vhodnější než trvalé mazání, protože marketplace pracuje s daty, která mohou být důležitá i po ukončení prodeje.

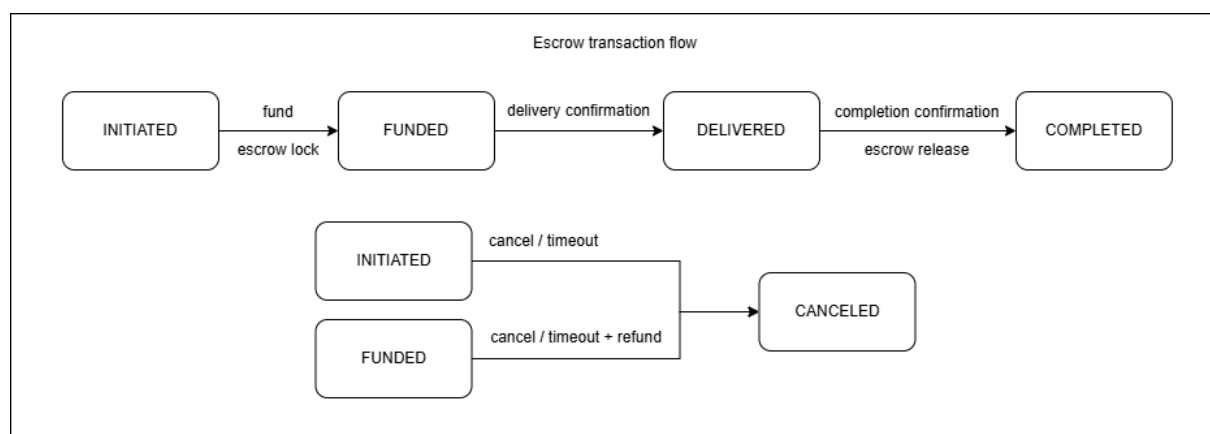
5.2.7 Obchody a escrow logika

Modul obchodů řídí proces nákupu mezi kupujícím a prodávajícím. Propojuje nabídky, interní peněženku, chat a stavovou logiku objednávky. Implementace se nachází v adresáři deals.

Obchod vzniká z konkrétní nabídky. Kupující zvolí množství a odešle požadavek na vytvoření obchodu. Backend ověří existenci a stav nabídky, dostupné množství, cenu a také to, zda kupující není vlastníkem dané nabídky. Nákup vlastních položek není povolen.

Při vytvoření obchodu se ukládá cenový snapshot. Obchod tak není závislý na aktuální ceně nabídky, která se může později změnit. Do entity obchodu se proto ukládá jednotková cena i celková částka platná v okamžiku vytvoření.

Financování obchodu probíhá přes interní peněženku. Backend nejprve ověří, zda má kupující dostatečný zůstatek. Pokud ano, částka se odečte z peněženky kupujícího a uzamkne se v escrow režimu. Prodávající ji v této fázi ještě nedostává.



Obrázek 12: Životní cyklus obchodu s využitím escrow mechanismu (vlastní zpracování)

Escrow chrání obě strany. Kupující má jistotu, že prostředky nejsou uvolněny před dodáním produktu nebo služby. Prodávající zároveň vidí, že kupující prostředky skutečně rezervoval.

Obchod prochází několika stavy. Po vytvoření a financování je aktivní, prodávající označí dodání a kupující následně potvrdí dokončení. Po potvrzení backend uvolní prostředky prodávajícímu a obchod označí jako dokončený. Pokud obchod neproběhne úspěšně, může být zrušen a uzamčené prostředky se vrátí kupujícímu.

Backend u každé operace kontroluje roli účastníka v obchodu. Kupující může financovat obchod a potvrdit dokončení, prodávající může označit dodání. Uživatel mimo daný obchod k těmto operacím přístup nemá.

Operace související se zůstatkem a stavem obchodu probíhají v databázových transakcích. Změna stavu obchodu, úprava peněženky a zápis do ledgeru se provedou společně. Pokud některý krok selže, transakce se neuloží. Tím se omezuje riziko nekonzistentního stavu, například vytvoření obchodu bez odpovídající finanční operace.

```

304     async complete(dealId: string, buyerId: string) {
305         await this.prisma.$transaction(async (tx) => {
329
330             await this.wallet.releaseEscrowToSeller(
331                 tx,
332                 deal.sellerId,
333                 deal.id,
334                 amount,
335             );
336
337             await tx.deal.update({
338                 where: { id: dealId },
339                 data: {
340                     status: 'COMPLETED',
341                     expiresAt: null,
342                     canceledByActor: null,
343                 },
344             });

```

Obrázek 13: Databázové transakce při dokončení obchodu (vlastní zpracování)

Modul obchodů pracuje také s časovými limity. Některé stavy by neměly zůstat aktivní neomezeně dlouho, proto je připravena timeout logika pro nedokončené obchody. Obchodní proces je propojen s konverzacemi, takže kupující a prodávající mohou během řešení objednávky komunikovat přímo v aplikaci.

5.2.8 Peněženka a ledger transakce

Modul peněženky spravuje interní zůstatky uživatelů a finanční operace marketplace platformy. Nachází se v adresáři wallet a úzce navazuje na escrow logiku v modulu obchodů.

Každý uživatel má vlastní peněženku s aktuálním disponibilním zůstatkem. Zůstatek se mění při dobítí, výběru, uzamčení prostředků v escrow, refundaci nebo uvolnění platby prodávajícímu. V aplikaci není integrována reálná platební brána; dobíjení a výběr prostředků jsou řešeny jako demonstrační interní operace. To umožňuje ověřit obchodní logiku a práci s transakcemi bez napojení na externí platební službu.

Každá změna zůstatku vytváří záznam v ledgeru. Ledger ukládá typ operace, částku, související obchod a další metadata. Neslouží pouze jako historie pro uživatele, ale také jako základ auditovatelnosti finančních operací.


```

198     async lockEscrow(
199         tx: Prisma.TransactionClient,
200         buyerId: string,
201         dealId: string,
202         amount: number,
203     ) {
204         const wallet = await this.getOrCreateWalletTx(tx, buyerId);
205
206         if (wallet.balance < amount) {
207             throw new ForbiddenException('Insufficient balance');
208         }
209
210         await tx.wallet.update({
211             where: { userId: buyerId },
212             data: { balance: { decrement: amount } },
213         });
214
215         await tx.walletTransaction.create({
216             data: {
217                 walletId: buyerId,
218                 userId: buyerId,
219                 type: 'ESCROW_LOCK',
220                 amount: -amount,
221                 dealId,
222             },
223         });
224     }

```

Obrázek 14: Práce s ledger transakcemi v modulu peněženky (vlastní zpracování)

Při financování obchodu backend ověří zůstatek kupujícího. Pokud je dostatečný, částka se odečte z jeho peněženky a v ledgeru vznikne záznam typu escrow lock. Prostředky se v této fázi nepřičítají prodávajícímu.

Po dokončení obchodu se částka uvolní prodávajícímu a vytvoří se odpovídající ledger transakce. Při zrušení obchodu vzniká refundace a prostředky se vrací kupujícímu. Všechny tyto operace probíhají v databázové transakci společně se zápisem do ledgeru.

Peněženka proto nepracuje pouze s jednou přepisovanou hodnotou zůstatku. Aktuální balance slouží pro rychlou kontrolu dostupných prostředků, zatímco ledger uchovává přesnou historii změn. Tento návrh je vhodný zejména u části aplikace, která simuluje finanční logiku.

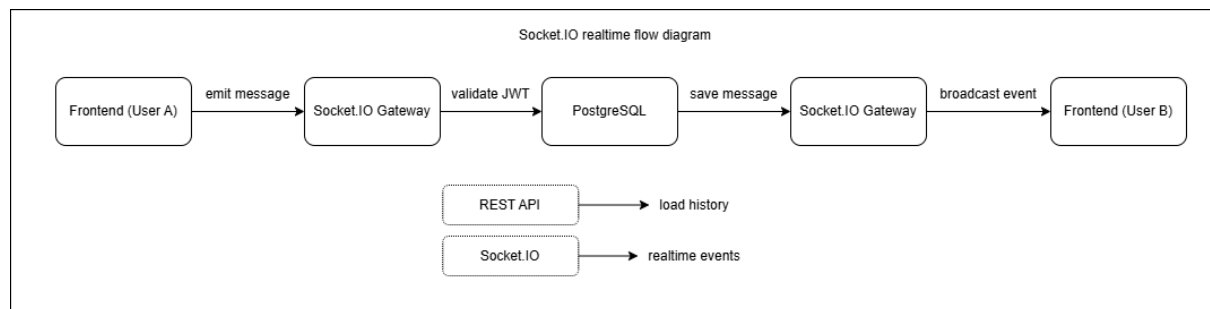
5.2.9 Chat, konverzace a zprávy

Komunikace mezi kupujícím a prodávajícím je řešena pomocí konverzací, zpráv a realtime vrstvy. Uživatelé tak mohou řešit dotazy k nabídce nebo detaily dodání přímo v aplikaci.

Konverzace představuje komunikační kanál mezi dvěma uživateli ve vztahu ke konkrétní nabídce nebo obchodu. Při vytvoření backend kontroluje, zda již odpovídající konverzace neexistuje, aby nevznikaly duplicitní chaty pro stejný účel.

Zprávy se ukládají do databáze a obsahují autora, text, čas vytvoření a případné přílohy. Díky trvalému ukládání může uživatel pokračovat v komunikaci i po obnovení stránky nebo novém přihlášení.

Historie konverzace se načítá pomocí REST API. Tento způsob je vhodný pro práci s uloženými daty a stránkování starších zpráv. Nové zprávy a okamžité události jsou přenášeny pomocí Socket.IO gateway v modulu chat.



Obrázek 15: Realtime komunikace pomocí Socket.IO gateway (vlastní zpracování)

Při odeslání zprávy frontend vyšle událost na Socket.IO server. Backend ověří identitu uživatele, zkontroluje účast v konverzaci, uloží zprávu do databáze a odešle ji příjemci. Pokud je příjemce připojen, zpráva se zobrazí okamžitě bez obnovení stránky.

Socket.IO se nepoužívá pouze pro chat. Stejná realtime vrstva přenáší také aktualizace inboxu, online stav uživatelů, systémové zprávy a změny stavu obchodů. REST API a WebSocket komunikace jsou tím odděleny podle účelu: REST slouží pro práci s uloženými daty, Socket.IO pro okamžité události.

Backend u konverzací kontroluje oprávnění. Uživatel musí být účastníkem dané konverzace, jinak nemůže číst její historii ani odesílat zprávy. Tato kontrola zabráňuje přístupu k cizí komunikaci.

5.2.10 Recenze a reputace uživatele

Modul recenzí je propojen s obchodním procesem a veřejnými profily prodávajících. Recenzi lze vytvořit pouze po dokončení obchodu, což omezuje možnost falešných hodnocení mimo skutečný obchodní vztah.

Recenze obsahuje číselné hodnocení a textový komentář. Hodnocení se promítá do reputace prodávajícího a zobrazuje se na jeho veřejném profilu. Textový komentář doplňuje informaci o průběhu obchodu, komunikaci nebo kvalitě dodání.

Backend při vytváření recenze kontroluje, zda obchod existuje, zda se ho aktuální uživatel účastnil a zda byl obchod dokončen. Zároveň brání vytvoření více recenzí ke stejnému obchodu. Po uložení nové recenze se aktualizuje reputace prodávajícího.

Reputace není založena pouze na jedné hodnotě. V profilu se pracuje s průměrným hodnocením, počtem hodnocení, dokončenými obchody a dalšími prvky, například

achievements nebo profilovými odznaky. Kupující tak získává lepší představu o dlouhodobé aktivitě a důvěryhodnosti prodávajícího.

```
13  async create(buyerId: string, dto: CreateReviewDto) {
14    return this.prisma.$transaction(async (tx) => {
15      const deal = await tx.deal.findUnique({
16        where: { id: dto.dealId },
17        select: { id: true, buyerId: true, sellerId: true, status: true },
18      });
19
20      if (!deal) throw new NotFoundException('Deal not found');
21      if (deal.buyerId !== buyerId)
22        throw new ForbiddenException('Not your deal');
23      if (deal.status !== 'COMPLETED')
24        throw new ForbiddenException('Deal is not completed');
25
26      const exists = await tx.review.findUnique({
27        where: { dealId: dto.dealId },
28      });
29      if (exists)
30        throw new ForbiddenException('Review already exists for this deal');
31
32      const review = await tx.review.create({
33        data: {
34          dealId: dto.dealId,
35          buyerId,
36          sellerId: deal.sellerId,
37          rating: dto.rating,
38          comment: dto.comment,
39        },
40      });
41
42      const updatedRows = await tx.$executeRaw`
43        UPDATE "User"
44        SET
45          "ratingCount" = "ratingCount" + 1,
46          "ratingAvg" = (("ratingAvg" * "ratingCount") + ${dto.rating}) / ("ratingCount" + 1)
47        WHERE "id" = ${deal.sellerId}
48      `;
```

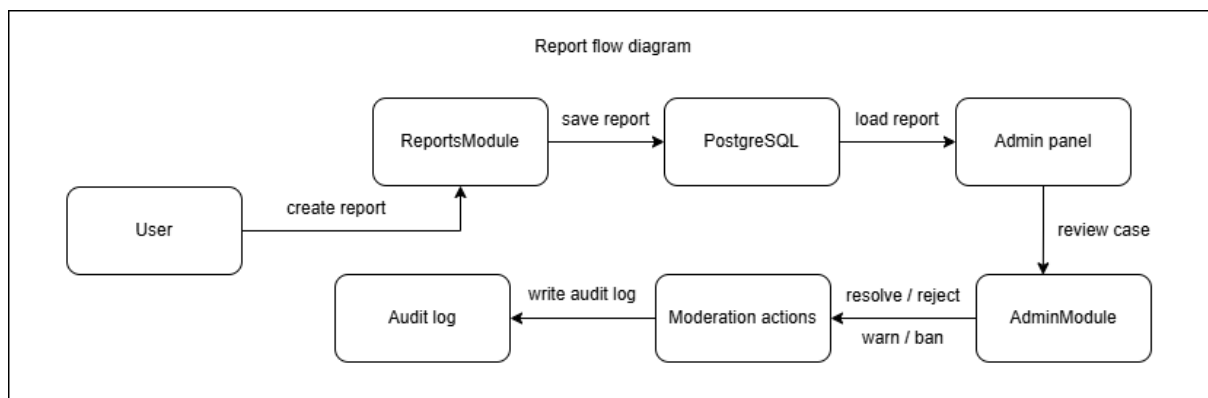
Obrázek 16: Validace recenzí a aktualizace reputace uživatele (vlastní zpracování)

Recenze se zobrazují hlavně na veřejném profilu prodávajícího a v detailu nabídky. Modul tím propojuje obchodní část aplikace s reputačním systémem a podporuje rozhodování kupujícího před vytvořením obchodu.

5.2.11 Reporty, moderace a administrace

Marketplace pracuje s uživatelským obsahem a obchodními interakcemi, proto backend obsahuje také moduly pro reportování a administraci. Reporty řeší hlášení problematických nabídek, uživatelů, recenzí, obchodů nebo zpráv.

Při vytvoření reportu backend ukládá typ cílového objektu, identifikátor cíle, důvod nahlášení, autora reportu a čas vytvoření. Zároveň ověřuje, že nahlášený objekt existuje. Administrátor poté může report zpracovat v administrátorském rozhraní, změnit jeho stav nebo provést moderátorský zásah.



Obrázek 17: Zpracování reportu v administrátorském systému (vlastní zpracování)

Administrátorský modul pokrývá správu uživatelů, rolí, zákazů, varování, nabídek, recenzí, reportů, achievementů a systémových oznámení. Běžný uživatel k těmto endpointům přístup nemá. Backend proto před provedením administrátorské akce ověřuje roli aktuálně přihlášeného uživatele.

```

13  @Injectable()
14  export class AdminGuard implements CanActivate {
15      canActivate(context: ExecutionContext): boolean {
16          const req = context.switchToHttp().getRequest<RequestWithUser>();
17
18          if (req.user?.role !== 'ADMIN') {
19              throw new ForbiddenException('Admin access required');
20          }
21
22          return true;
23      }
24  }

```

Obrázek 18: Kontrola administrátorských oprávnění pomocí AdminGuard (vlastní zpracování)

U citlivých administrátorských operací je důležitá dohledatelnost. Změna role, zablokování účtu, moderace reportu nebo odstranění obsahu mohou výrazně ovlivnit uživatele platformy. Proto backend vytváří auditní záznamy, které umožňují zpětně zjistit, kdo akci provedl a kdy k ní došlo.

Administrace není oddělená od zbytku aplikace pouze vizuálně. Je vynucena na úrovni backendu pomocí guardů a kontrol oprávnění. Tím se snižuje riziko, že by uživatel bez administrátorské role mohl provést moderátorský zásah přímým voláním API.

5.2.12 Notifikace, achievementy a doplňkové funkce

Backend obsahuje také funkce, které rozšiřují základní obchodní workflow. Patří mezi ně systémové notifikace, achievementy, profilové odznaky, automatické zprávy a vyhodnocování aktivity prodávajících.

Notifikace informují uživatele o důležitých událostech v aplikaci. Vznikají například při vytvoření obchodu, přijetí zprávy, změně stavu objednávky, dokončení transakce nebo

administrátorském oznámení. Ukládají se do databáze, takže uživatel může zobrazit jejich historii i po opětovném přihlášení.

Některé notifikace jsou zároveň doručovány realtime pomocí Socket.IO. To se hodí hlavně u událostí, které mají být uživateli zobrazeny okamžitě, například nová zpráva nebo aktualizace inboxu.

Achievements fungují jako motivační vrstva nad běžnou aktivitou uživatele. Mohou být přiděleny za dokončené obchody, dlouhodobou aktivitu, pozitivní hodnocení nebo další události. Některé achievements vznikají automaticky na základě aktivity, jiné může vytvořit nebo přiřadit administrátor.

Profilové odznaky umožňují uživateli zvýraznit vybrané achievements ve veřejném profilu nebo u nabídek. Tyto prvky nejsou nezbytné pro samotné dokončení obchodu, ale zlepšují prezentaci uživatele a podporují reputační vrstvu platformy.

Systém zároveň obsahuje mechanismus pro vyhodnocování nejlepších prodávajících. Tato část pracuje s aktivitou uživatelů, dokončenými obchody a hodnocením. Výsledkem může být odměna, achievement nebo systémová notifikace.

5.2.13 Nahrávání souborů

Upload souborů je využit u obrázků nabídek, avatarů uživatelů a příloh zpráv. Backend přijímá soubory přes požadavky typu multipart/form-data a ukládá je do lokálního serverového adresáře uploads.

Před uložením se kontroluje autentizace uživatele, oprávnění k dané operaci, MIME typ a velikost souboru. Tím se omezuje riziko nahrání nepodporovaného nebo nebezpečného obsahu.

```
54 @ApiOperation({ summary: 'Upload chat media (image/video)' })
55 @Post('upload-media')
56 @UseInterceptors(
57   FilesInterceptor('files', MESSAGE_MEDIA_MAX_FILES, {
58     storage: diskStorage({
59       destination: 'uploads/messages',
60       filename: (_req, file, cb) => {
61         const extension = extname(file.originalname || '').toLowerCase();
62         const normalizedExtension = extension.length > 0 ? extension : '.bin';
63         cb(
64           null,
65           `${Date.now()}-${Math.round(Math.random() * 1e9)}${normalizedExtension}`,
66         );
67       },
68     }),
69     limits: { fileSize: MESSAGE_MEDIA_MAX_FILE_SIZE_BYTES },
70     fileFilter: (_req, file, cb) => {
71       if (!ALLOWED_MEDIA_MIME_TYPES.has(file.mimetype)) {
72         cb(new BadRequestException('Unsupported media file type'), false);
73         return;
74       }
75       cb(null, true);
76     },
77   }),
78 )
79 uploadMedia(
```

Obrázek 19: Validace nahrávaných souborů při uploadu médií (vlastní zpracování)

Databáze neukládá binární obsah souborů. Ukládá pouze cestu k souboru a související metadata. Tento způsob snižuje zatížení databáze a zjednodušuje práci se statickými soubory.

Backend zároveň kontroluje vlastnictví souvisejících dat. Uživatel může měnit pouze avatar vlastního profilu nebo obrázky vlastních nabídek. U zpráv se kontroluje účast v konverzaci.

Lokální ukládání souborů odpovídá rozsahu bakalářské práce a lokálnímu Docker prostředí. Návrh je ale oddělen od ostatní business logiky, takže v budoucnu by bylo možné lokální úložiště nahradit cloudovým řešením bez zásadní změny doménových modulů.

5.3 Implementace frontendové části

Frontendová část aplikace je implementována pomocí knihovny React, jazyka TypeScript a nástroje Vite. Slouží jako uživatelské rozhraní marketplace platformy a propojuje veřejný katalog, autentizaci, správu účtu, obchodní proces, interní peněženku, chat a administrátorskou část.

Aplikace je vytvořena jako Single Page Application. Po načtení stránky se další navigace mezi částmi rozhraní provádí na straně klienta bez úplného obnovení dokumentu. Komunikace s backendem probíhá přes REST API a v částech vyžadujících okamžitou aktualizaci také pomocí Socket.IO.

Frontend neobsahuje hlavní business logiku obchodů nebo finančních operací. Tyto kontroly zůstávají v backendu. Klientská část se zaměřuje na zobrazení dat, zpracování vstupů uživatele, volání API, práci s relací a vizuální zpětnou vazbu během jednotlivých operací.

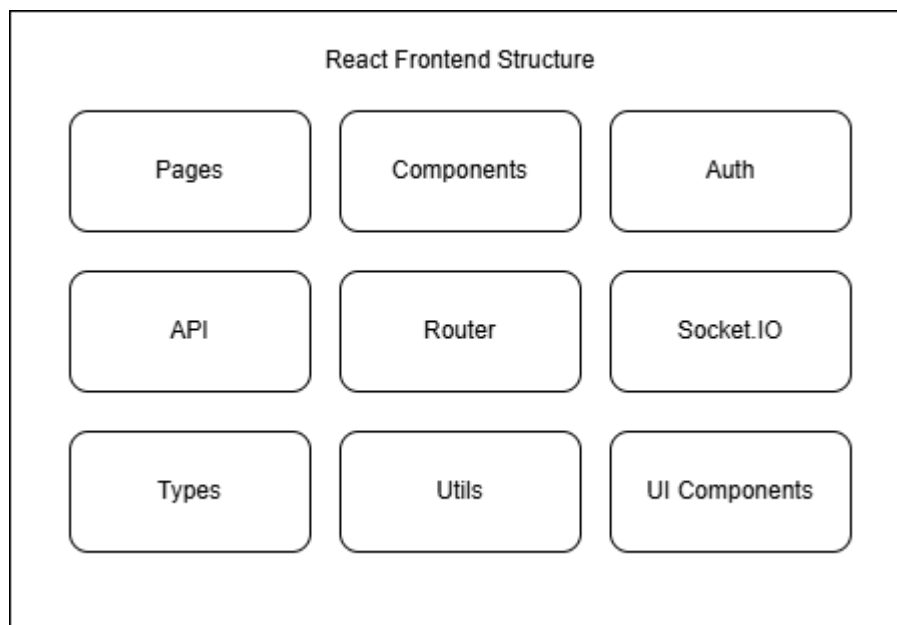
5.3.1 Struktura React aplikace

Zdrojové soubory frontendové části jsou umístěny v adresáři apps/frontend/src. Struktura je rozdělena podle odpovědnosti jednotlivých částí aplikace. Hlavní vstup do aplikace je v souboru main.tsx, routování je definováno v router.tsx a sdílená autentizační logika je umístěna v AuthContext.tsx.

Adresář pages obsahuje hlavní obrazovky aplikace. Patří sem katalog nabídek, detail nabídky, přihlášení, registrace, peněženka, inbox, detail obchodu, profil uživatele, veřejný profil prodávajícího, správa vlastních nabídek a administrátorské rozhraní.

Znovupoužitelné prvky uživatelského rozhraní jsou uloženy v adresáři components. Jsou zde například navigace, formuláře, karty nabídek, dialogová okna, prvky chatu, stavové badge, komponenty pro profil a části sdílené napříč více stránkami.

Komunikace s backendem je oddělena do adresáře api. Soubor http.ts definuje Axios klienta pro REST požadavky a socket.ts obsahuje klientskou konfiguraci Socket.IO. Tím se síťová komunikace nepropisuje přímo do všech komponent a lze ji spravovat na jednom místě.



Obrázek 20: Struktura frontendové části React aplikace (vlastní zpracování)

5.3.2 Routing a chráněné stránky

Směrování aplikace je řešeno knihovnou React Router. Router rozděluje obrazovky na veřejné, chráněné a administrátorské části.

Veřejné stránky jsou dostupné bez přihlášení. Jedná se například o katalog nabídek, detail nabídky, přihlášení, registraci, žebříček prodávajících a veřejný profil prodávajícího.

Chráněné stránky vyžadují přihlášeného uživatele. Patří sem peněženka, inbox, konverzace, detail obchodu, správa vlastních nabídek, nastavení profilu a stránky spojené s obchodním procesem. Přístup k nim je řízen komponentou `RequireAuth`, která ověřuje stav uživatelské relace.

Administrátorská část používá samostatnou ochranu pomocí `RequireAdmin`. Tato komponenta kontroluje nejen přihlášení, ale také roli uživatele. Pokud uživatel nemá administrátorské oprávnění, stránka administrace se mu nezobrazí.

```

76 export const router = createBrowserRouter([
77   {
78     element: <App />,
79     children: [
80       { path: "/", element: withSuspense(<ListingsPage />) },
81       { path: "/top-sellers", element: withSuspense(<TopSellersPage />) },
82       { path: "/listings/:id", element: withSuspense(<ListingPage />) },
83
84       { path: "/login", element: withSuspense(<LoginPage />) },
85       { path: "/register", element: withSuspense(<RegisterPage />) },
86       { path: "/verify-email", element: withSuspense(<VerifyEmailPage />) },
87       { path: "/forgot-password", element: withSuspense(<ForgotPasswordPage />) },
88       { path: "/reset-password", element: withSuspense(<ResetPasswordPage />) },
89
90       {
91         element: (
92           <RequireAuth>
93             <Outlet />
94           </RequireAuth>
95         ),
96         children: [
97           { path: "/deals", element: withSuspense(<DealsPage />) },
98           { path: "/deals/:id", element: withSuspense(<DealRoomPage />) },
99           { path: "/wallet", element: withSuspense(<WalletPage />) },
100          { path: "/conversations/:id", element: withSuspense(<ConversationPage />) },
101        ],
102      },
103      {
104        path: "/profile",
105        element: (
106          <RequireAuth>
107            {withSuspense(<ProfilePage />)}
108          </RequireAuth>
109        ),
110      },
111    ],
112  },
113 ],);

```

Obrázek 21: Implementace routingu a chráněných stránek ve frontendové části (vlastní zpracování)

Kontrola na frontendu slouží hlavně pro správné chování rozhraní a navigace. Skutečné ověření oprávnění provádí backend při každém požadavku. Uživatel tak nemůže obejít omezení pouze přímým voláním API.

5.3.3 API klient a práce s tokenem

Pro komunikaci s backendem je použit Axios klient definovaný v souboru http.ts. Klient má nastavenou základní adresu API a používá volbu withCredentials, aby se k požadavkům automaticky přikládala autentizační cookie.

JWT token se na straně klienta neukládá do localStorage. Po přihlášení jej backend uloží do httpOnly cookie. Frontend s tokenem nepracuje přímo, pouze odesílá požadavky s povolenými credentials. Tento způsob omezuje riziko přístupu k tokenu z JavaScriptu.

Autentizační stav aplikace spravuje AuthContext. Při spuštění aplikace se odešle požadavek na endpoint /auth/me, podle kterého frontend zjistí, zda je uživatel přihlášený. Stejná logika se používá po přihlášení, odhlášení nebo změně údajů účtu.

Při odhlášení frontend zavolá backendový endpoint pro zrušení relace, vyčistí lokální stav uživatele a odpojí websocketové spojení. Tím se zabrání situaci, kdy by po odhlášení zůstala aktivní realtime komunikace původního uživatele.

API vrstva je využívána napříč celou aplikací. Stránky přes ni načítají nabídky, obchody, konverzace, zprávy, transakce, profily, reporty i administrátorská data.

5.3.4 Autentizační obrazovky

Autentizační část obsahuje přihlášení, registraci, ověření e-mailu, opětovné odeslání ověřovacího odkazu a obnovu hesla. Tyto obrazovky tvoří vstupní bod pro práci s neveřejnými funkcemi aplikace.

Přihlašovací formulář přijímá e-mail a heslo. Po odeslání frontend zavolá endpoint pro přihlášení a při úspěšné odpovědi obnoví stav aktuálního uživatele přes AuthContext. Uživatel je následně přesměrován do aplikace.

Obrázek 22: Přihlašovací obrazovka aplikace (vlastní zpracování)

Registrační formulář sbírá základní údaje nového účtu. Frontend provádí základní kontrolu vstupů, například vyplnění povinných polí nebo shodu hesel. Finální validace ale probíhá na backendu, kde se kontroluje také unikátnost e-mailu a bezpečné uložení hesla.

TradeMarket

📋 Listings👤 Top Sellers

→ Login

👤 Register

TradeMarket

Create an account

Join the secure gaming marketplace

Display name

👤 Your username

Email

✉ your@email.com

Password

🔒 Create a password

👁

Confirm password

🔒 Confirm your password

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

One account works for both buying and selling.

Create account

Already have an account? [Sign in](#)

Buyers and sellers share one account model.

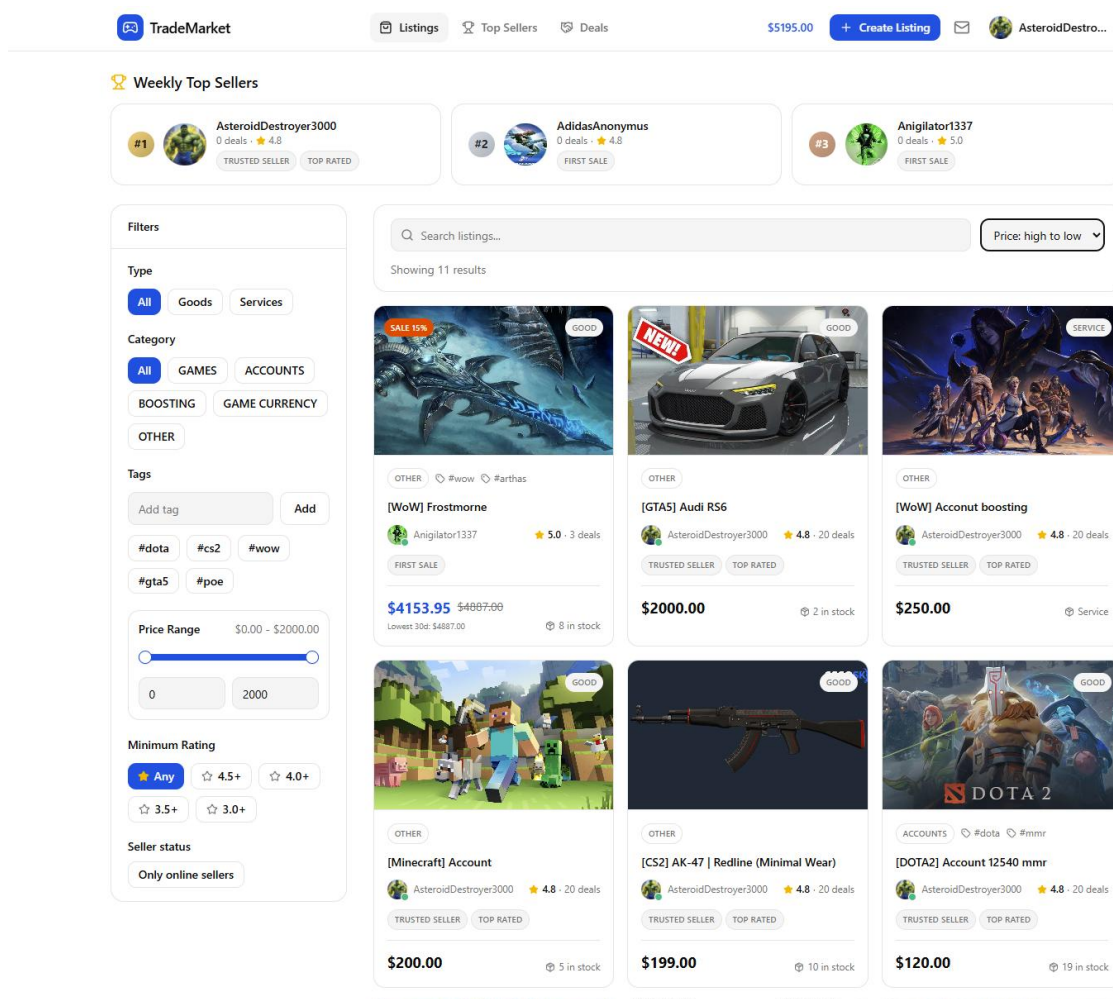
Obrázek 23: Registrační obrazovka aplikace (vlastní zpracování)

Po registraci je uživatel informován o nutnosti ověření e-mailové adresy. Obrazovka pro ověření zpracovává token z odkazu a podle odpovědi backendu zobrazí výsledek. Obnova hesla funguje podobně: uživatel zadá e-mail, obdrží odkaz a následně nastaví nové heslo.

Autentizační obrazovky obsahují také zobrazení chybových stavů. Frontend informuje uživatele například o neplatných přihlašovacích údajích, neověřeném účtu, expirovaném tokenu nebo chybě při odeslání požadavku.

5.3.5 Katalog a detail nabídky

Katalog nabídek představuje hlavní veřejnou část aplikace. Zobrazuje aktivní digitální produkty a služby, se kterými může uživatel dále pracovat. Nabídky jsou zobrazeny formou karet obsahujících název, cenu, kategorii, typ položky, obrázek a základní informace o prodávajícím.



Obrázek 24: Katalog nabídek marketplace aplikace (vlastní zpracování)

Stránka katalogu podporuje filtrování a řazení. Uživatel může vyhledávat podle textu, filtrovat podle kategorie, typu nabídky, ceny nebo hodnocení prodávajícího. Parametry se promítají do API požadavku, takže backend vrací pouze odpovídající výsledky.

V katalogu se zobrazují také informace spojené s reputací prodávajícího. Uživatel tak nevidí pouze samotnou nabídku, ale i kontext prodeje. U nabídek se slevou frontend zvýrazňuje původní a aktuální cenu.

Detail nabídky obsahuje podrobnější popis produktu nebo služby, cenu, dostupnost, informace o prodávajícím, historii ceny, recenze a akce dostupné podle role uživatele. Kupující zde může zahájit konverzaci nebo vytvořit obchod.

TradeMarket

Listings

Top Sellers

Deals

\$5195.00

Create Listing

AsteroidDestro...

Back to listings

SALE 15%

GOOD

[WoW] Frostmorne

Frostmorne — The Legendary Blade of the Lich King

Own one of the most iconic weapons in fantasy gaming history — Frostmorne, the cursed runeblade wielded by the Lich King himself. A symbol of absolute power, darkness, and domination, this weapon is perfect for collectors, roleplayers, PvP enthusiasts, or anyone who wants to stand out in-game.

What You Get

Legendary Frostmorne

Unique visual appearance & prestige

Perfect for transmogs / showcases / collectors

A true piece of gaming history

Why Buy Frostmorne?

"Whomsoever takes up this blade shall wield power eternal."

Frostmorne is more than just a weapon — it's a status symbol. Instantly recognizable by veteran players and fans alike, this blade brings unmatched atmosphere and legendary aesthetics to your character.

Advantages

✓ Fast delivery

✓ Professional communication

✓ Support after purchase if needed

Become the Lich King

OTHER

wow

arthas

lichking

Anigilator1337

5.0 (3 reviews)

\$4887.00

\$4153.95

GOOD

Sale ends: 14.05.2026, 10:00:00

Price History

0.0%

MinPriceOnSales

\$4153.95

MinPriceNoSales

\$4887.00

Change

\$0.00

Anigilator1337

Online

Report

RELATED LISTING

[WoW] Frostmorne

\$4887.00

\$4153.95

Absolutely. It's very iconic and recognizable — perfect for collectors, PvP, and giving your character a legendary look.

20:28:40

Sounds awesome 🤩 Is the transaction safe?

20:28:49

Yes, everything is quick and secure. I'll also be available if you have any questions after the purchase.

20:28:56

I've always loved the Lich King and Northrend theme.

20:29:04

Then Frostmorne will fit perfectly 🌟 It's truly one of the most iconic blades in Warcraft history.

20:29:12

Great, I'll order it then.

20:29:13

Type a message...

Send

TOTAL

\$4153.95

QUANTITY

1

8 in stock

Buy now

Report listing

Report this listing

Obrázek 25: Detail nabídky s informacemi o produktu a prodávajícím (vlastní zpracování)

Pokud je uživatel vlastníkem nabídky, rozhraní místo nákupních akcí zobrazí možnosti pro úpravu, archivaci nebo obnovení nabídky. Frontend tak přizpůsobuje dostupné akce aktuálnímu uživateli, ale vlastnické kontroly zůstávají i na backendu.

Detail nabídky je propojen s obchodním workflow. Při nákupu uživatel zvolí množství a odešle požadavek na vytvoření obchodu. Backend následně ověří dostupnost, cenu, vlastnictví a zůstatek kupujícího.

5.3.6 Správa nabídek

Správa nabídek je určena prodávajícímu. Uživatel zde vytváří nové nabídky, upravuje existující položky, archivuje neaktivní nabídky a může je znovu obnovit.

60

Formulář nabídky pracuje s názvem, popisem, kategorií, typem položky, cenou, tagy, dostupným množstvím a obrázkem. U digitálních služeb se některá pole chovají jinak než u položek typu zboží, například práce se skladovým množstvím.

TradeMarket Listings Top Sellers Deals \$5195.00 + Create Listing AsteroidDestro...

Edit listing

Title
[CS2] AK-47 | Redline (Minimal Wear)

Description
 🔥 Why Choose Redline?
 The Redline skin is a fan favorite for its clean lines, subtle carbon-fiber detailing, and striking red accents. It's instantly recognizable, giving your AK-47 a professional and stylish edge. Whether you're climbing ranks, streaming, or just flexing in your inventory, Redline delivers.
 ✅ Advantages
 ✓ Fast delivery
 ✓ Professional communication
 ✓ Support after purchase if needed
 ⭐ Stand Out on the Battlefield
 Equip the AK-47 | Redline and let your style speak as loudly as your firepower. Minimal Wear ensures the skin looks great without compromising that battle-hardened vibe.

Listing image
 Выберите файл Файл не выбран

Type **Category**
 GOOD Other

Tags
 Type tag and press Enter

Price (USD) **Quantity in stock**
 199,00 10

Flash Sale ☒
 Allowed base price range: \$193.03 - \$204.97
 Discount range: 5% - 70%
Discount percent
 e.g. 20
Sale starts at **Sale ends at**
 DD.MM.YYYY --:--:-- DD.MM.YYYY --:--:--
 Fill all three fields to activate sale.

Save changes

Obrázek 26: Formulář pro vytvoření nebo úpravu nabídky (vlastní zpracování)

Cena je na frontendu zadávána v běžném formátu pro uživatele. Před odesláním se převede do formátu používaného backendem. Tím se odděluje uživatelský vstup od interní reprezentace částek.

Formulář podporuje také nahrání obrázku nabídky. Soubor se odesílá na samostatný upload endpoint a po úspěšném nahrání se získaná cesta použije při vytvoření nebo úpravě nabídky.

Při úpravě nabídky lze nastavit slevu. Frontend zobrazuje pole pro výši slevy a časové omezení akce. Pravidla pro platnost slevy kontroluje backend, ale rozhraní pomáhá uživateli zadat hodnoty ve správném formátu.

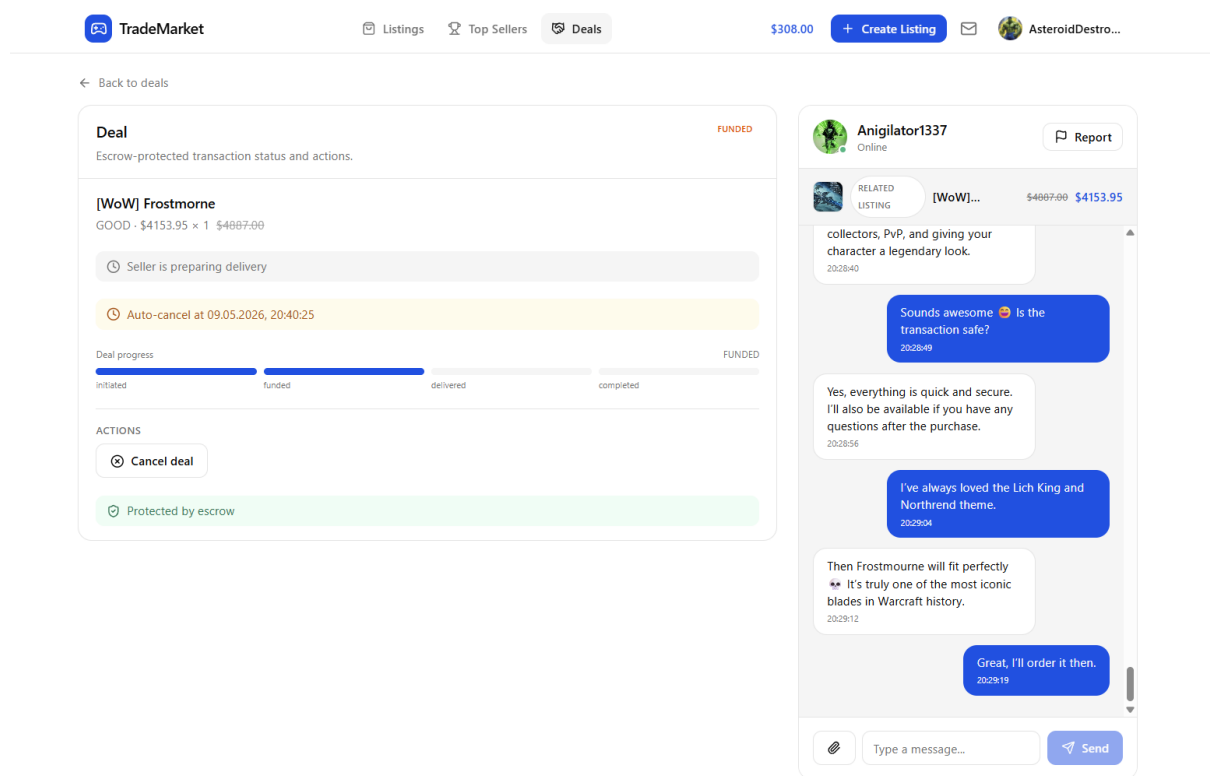
Stránka vlastních nabídek rozděluje položky podle stavu. Prodávající tak vidí aktivní i archivované nabídky a může rychle provést potřebnou akci. Archivace je použita místo trvalého odstranění, aby se zachovala návaznost na obchody a historii.

5.3.7 Obchody a peněženka

Obchodní část frontendové aplikace zobrazuje seznam obchodů, detail konkrétního obchodu, stav escrow procesu a dostupné akce podle role uživatele.

Seznam obchodů umožňuje rozlišit nákupy a prodeje. Každá položka zobrazuje základní informace o nabídce, protistraně, ceně, stavu a datu vytvoření. Uživatel tak získá rychlý přehled o probíhajících i dokončených obchodech.

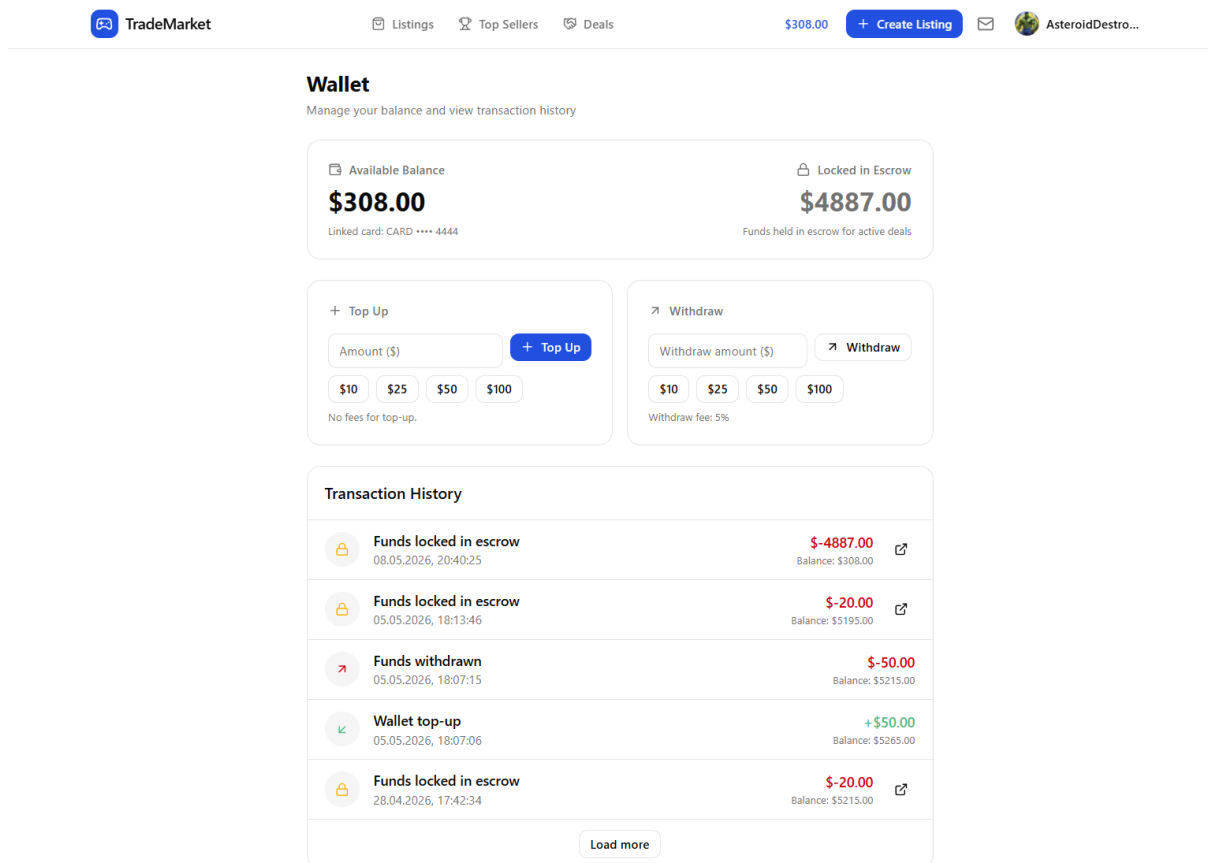
Detail obchodu zobrazuje celý aktuální stav procesu. Kupující vidí, zda jsou prostředky uzamčeny v escrow, zda prodávající označil dodání a zda je možné obchod dokončit nebo zrušit. Prodávající na stejné stránce vidí vlastní dostupné akce, například potvrzení dodání.



Obrázek 27: Detail obchodu a escrow procesu (vlastní zpracování)

Rozhraní se mění podle stavu obchodu. Jiná tlačítka jsou dostupná u financovaného obchodu, jiná po dodání a jiná po dokončení nebo zrušení. Frontend tím pomáhá uživateli pochopit aktuální fázi procesu, zatímco backend rozhoduje, zda je daná akce skutečně povolena.

Peněženka zobrazuje aktuální disponibilní zůstatek, uzamčené prostředky a historii transakcí. Uživatel může provést demonstrační dobítí nebo výběr prostředků. Výběr zahrnuje zobrazení poplatku a výsledné částky.



Obrázek 28: Rozhraní interní peněženky uživatele (vlastní zpracování)

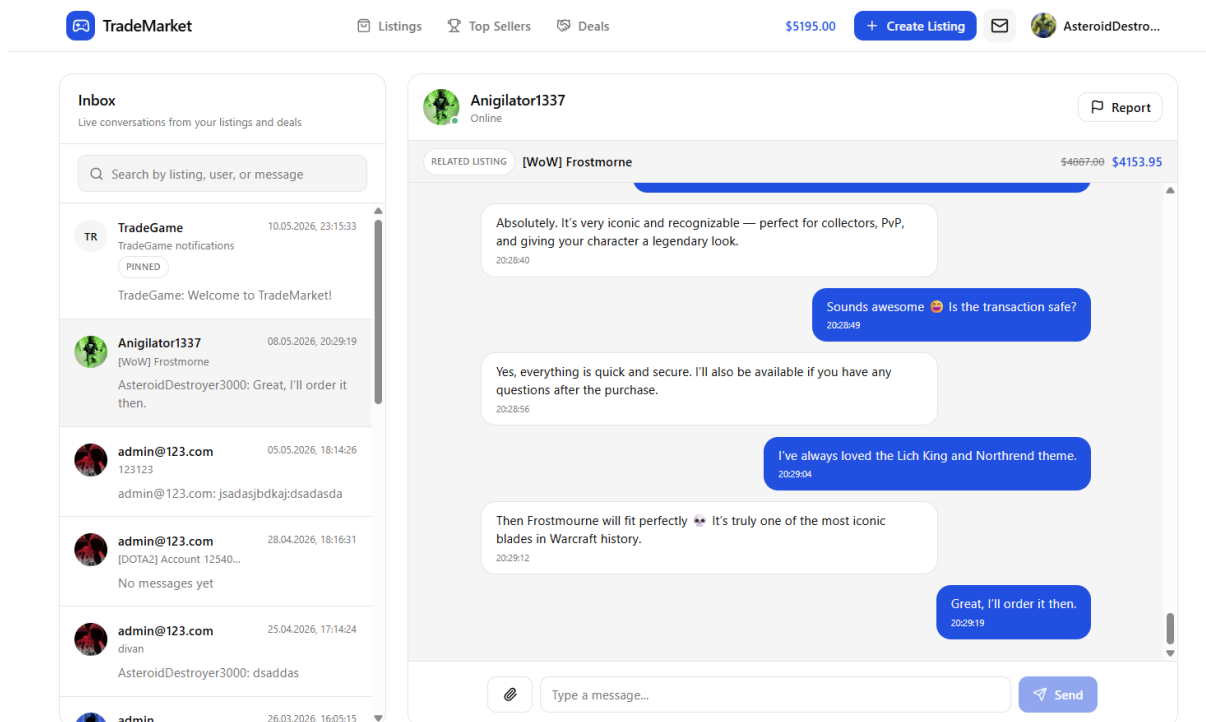
Historie transakcí obsahuje typ operace, částku, datum, stav a případnou vazbu na obchod. Uživatel tak může dohledat, kdy došlo k dobití, uzamčení prostředků, refundaci nebo uvolnění platby prodávajícímu.

Frontend zde plní hlavně prezentační roli. Výpočet dostupného zůstatku, uzamčení prostředků, refundace a uvolnění platby probíhá v backendu v databázových transakcích.

5.3.8 Chat a inbox

Chatová část propojuje frontend s realtime vrstvou backendu. Uživatel může komunikovat s protistranou v rámci konkrétní nabídky nebo obchodu, posílat textové zprávy a pracovat s přílohami.

Inbox zobrazuje seznam konverzací. U každé položky je uveden název související nabídky, protistrana, poslední zpráva, čas poslední aktivity a počet nepřečtených zpráv. Konverzace se aktualizují přes Socket.IO, takže nová zpráva se v inboxu projeví bez obnovení stránky.



Obrázek 29: Inbox a realtime chat mezi uživateli (vlastní zpracování)

Samotná konverzace využívá komponentu `ConversationView`. Po otevření se uživatel připojí do místnosti odpovídající dané konverzaci. Při opuštění stránky se z místnosti odpojí. Tím se omezuje doručování událostí pouze na relevantní uživatele.

Odeslání zprávy probíhá přes websocketovou událost. Frontend předá text a případná média, backend ověří oprávnění, uloží zprávu a odešle ji účastníkům konverzace. Starší zprávy se načítají přes REST API, což zjednodušuje stránkování historie.

Chat podporuje také práci s médii. Uživatel může před odesláním nahrát soubory, které se následně připojí ke zprávě. Frontend zobrazuje náhledy příloh a umožňuje jejich odstranění před odesláním.

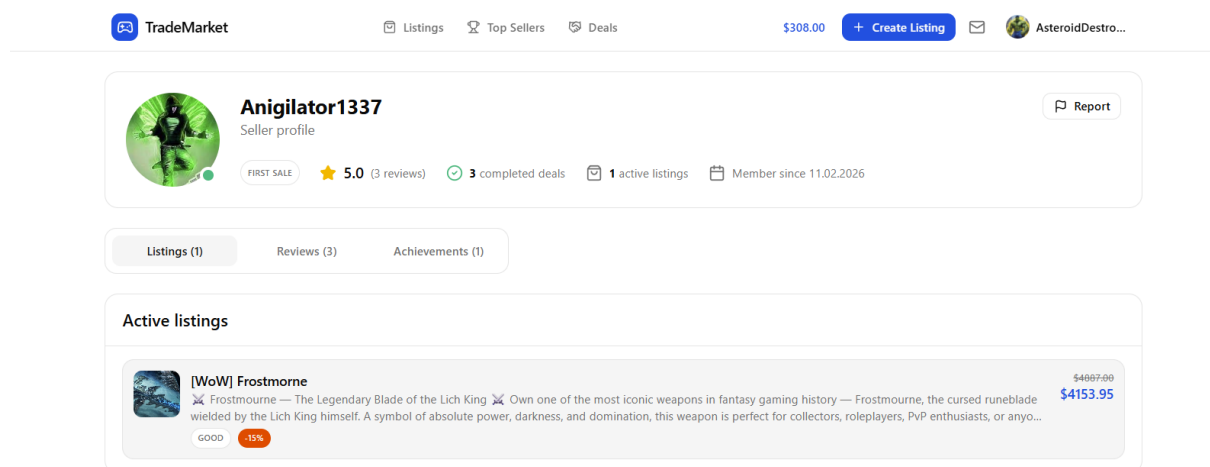
Inbox obsahuje i systémovou konverzaci pro oznámení aplikace. Ta slouží například pro administrátorské zprávy, informace o obchodech nebo systémové události. Díky tomu jsou běžné uživatelské konverzace a systémové zprávy oddělené, ale dostupné v jednom rozhraní.

5.3.9 Uživatelský profil a veřejný profil prodávajícího

Frontend rozlišuje soukromý profil přihlášeného uživatele a veřejný profil prodávajícího. Soukromá část slouží pro správu účtu, zatímco veřejný profil prezentuje důvěryhodnost uživatele ostatním návštěvníkům.

V nastavení profilu může uživatel upravit zobrazované jméno, avatar, platební údaje, heslo a vybrané profilové prvky. Rozhraní je rozděleno do tematických částí, aby nastavení účtu, bezpečnost a vzhled profilu nebyly smíchané v jedné obrazovce.

Veřejný profil prodávajícího zobrazuje údaje důležité pro obchodní rozhodnutí kupujícího. Patří sem hodnocení, počet recenzí, aktivní nabídky, achievementy, profilové odznaky a online stav.



Obrázek 30: Veřejný profil prodávajícího (vlastní zpracování)

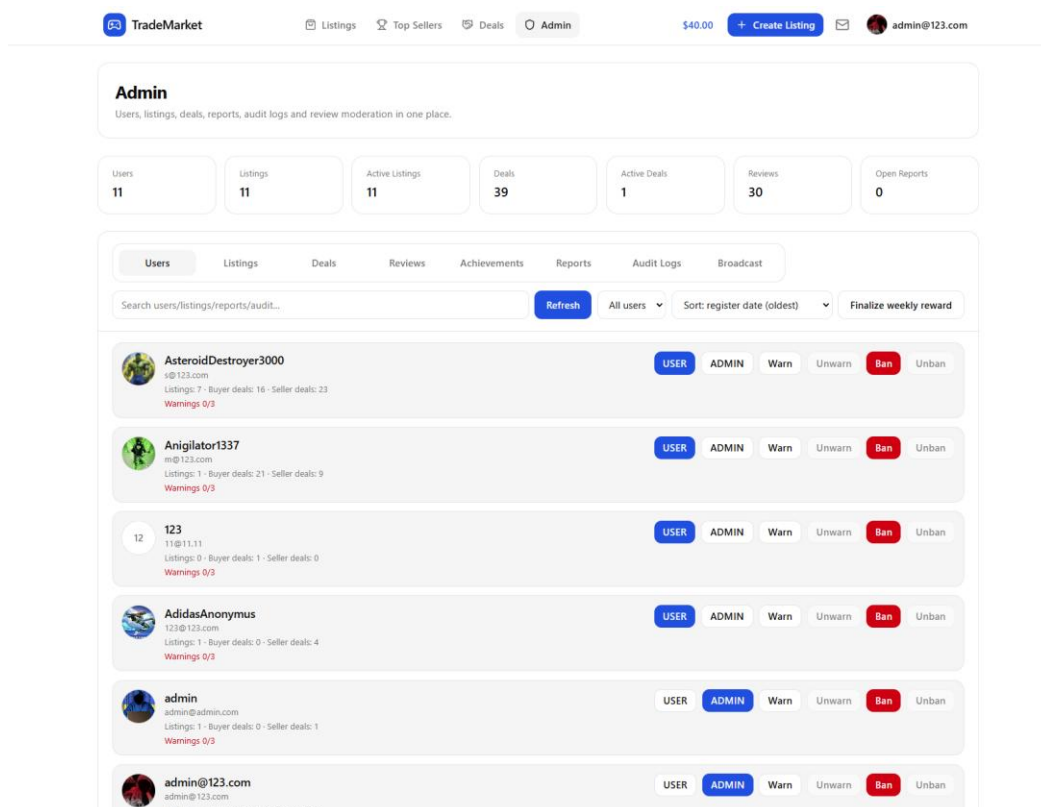
Profil je propojen s katalogem i detailem nabídky. Kupující se tak může před nákupem podívat na předchozí aktivitu prodávajícího a lépe vyhodnotit riziko obchodu.

Achievementy a odznaky mají v profilu prezentační funkci. Uživatel si může vybrat, které získané odznaky chce zobrazovat. Frontend tyto prvky používá v profilu a u nabídek, aby byla reputace prodávajícího viditelná i mimo detailní profil.

Veřejný profil zároveň obsahuje možnost nahlášení uživatele. Report se po odeslání předá backendu a dále jej řeší administrátor v moderátorském rozhraní.

5.3.10 Administrátorské rozhraní

Administrátorská část soustředí nástroje pro správu a moderaci platformy. Přístup k ní je omezen pomocí RequireAdmin a dostupné akce znovu kontroluje backend.



Obrázek 31: Administrátorské rozhraní marketplace aplikace (vlastní zpracování)

Rozhraní obsahuje přehled uživatelů, nabídek, obchodů, recenzí, reportů, auditních záznamů, achievementů a systémových oznámení. Administrátor může vyhledávat v datech, zobrazit detail vybraného záznamu a provést moderátorský zásah.

Práce s reporty patří mezi hlavní části administrace. Administrátor vidí nahlášený objekt, důvod reportu a související kontext. Podle situace může změnit stav reportu, archivovat nabídku, zablokovat uživatele, odstranit avatar, smazat recenzi nebo provést jinou dostupnou akci.

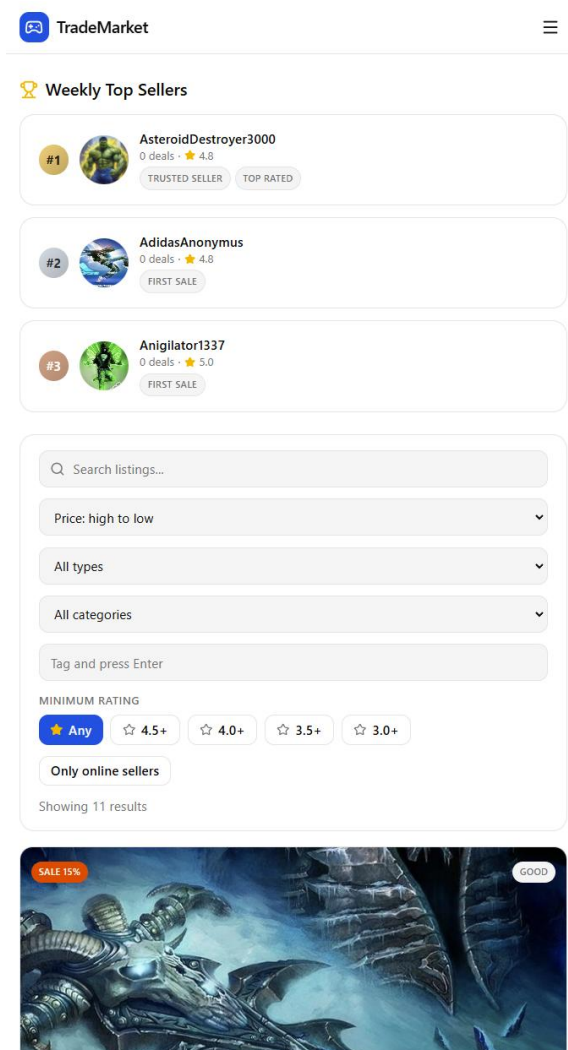
Administrace zahrnuje také správu achievementů. Administrátor může vytvářet nové achievementy, upravovat jejich parametry a ručně je přidělovat uživatelům. Dále může rozesílat systémová oznámení, která se uživatelům zobrazují v inboxu.

Frontend u administrace klade důraz na přehlednost, protože na jedné stránce pracuje s více typy dat. Jednotlivé oblasti jsou proto odděleny do samostatných pohledů nebo sekcí. Citlivé operace se odesílají na backendové endpointy, kde se provede finální autorizace a případné zapsání auditního záznamu.

5.3.11 Stylování a responzivita

Stylování frontendové části je řešeno pomocí Tailwind CSS a vlastních znovupoužitelných komponent. Rozhraní je navrženo tak, aby zvládalo větší množství dat, stavů a uživatelských akcí bez ztráty přehlednosti.

Aplikace používá jednotné vizuální prvky pro tlačítka, formuláře, karty, modální okna, stavové štítky a prázdné stavy obrazovek. Tím se omezuje duplicita a jednotlivé části aplikace působí konzistentně.



Obrázek 32: Responzivní zobrazení frontendové části aplikace (vlastní zpracování)

Responzivita je řešena úpravou rozložení podle velikosti obrazovky. Na desktopu se více využívá vícesloupcové uspořádání, zatímco na menších obrazovkách se prvky řadí pod sebe. Tento princip je použit například v katalogu, detailu nabídky, profilu, peněženke a administraci.

Stavy obchodů, transakcí, reportů nebo nabídek jsou vizuálně odlišeny pomocí badge prvků a barevného zvýraznění. Uživatel tak rychle pozná, zda je obchod financovaný, dodaný, dokončený nebo zrušený.

Při návrhu rozhraní byl kladen důraz hlavně na srozumitelnost workflow. Marketplace aplikace obsahuje více navazujících procesů, například nákup, escrow, chat, recenzi nebo report. Frontend proto zobrazuje aktuální stav, dostupné akce a zpětnou vazbu po dokončení operace.

5.4 Implementace databázové vrstvy

Databázová vrstva je postavena na PostgreSQL a Prisma ORM. Uchovává data uživatelů, nabídek, obchodů, peněženek, ledger transakcí, konverzací, zpráv, recenzí, reportů, notifikací a administrátorských záznamů.

Datový model odpovídá modulární struktuře backendu. Každá hlavní oblast aplikace má vlastní sadu entit a vazeb, ale všechny části pracují nad jednou relační databází. To je výhodné zejména

u obchodní a finanční logiky, kde je nutné provádět více navazujících změn v jedné databázové transakci.

Prisma v projektu funguje jako vrstva mezi NestJS aplikací a PostgreSQL databází. Backend díky ní pracuje s databázovými entitami přes typově bezpečný klient, zatímco samotná struktura tabulek, vztahů, enumů a indexů je popsána v souboru `schema.prisma`.

5.4.1 Prisma schema

Hlavní datový model je definován v souboru `schema.prisma`. Tento soubor obsahuje modely databázových entit, jejich atributy, relační vazby, enum typy, unikátní omezení a indexy.

Každý model odpovídá jedné hlavní tabulce nebo podpůrné entitě aplikace. Například model `User` reprezentuje uživatele, `Listing` nabídku, `Deal` obchod, `Wallet` interní peněženku a `WalletTransaction` ledger záznam. Komunikační část je pokryta modely `Conversation` a `Message`.

Součástí schématu jsou také enum typy. Ty určují povolené hodnoty pro role uživatelů, typy nabídek, stavy obchodů, typy transakcí, stavy reportů nebo typy nahlášeného obsahu. Použití enumů omezuje ukládání neplatných stavů a pomáhá udržet konzistenci dat.

Prisma na základě schématu generuje klienta pro TypeScript. Backend tak při práci s databází využívá typy odvozené přímo z datového modelu. To snižuje riziko chyb způsobených nesouladem mezi databází a aplikačním kódem.

5.4.2 Hlavní entity systému

Centrální entitou je `User`. Ukládá autentizační údaje, profilové informace, roli uživatele, stav blokace, reputační hodnoty a vazby na další části aplikace. Uživatel může vystupovat jako kupující, prodávající nebo administrátor.

Nabídky jsou reprezentovány modelem `Listing`. Obsahují název, popis, cenu, typ nabídky, kategorii, obrázek, tagy, dostupné množství, stav a případné slevové informace. Každá nabídka je propojena s prodávajícím. Pro sledování změn cen je vytvořen model `ListingPriceHistory`.

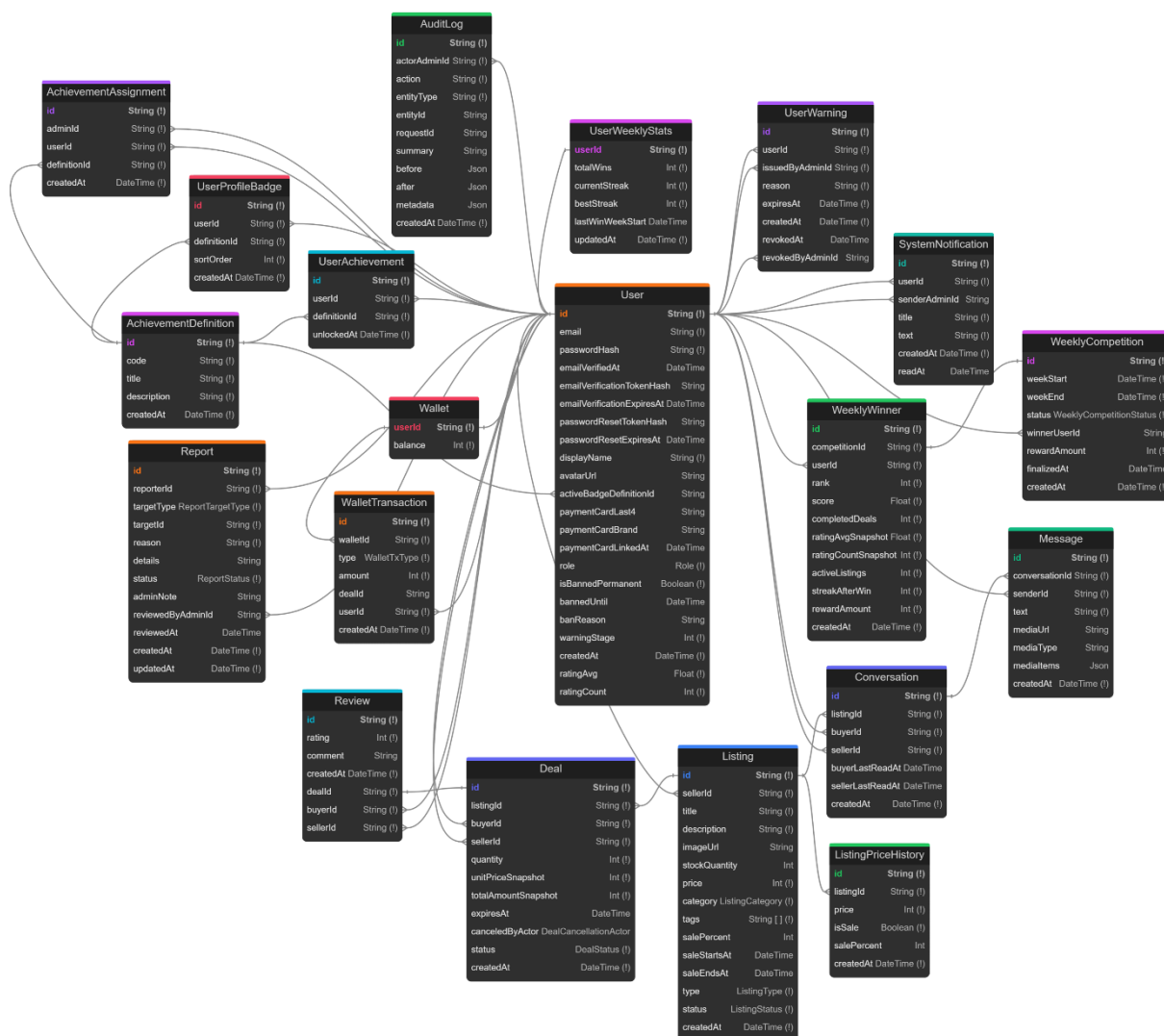
Obchodní proces je uložen v modelu `Deal`. Ten propojuje kupujícího, prodávajícího a konkrétní nabídku. Kromě stavu obchodu ukládá také množství, cenový snapshot, celkovou částku, čas expirace a případně informaci o tom, kdo obchod zrušil. Cenový snapshot je důležitý, protože nabídka může po vytvoření obchodu změnit cenu.

Finanční část tvoří modely `Wallet` a `WalletTransaction`. Peněženka uchovává aktuální zůstatek uživatele, zatímco ledger zaznamenává jednotlivé změny zůstatku. Mezi typy transakcí patří například dobití, výběr, escrow lock, escrow release, refund nebo weekly reward. Částky jsou ukládány jako celá čísla v nejmenších jednotkách měny, aby se předešlo problémům s desetinnými čísly.

Komunikace mezi uživateli je řešena modely `Conversation` a `Message`. Konverzace je navázána na nabídku, kupujícího a prodávajícího. Zprávy obsahují autora, text, čas vytvoření a případná média. Model konverzace zároveň ukládá informace potřebné pro počítání nepřečtených zpráv.

Recenze jsou uloženy v modelu `Review` a jsou navázány na dokončený obchod. Reporty používají model `Report`, který umožňuje nahlásit nabídku, uživatele, recenzi, obchod nebo

zprávu. Administrátorskou část doplňuje AuditLog, kde se ukládají vybrané zásahy administrátora.



Obrázek 33: ER diagram hlavních entit systému (vlastní zpracování)

5.4.3 Vazby mezi entitami

Datový model využívá relační vazby mezi hlavními doménovými objekty. Uživatel je propojen s nabídkami, obchody, recenzemi, peněženkou, zprávami, reporty, notifikacemi a administrátorskými záznamy.

Vazba mezi uživatelem a peněženkou je navržena jako vztah 1:1. Penženka používá userId jako primární klíč, takže jeden uživatel má právě jednu interní penženku.

Vztah mezi nabídkou a obchodem je 1:N. Jedna nabídka může být použita ve více obchodech, ale každý obchod je vytvořen z jedné konkrétní nabídky. Obchod je zároveň propojen s kupujícím a prodávajícím, což umožňuje rozlišit roli uživatele v daném procesu.

Recenze je navázána na obchod pomocí unikátní vazby. Jeden obchod tak může mít nejvýše jednu recenzi. Tím se zabraňuje opakovanému hodnocení stejné transakce.

Konverzace je propojena s nabídkou, kupujícím a prodávajícím. V databázi je nastavena unikátní kombinace listingId a buyerId, aby mezi stejným kupujícím a prodávajícím k jedné nabídce nevznikaly duplicitní konverzace.

Achievementsy a profilové odznaky používají spojovací entity, například UserAchievement a UserProfileBadge. Ty propojují uživatele s definicemi achievementů a umožňují oddělit samotnou definici od konkrétního získání uživatelem.

5.4.4 Transakční data a auditovatelnost

Nejcitlivější částí databázového modelu jsou obchody a finanční operace. V těchto místech nestačí pouze uložit aktuální stav. Je potřeba zachovat i historii změn a zajistit, aby navazující operace proběhly společně.

Při vytvoření obchodu se ukládá stav obchodu, účastníci, množství a cenový snapshot. Pokud je obchod financován, backend zároveň upraví zůstatek peněženky kupujícího a vytvoří ledger transakci typu ESCROW_LOCK. Tyto změny probíhají v jedné databázové transakci.

Při dokončení obchodu se stav změní na dokončený, prostředky se uvolní prodávajícímu a do ledgeru se zapíše odpovídající transakce. Při zrušení financovaného obchodu vzniká refundace a prostředky se vrací kupujícímu.

Ledger umožňuje zpětně dohledat, proč se změnil zůstatek uživatele. Aktuální hodnota v peněžence slouží pro rychlé zobrazení a kontrolu dostupných prostředků, zatímco WalletTransaction uchovává historii jednotlivých operací.

Auditovatelnost je použita také u administrátorských zásahů. Model AuditLog ukládá administrátora, typ akce, dotčenou entitu, čas provedení a podle potřeby také stav před změnou a po změně. To je důležité hlavně u moderace, blokování uživatelů, změn rolí nebo zásahů do obsahu.

5.4.5 Migrace a inicializace databáze

Databázové změny jsou spravovány pomocí Prisma migrací. Každá migrace popisuje konkrétní změnu schématu, například vytvoření tabulky, přidání sloupce, úpravu vazby nebo vytvoření indexu.

Migrace jsou uloženy v adresáři prisma/migrations. Díky tomu lze sledovat vývoj databázového modelu v čase a spustit stejnou strukturu databáze v jiném prostředí. To je důležité při lokálním vývoji i při spuštění aplikace v Dockeru.

Při startu backendového kontejneru se migrace aplikují na PostgreSQL databázi. Následně se vygeneruje Prisma klient, který odpovídá aktuální podobě schématu. Backend tak pracuje s databázovým modelem, který je sladěný se zdrojovým kódem aplikace.

Inicializace databáze je navázána na Docker Compose konfiguraci. Databázový kontejner PostgreSQL se spustí jako samostatná služba a backend se k němu připojuje pomocí hodnot z environment proměnných.

5.4.6 Indexy a optimalizace dotazů

Datový model obsahuje indexy u polí, která se často používají při filtrování, řazení nebo vyhledávání souvisejících záznamů. Cílem není předčasná optimalizace všech tabulek, ale podpora dotazů, které aplikace skutečně provádí.

U nabídek jsou indexy použity například pro prodávajícího a kategorii. Katalog tak může rychleji filtrovat nabídky podle vlastníka nebo tematické oblasti.

Historie cen používá index nad kombinací nabídky a času vytvoření. To je vhodné při načítání vývoje ceny a při kontrole minimální ceny za poslední období.

U konverzací a zpráv jsou indexy zaměřeny na uživatele, čas vytvoření a příslušnost ke konverzaci. Tyto dotazy se používají při načítání inboxu, starších zpráv a nepřečtených zpráv.

Obchody používají indexy nad účastníky, stavem a časovými údaji. To pomáhá při zobrazení obchodů kupujícího nebo prodávajícího a při práci s časově omezenými obchody.

Reporty a auditní záznamy jsou indexovány podle stavu, typu cíle, administrátora, akce a času vytvoření. Administrátorské rozhraní tak může efektivněji zobrazovat otevřené reporty nebo historii zásahů.

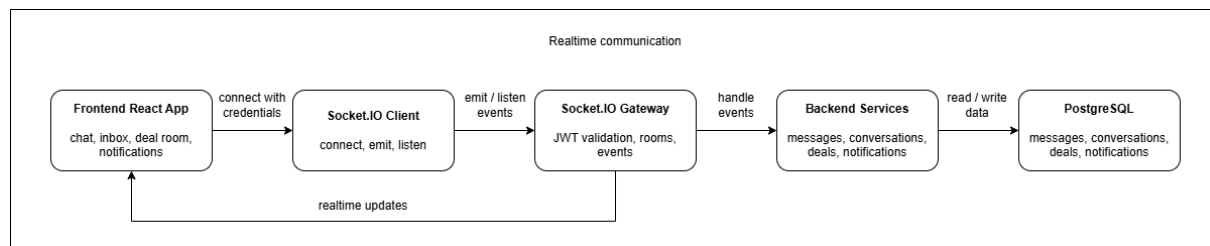
Indexy byly zvoleny podle reálných přístupových vzorů aplikace: katalog, detail nabídky, inbox, obchodní proces, ledger, reporty a administrace. Databázový model tím podporuje nejen ukládání dat, ale i běžné dotazy, které aplikace při provozu provádí.

5.5 Realtime komunikace

Realtime komunikace je v aplikaci použita tam, kde běžné načítání dat přes REST API nestačí. Týká se hlavně chatu, aktualizace inboxu, online stavu uživatelů, systémových zpráv a změn stavu obchodů.

Implementace je založena na Socket.IO gateway v backendovém modulu chat. REST API zůstává určeno pro načítání uložených dat, například historie zpráv, detailu obchodu nebo seznamu konverzací. Socket.IO přenáší pouze okamžité události, které mají být zobrazeny bez obnovení stránky.

Gateway neobsahuje hlavní business logiku. Přijímá websocketové události, ověřuje připojeného uživatele, volá příslušné service vrstvy a doručuje výsledek konkrétním klientům. Ukládání zpráv, kontrola účasti v konverzaci nebo změna stavu obchodu zůstávají v doménových službách.



Obrázek 34: Princip realtime komunikace mezi frontendem a backendem (vlastní zpracování)

5.5.1 Socket.IO gateway

Socket.IO gateway je umístěna v souboru chat.gateway.ts. Při navázání spojení backend ověří JWT token a spojí websocketové připojení s konkrétním uživatelem. Token lze načíst z cookie nebo z autorizačních údajů požadavku, stejně jako u běžné autentizace v backendu.

Po úspěšném připojení je klient zařazen do uživatelské místnosti ve tvaru user:<id>. Díky tomu může backend posílat události pouze konkrétnímu uživateli, například aktualizaci inboxu, systémovou zprávu nebo změnu stavu obchodu.

Pro chat se používají místnosti ve tvaru conversation:<id>. Po otevření konverzace frontend odešle událost conversation:join a klient se připojí do odpovídající místnosti. Při opuštění konverzace odešle conversation:leave. Tím se doručování zpráv omezuje na uživatele, kteří s danou konverzací pracují.

Gateway zároveň eviduje aktivní připojení uživatelů. Pokud má uživatel alespoň jedno aktivní websocketové spojení, je považován za online. Při odpojení posledního spojení se jeho stav změnil na offline.

5.5.2 Události chatu

Chatová komunikace je založena na události message:send. Frontend při odeslání zprávy předá identifikátor konverzace, text a případně informace o přiložených médiích. Backend poté ověří uživatele, zkontroluje jeho účast v konverzaci, uloží zprávu do databáze a rozešle ji účastníkům.

Nová zpráva se klientům doručuje událostí message:new do místnosti conversation:<id>. Otevřená konverzace se tak aktualizuje okamžitě. Inbox se zároveň obnoví pomocí události inbox:update, která se posílá do uživatelské místnosti příjemce.

Historie zpráv se nenačítá přes Socket.IO. Starší zprávy, stránkování a detail konverzace zůstávají v REST API. Websocketová vrstva tím zůstává zaměřená pouze na nové události, ne na kompletní správu uložené historie.

U zpráv s médii frontend nejprve nahraje soubory přes upload endpoint. Do websocketové události se poté předávají metadata již uložených médií. Backend podporuje i více příloh v jednom odeslaném sdělení.

Kontrola oprávnění probíhá při každé chatové události. Uživatel nemůže poslat zprávu do konverzace, ve které není účastníkem. Stejně pravidlo platí i pro připojení do místnosti konverzace.

5.5.3 Online stav a inbox

Online stav se odvozuje z aktivních websocketových spojení. Jeden uživatel může mít otevřenou aplikaci ve více kartách nebo zařízeních, proto gateway pracuje s počtem připojení, ne pouze s jednou hodnotou online/offline.

Frontend může ověřit stav konkrétního uživatele pomocí události presence:check. Backend vrátí informaci, zda je daný uživatel aktuálně online. Při změně stavu se ostatním klientům posílá událost presence:update.

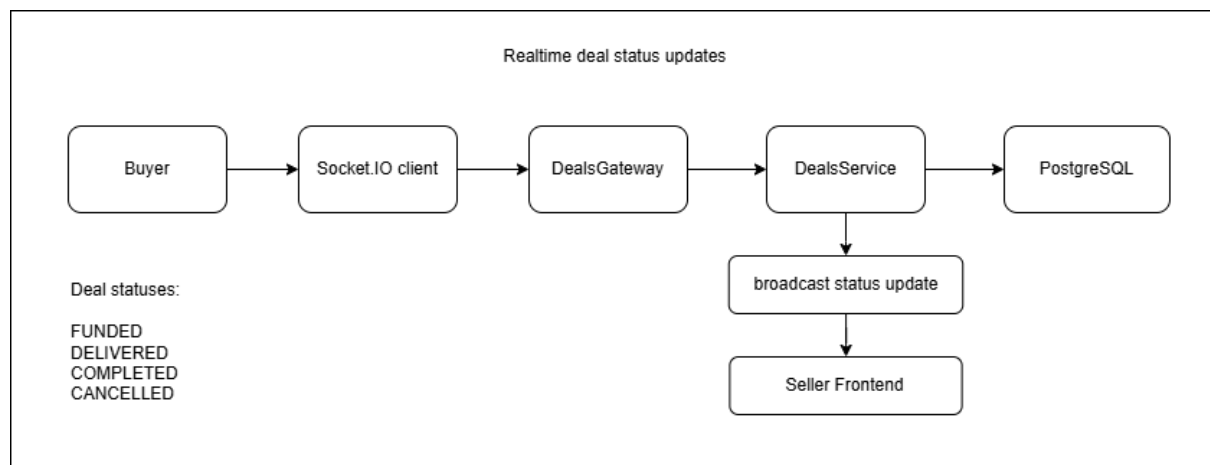
Inbox se aktualizuje při nových zprávách, systémových oznámeních a dalších událostech souvisejících s komunikací. Nová zpráva tedy nezmění pouze otevřený chat, ale také pořadí konverzací, náhled poslední zprávy a počet nepřečtených zpráv.

Systémové zprávy používají stejnou realtime infrastrukturu jako běžné konverzace, ale jsou odděleny jako zvláštní systémový tok. Backend může uživateli poslat například administrátorské oznámení nebo informaci o změně v aplikaci pomocí událostí `system:message:new` a `system:inbox:refresh`.

5.5.4 Aktualizace stavu obchodu

Realtime vrstva se používá také u obchodního workflow. Změny stavu obchodu se mají projevit oběma stranám bez ručního obnovení stránky, protože kupující i prodávající mohou být ve stejné chvíli v detailu obchodu nebo v chatu.

Při změně obchodu backend nejprve provede samotnou operaci v příslušné service vrstvě. Může jít například o označení dodání, dokončení obchodu, zrušení nebo refundaci. Po úspěšném uložení změny odešle událost `deal:update` do uživatelských místností kupujícího a prodávajícího.



Obrázek 35: Realtime aktualizace stavu obchodu pomocí Socket.IO (vlastní zpracování)

Frontend po přijetí `deal:update` obnoví stav zobrazeného obchodu. Uživatel tak ihned vidí, zda je obchod financovaný, dodaný, dokončený nebo zrušený. Rozhraní podle nového stavu upraví dostupná tlačítka a informační prvky.

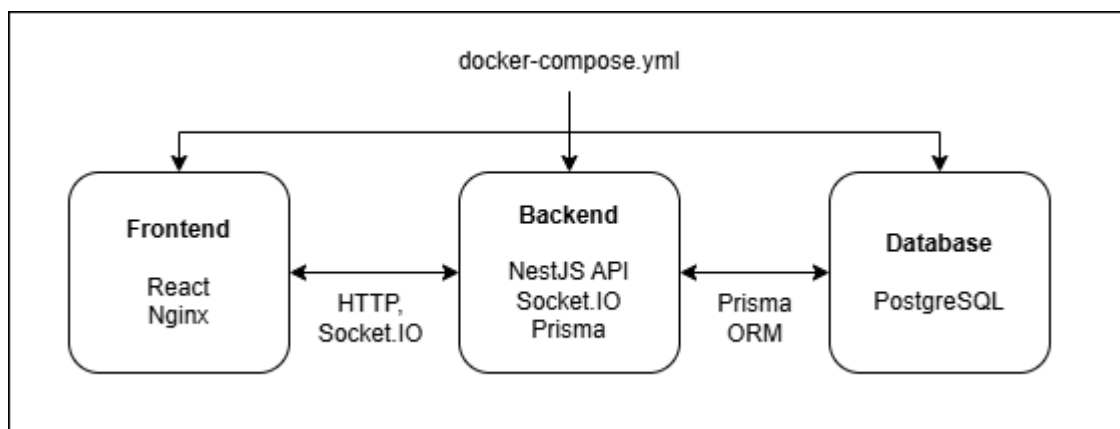
Socket.IO zde neslouží jako zdroj pravdy. Skutečný stav obchodu je uložen v databázi a mění se pouze přes backendovou business logiku. Realtime událost pouze informuje klienty, že došlo ke změně a že mají aktualizovat zobrazená data.

Toto rozdělení je důležité hlavně u escrow procesu. Finanční operace, změna stavu obchodu a ledger záznam probíhají v backendu v databázové transakci. Teprve po úspěšném dokončení se změna odešle klientům přes websocketovou vrstvu.

5.6 Docker a spuštění aplikace

Aplikace je připravena pro lokální spuštění pomocí Dockeru a Docker Compose. Kontejnerizované prostředí obsahuje tři hlavní služby: databázi PostgreSQL, backendovou aplikaci a frontendovou aplikaci distribuovanou pomocí Nginx serveru.

Docker zde neslouží pouze jako pomocný nástroj pro spuštění databáze. Celé prostředí je popsáno v jednom konfiguračním souboru, takže backend, frontend i databáze používají jednotné nastavení a lze je spustit společně. To zjednodušuje lokální vývoj i ověření funkčnosti aplikace mimo vývojové prostředí IDE.



Obrázek 36: Kontejnerizovaná architektura aplikace pomocí Docker Compose (vlastní zpracování)

Na obrázku 36 je znázorněna základní struktura kontejnerizovaného prostředí. Frontend je dostupný přes Nginx, backend poskytuje REST API a Socket.IO komunikaci a databáze ukládá perzistentní data aplikace. Backend zároveň zpřístupňuje nahrané soubory z adresáře uploads.

5.6.1 Docker Compose

Hlavní konfigurace je umístěna v souboru `docker-compose.yml`. Definuje služby db, backend a frontend, jejich porty, proměnné prostředí, závislosti, healthchecky a persistentní volumes.

Databázová služba používá image `postgres:16`. Data nejsou uložena pouze uvnitř kontejneru, ale v persistentním volume `marketplace_pgdata`. Díky tomu se databáze neztratí při zastavení nebo opětovném vytvoření kontejneru.

Backendový kontejner se sestavuje z adresáře `apps/backend`. Po spuštění se připojuje k databázi přes interní Docker síť a poskytuje API na portu 3000. Jeho dostupnost se ověřuje pomocí endpointu `/health`. Backend se spouští až po úspěšném healthchecku databáze, aby při startu nepracoval s nedostupným PostgreSQL serverem.

Frontendový kontejner se sestavuje z adresáře `apps/frontend`. Výsledná React aplikace je po buildu obsluhována pomocí Nginx serveru a zpřístupněna na portu 8080. Frontend čeká na dostupný backend, protože pro správné fungování potřebuje přístup k API i websocketovému serveru.

Konfigurace používá také volume `marketplace_uploads`. To uchovává uživatelské soubory nahrané přes backend, například avatary, obrázky nabídek nebo přílohy zpráv. Uploady tak nejsou svázány pouze s životním cyklem jednoho backendového kontejneru.

5.6.2 Konfigurace prostředí

Konfigurace aplikace je oddělena od zdrojového kódu pomocí environment proměnných. Backend využívá zejména připojení k databázi, port aplikace, režim běhu, JWT konfiguraci, CORS nastavení a údaje pro e-mailovou komunikaci.

V Docker Compose je databázová adresa nastavena tak, aby backend nepřístupoval k PostgreSQL přes localhost, ale přes název služby db v interní Docker síti. Tím se propojení mezi kontejnery řídí přímo názvy služeb definovanými v compose konfiguraci.

Frontend používá proměnné `VITE_API_URL` a `VITE_WS_URL`. Ty určují adresu backendového REST API a websocketového serveru. Na rozdíl od backendových proměnných se tyto hodnoty zapracují už během buildu frontendové aplikace, proto musí být nastaveny před sestavením frontendového kontejneru.

Soubor `.env.example` slouží jako vzor pro vytvoření lokální konfigurace. Skutečné hodnoty se ukládají do lokálních `.env` souborů, které nejsou určeny pro verzování. Tím se oddělují citlivé údaje od veřejné části projektu.

5.6.3 Lokální spuštění

Po stažení projektu je nejprve nutné připravit konfigurační soubory podle `.env.example`.

Docker Compose vytvoří databázový kontejner, sestaví backend a frontend a spustí služby ve správném pořadí. Nejprve se ověří dostupnost PostgreSQL, poté se spustí backend a nakonec frontendová část.

Backend při startu aplikuje databázové migrace a spustí produkční build NestJS aplikace. Po úspěšném startu je dostupné REST API, Socket.IO komunikace, health endpoint a statické soubory z adresáře `uploads`.

Frontend je po spuštění dostupný ve webovém prohlížeči na portu 8080. Odtud lze ověřit hlavní části aplikace, například registraci, přihlášení, katalog nabídek, vytvoření obchodu, escrow workflow, peněženku, chat a administrátorské rozhraní.

Pokud není odstraněno persistentní volume, databázová data a nahrané soubory zůstávají zachovány i po zastavení kontejnerů.

6 Testování

Testování bylo zaměřeno na ověření hlavních funkcí marketplace platformy a na kontrolu scénářů, ve kterých může vzniknout nekonzistentní stav. Nešlo pouze o kontrolu zobrazení jednotlivých stránek, ale hlavně o ověření obchodního workflow, práce s interní peněženkou, escrow logiky, oprávnění uživatelů a realtime komunikace.

V projektu byly použity automatizované backendové testy, základní end-to-end testy backendového API a manuální testování uživatelských scénářů. Automatizované testy pokrývají hlavně servisní vrstvy, kde je umístěna business logika. Frontend, chat a vybrané realtime scénáře byly ověřovány praktickým průchodem aplikace ve více uživatelských rolích.

Testování probíhalo průběžně během implementace. Po dokončení jednotlivých modulů byly ověřovány nejen samostatné funkce, ale také jejich propojení s ostatními částmi aplikace. Největší pozornost byla věnována autentizaci, správě nabídek, obchodům, peněžence, ledger transakcím, chatu, reportům a administrátorským funkcím.

6.1 Cíle testování

Hlavním cílem testování bylo ověřit, že aplikace správně zpracovává hlavní uživatelské, obchodní a administrátorské scénáře. Vzhledem k zaměření práce bylo nutné testovat nejen běžné operace, ale také chybové stavy, kontrolu oprávnění a návaznost více kroků v jednom procesu.

U obchodní logiky bylo ověřováno vytvoření obchodu, uzamčení prostředků v escrow mechanismu, označení dodání, dokončení obchodu, uvolnění prostředků prodávajícímu a refundace při zrušení. Tyto scénáře jsou pro aplikaci klíčové, protože mění stav obchodu, peněženky i ledger záznamů.

Dalším cílem bylo ověřit, že uživatel může pracovat pouze s daty, ke kterým má oprávnění. Testy a manuální kontroly se proto zaměřily na chráněné endpointy, vlastnictví nabídek, účast v obchodu, přístup ke konverzacím a administrátorská oprávnění.

Testování se zaměřilo také na použitelnost hlavních obrazovek. Ověřeno bylo, zda lze projít základní workflow od registrace přes vytvoření nabídky až po dokončení obchodu, zobrazení transakce v peněžence a vytvoření recenze.

6.2 Backendové testování

Backendové testování bylo realizováno pomocí frameworku Jest. Testy se zaměřily hlavně na servisní vrstvy jednotlivých modulů, protože právě zde jsou implementována pravidla aplikace, validační kontroly, práce se stavy a návaznost na databázové operace.

Automatizované unit testy byly připraveny pro moduly autentizace, nabídek, obchodů, peněženky, recenzí a health endpoint aplikace. Tyto části pokrývají hlavní funkce backendu a zároveň scénáře, ve kterých je nutné správně reagovat na chybové vstupy nebo neplatný stav systému.

U autentizace byly testovány scénáře registrace, přihlášení, práce s neplatnými údaji, duplicitním e-mailem a neověřeným uživatelem. Testy zároveň ověřují, že backend správně reaguje v případě, kdy uživatel neexistuje nebo není možné načíst jeho profil.

V modulu nabídek bylo ověřováno načítání katalogu, filtrování, vytvoření nabídky, úprava, archivace a obnovení. Důležitou část tvořily vlastnické kontroly. Uživatel nesmí měnit nabídku, která patří jinému prodávajícímu, bez ohledu na to, zda se mu daná akce zobrazuje ve frontendové části.

Největší důraz byl kladen na moduly obchodů a peněženky. Testy ověřovaly vytvoření obchodu, kontrolu účastníků, změny stavů, blokaci prostředků v escrow, uvolnění částky prodávajícímu a refundaci kupujícímu. U těchto scénářů bylo nutné sledovat nejen samotný stav obchodu, ale také změny zůstatků a vznik odpovídajících ledger transakcí.

```
116 describe('lockEscrow', () => {
117   let txEscrow: WalletTxMock;
118
119   beforeEach(() => {
120     txEscrow = {
121       user: { findUnique: jest.fn() },
122       wallet: { upsert: jest.fn(), update: jest.fn() },
123       walletTransaction: { create: jest.fn() },
124     };
125   });
126
127   it('should lock funds when balance is sufficient', async () => {
128     txEscrow.wallet.upsert.mockResolvedValue({
129       userId: 'u1',
130       balance: 10000,
131     });
132
133     await service.lockEscrow(
134       txEscrow as unknown as Prisma.TransactionClient,
135       'u1',
136       'deal1',
137       5000,
138     );
139
140     expect(txEscrow.wallet.update).toHaveBeenCalledWith({
141       where: { userId: 'u1' },
142       data: { balance: { decrement: 5000 } },
143     });
144     expect(txEscrow.walletTransaction.create).toHaveBeenCalledWith({
145       data: {
146         walletId: 'u1',
147         userId: 'u1',
148         type: 'ESCROW_LOCK',
149         amount: -5000,
150         dealId: 'deal1',
151       },
152     });
153   });
154 }
```

Obrázek 37: Unit test escrow operace ve wallet service pomocí frameworku Jest (vlastní zpracování)

U recenzí bylo ověřováno, že hodnocení lze vytvořit pouze k dokončenému obchodu a že jeden obchod nemůže mít více recenzí od stejného kupujícího. Testována byla také aktualizace reputace prodávajícího po vytvoření nové recenze.

End-to-end testy backendového API ověřovaly základní dostupnost aplikace, health endpoint, validační pravidla autentizačních endpointů, veřejné endpointy nabídek, ochranu chráněných

tras a vybrané scénáře práce s nabídkami a recenzemi. Tyto testy sloužily jako kontrola, že jednotlivé části backendu spolu správně komunikují přes HTTP rozhraní.

```

180 describe('Auth Flow', () => {
181   it('POST /auth/register - should register new user', async () => {
182     prisma.user.findUnique.mockResolvedValue(null);
183     prisma.user.create.mockResolvedValue(mockUser);
184
185     const response = await request(app.getHttpServer())
186       .post('/auth/register')
187       .send({
188         email: 'test@test.com',
189         password: '123456',
190         displayName: 'Test User',
191       })
192       .expect(201);
193
194     expect(response.body.emailVerificationRequired).toBe(true);
195     expect(response.body.user.email).toBe('test@test.com');
196   });
197
198   it('POST /auth/register - should reject duplicate email', async () => {
199     prisma.user.findUnique.mockResolvedValue(mockUser);
200
201     return request(app.getHttpServer())
202       .post('/auth/register')
203       .send({
204         email: 'test@test.com',
205         password: '123456',
206         displayName: 'Test',
207       })
208       .expect(400);
209   });
210
211   it('POST /auth/register - should validate input', () => {
212     return request(app.getHttpServer())

```

Obrázek 38: End-to-end test autentizačního endpointu backendového API (vlastní zpracování)

Tabulka 7: Přehled vybraných testovacích scénářů backendové části

Oblast testování	Testovaný scénář	Typ testu	Očekávaný výsledek
Autentizace	Registrace nového uživatele	Unit / E2E	Uživatel je vytvořen a systém připraví ověření e-mailu
Autentizace	Přihlášení s neplatnými údaji	Unit / E2E	Backend odmítne přihlášení a nevrátí autentizační token
Autorizace	Přístup k chráněnému endpointu bez autentizace	E2E	Požadavek je odmítnut
Nabídky	Úprava cizí nabídky	Unit	Backend odmítne operaci z důvodu nedostatečného oprávnění

Nabídky	Archivace a obnovení vlastní nabídky	Unit	Stav nabídky je správně změněn
Obchody	Vytvoření obchodu s nedostatečným zůstatkem	Unit	Obchod není vytvořen a zůstatek uživatele se nezmění
Obchody	Změna stavu obchodu neoprávněným uživatelem	Unit	Backend odmítne operaci
Escrow	Blokace prostředků kupujícího	Unit	Zůstatek kupujícího je snížen a vznikne ledger záznam
Escrow	Dokončení obchodu	Unit	Prostředky jsou uvolněny prodávajícímu
Escrow	Zrušení financovaného obchodu	Unit	Prostředky jsou vráceny kupujícímu
Recenze	Vytvoření recenze po dokončení obchodu	Unit	Recenze je vytvořena a reputace prodávajícího je aktualizována
Recenze	Opakované hodnocení stejného obchodu	Unit	Druhá recenze je odmítnuta
Health endpoint	Kontrola dostupnosti aplikace a databáze	E2E	Endpoint vrací stav aplikace a databáze

Zdroj: (vlastní zpracování)

6.3 Frontendové testování

Frontendová část byla ověřována především manuálním průchodem hlavních uživatelských scénářů. V projektu zatím není samostatná sada automatizovaných frontendových unit nebo end-to-end testů, proto bylo testování zaměřeno na praktické ověření chování aplikace v prohlížeči.

Testování začínalo veřejnou částí aplikace. Ověřeno bylo načtení katalogu, filtrování nabídek, zobrazení detailu nabídky, veřejný profil prodávajícího a základní responzivní chování rozhraní.

U autentizačních obrazovek byly kontrolovány scénáře registrace, přihlášení, odhlášení, ověření e-mailu, obnova hesla a chybové stavy při neplatných údajích. Součástí kontroly bylo také ověření, že po přihlášení aplikace správně načte aktuálního uživatele a zpřístupní chráněné stránky.

U prodávajícího bylo testováno vytvoření nabídky, úprava údajů, nahrání obrázku, archivace a obnovení položky. U kupujícího bylo ověřeno vytvoření obchodu, práce s detailem obchodu, zobrazení escrow stavu, dokončení objednávky a následné vytvoření recenze.

Peněženka byla testována z pohledu zobrazení zůstatku, uzamčených prostředků a historie transakcí. Manuálně bylo ověřeno, že se po obchodních operacích mění údaje v rozhraní podle odpovědi backendu.

Administrátorská část byla kontrolována samostatně. Ověřeno bylo zobrazení přehledů, práce s reporty, blokace uživatele, změna role, správa achievementů a rozeslání systémového oznámení. Důležité bylo také ověřit, že běžný uživatel administrátorské rozhraní nevidí a že backend chrání příslušné endpointy i při přímém volání API.

6.4 Testování realtime komunikace

Realtime komunikace byla ověřována praktickým testováním ve více klientech. Scénáře byly prováděny s různými uživateli, aby bylo možné zkontrolovat chování chatu, inboxu, online stavu a aktualizací obchodů.

U chatu bylo testováno vytvoření nebo načtení konverzace, připojení do websocketové místnosti, odeslání zprávy, okamžité doručení druhé straně a načtení historie po obnovení stránky. Kontrolováno bylo také chování inboxu, zejména aktualizace poslední zprávy a počtu nepřečtených zpráv.

Online stav byl ověřován pomocí přihlášení a odhlášení uživatele ve více oknech prohlížeče. Cílem bylo zjistit, zda se stav uživatele správně mění při navázání a ukončení websocketového spojení.

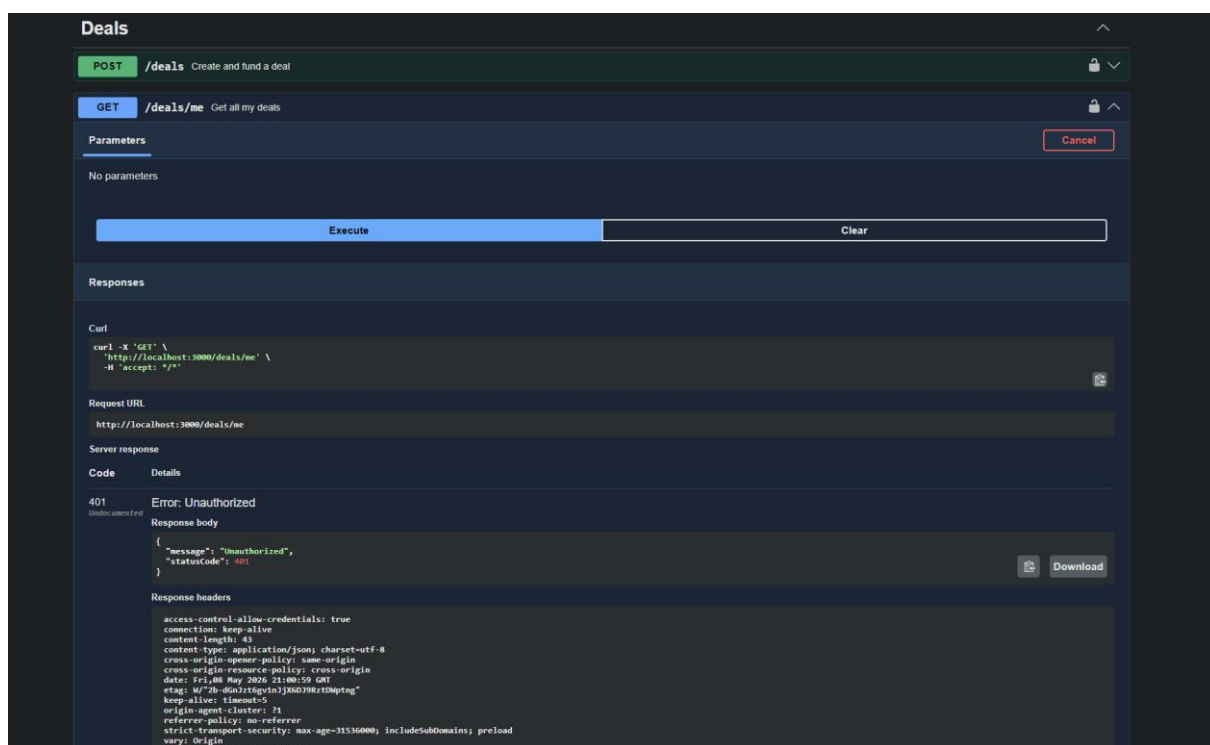
U obchodů bylo ověřeno, že změna stavu se zobrazí oběma stranám bez ručního obnovení stránky. Testovány byly hlavně situace, kdy prodávající označí dodání, kupující dokončí obchod nebo je obchod zrušen.

Systémové notifikace byly kontrolovány v návaznosti na administrátorské oznámení a události v aplikaci. Ověřeno bylo jejich zobrazení v inboxu a doručení přihlášenému uživateli přes realtime vrstvu.

6.5 Testování bezpečnosti

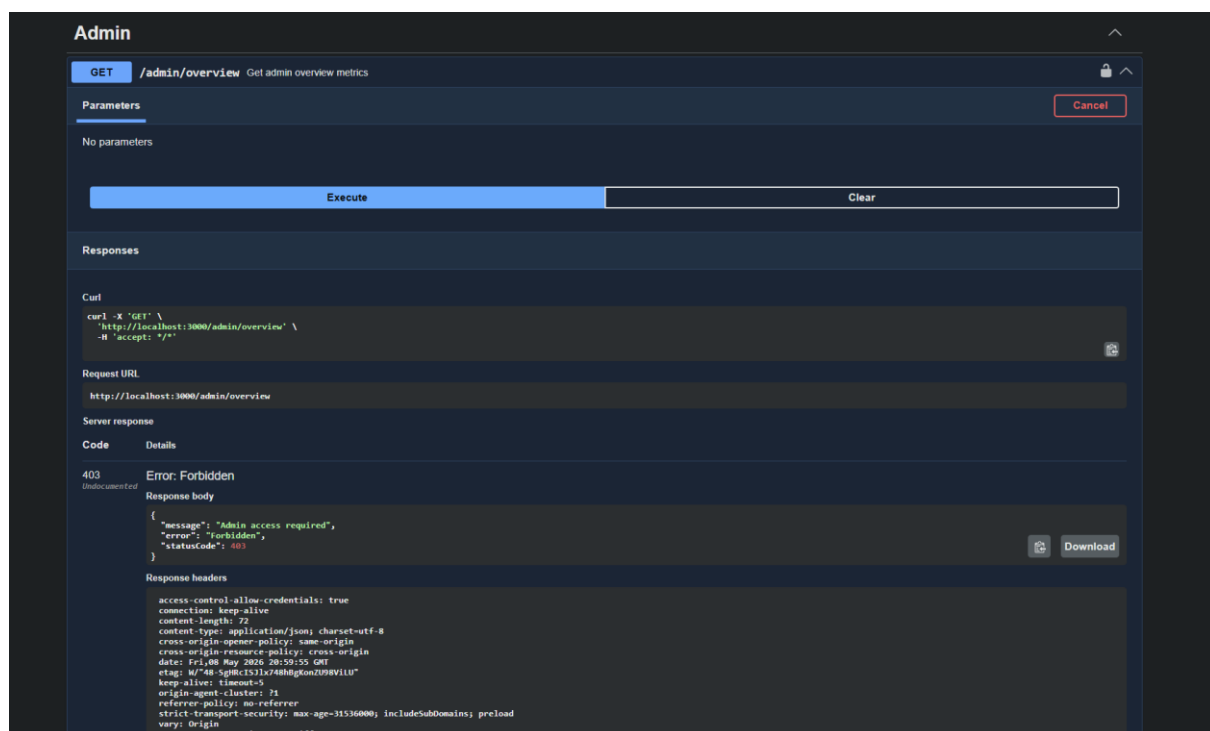
Bezpečnostní testování bylo zaměřeno na základní ochranu API, oprávnění uživatelů a validaci vstupních dat. Cílem nebylo provést plnohodnotný penetrační test, ale ověřit, že aplikace správně chrání citlivé části systému a nevychází pouze ze stavu frontendového rozhraní.

První část tvořila kontrola autentizace. Ověřeno bylo, že chráněné endpointy odmítají požadavky bez platného přihlášení a že frontend po odhlášení zruší uživatelskou relaci i websocketové spojení.



Obrázek 39: Odepření přístupu k chráněnému endpointu bez autentizace (vlastní zpracování)

Další část se týkala autorizace. Testováno bylo, že uživatel nemůže upravovat cizí nabídky, číst cizí konverzace, měnit obchod, ve kterém není účastníkem, nebo provádět administrátorské operace bez odpovídající role.



Obrázek 40: Ochrana administrátorského endpointu pomocí role-based autorizace (vlastní zpracování)

Validace vstupních dat byla ověřena na DTO objektech a vybraných API endpointech. Backend odmítá požadavky s chybějícími nebo neplatnými hodnotami, například při registraci, vytváření nabídky, práci s obchodem nebo odesílání formulářů.

Bezpečnostní testování potvrdilo, že hlavní pravidla jsou vynucena na backendové straně. Frontend může skrýt tlačítko nebo zjednodušit navigaci, ale rozhodující kontrola probíhá až při zpracování požadavku na serveru.

6.6 Uživatelské testování

Uživatelské testování probíhalo formou průchodu hlavními scénáři v roli kupujícího, prodávajícího a administrátora. Cílem bylo ověřit, zda jednotlivé obrazovky dávají smysl při běžném používání a zda uživatel rozumí návaznosti obchodního procesu.

V roli prodávajícího bylo ověřeno vytvoření profilu, publikování nabídky, úprava položky, komunikace s kupujícím a označení dodání. V roli kupujícího byl testován průchod katalogem, otevření detailu nabídky, vytvoření obchodu, chat s prodávajícím, dokončení obchodu a vytvoření recenze.

Administrátorský scénář zahrnoval kontrolu reportů, moderaci obsahu, změnu stavu uživatele, práci s achievementy a odeslání systémového oznámení. Tyto kroky ověřily, že administrace navazuje na běžné uživatelské akce a není izolovanou částí bez vazby na zbytek systému.

Během manuálního testování byly sledovány také chybové stavy a zpětná vazba rozhraní. Kontrolováno bylo, zda aplikace zobrazuje srozumitelnou chybu při neúspěšné operaci, prázdném seznamu, neplatném formuláři nebo nedostatečném oprávnění.

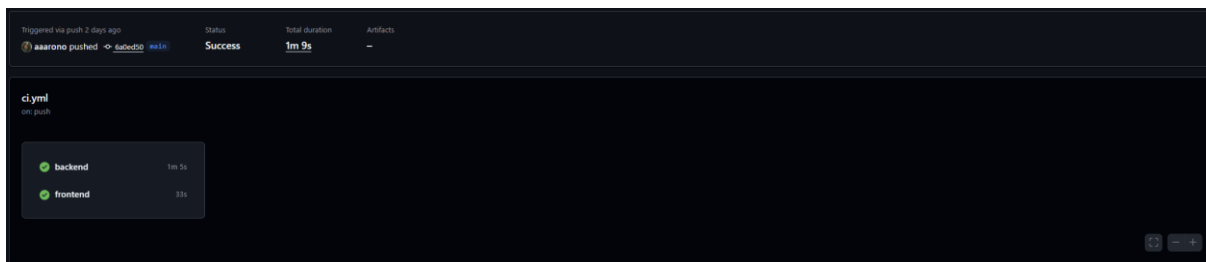
6.7 Shrnutí testování

Testování potvrdilo funkčnost hlavních částí marketplace platformy. Automatizované backendové testy ověřily vybrané service vrstvy, validační pravidla, vlastnické kontroly, stavové přechody obchodů a práci s peněženkou a ledgerem. Největší důraz byl kladen na obchodní a finanční logiku, protože právě zde je nutné zachovat konzistentní stav napříč více entitami.

Manuální frontendové testování ověřilo hlavní uživatelské scénáře: registraci, přihlášení, katalog nabídek, detail nabídky, vytvoření a správu nabídky, obchodní proces, peněženko, profil, inbox a administraci.

Realtime testování potvrdilo, že chat, inbox, online stav, systémové zprávy a změny obchodů fungují bez obnovení stránky. Bezpečnostní kontroly zároveň ukázaly, že chráněné operace nejsou závislé pouze na frontendovém rozhraní, ale jsou vynuceny backendem.

Součástí kontroly byla také CI pipeline, která spouští lint, testy a build proces pro backendovou i frontendovou část aplikace. Tím vznikla základní automatizovaná kontrola kvality před sloučením změn do projektu.



Obrázek 41: Automatizované spuštění testů a build procesu pomocí GitHub Actions (vlastní zpracování)

Výsledkem testování je ověření, že aplikace podporuje hlavní workflow marketplace platformy: publikování nabídky, komunikaci uživatelů, vytvoření obchodu, escrow operaci, dokončení transakce, zápis do ledgeru a následné hodnocení.

6.8 Omezení testování

Testování mělo několik omezení. Automatizované testy se zaměřily hlavně na backend a klíčovou business logiku. Frontendová část byla ověřována převážně manuálně a projekt zatím neobsahuje samostatnou sadu automatizovaných frontendových unit nebo end-to-end testů.

Podobné omezení platí pro realtime komunikaci. Chat, inbox, online stav, systémové notifikace a aktualizace obchodů byly testovány prakticky mezi více klienty, ale websocketová vrstva není pokryta rozsáhlými automatizovanými integračními testy.

Část backendových end-to-end testů pracuje s mockovanou databázovou vrstvou. Tyto testy jsou vhodné pro základní kontrolu API, validace a chráněných tras, ale nenahrazují plnohodnotné integrační testování nad reálnou databází a kompletním produkčním prostředím.

Interní peněženka a finanční operace v aplikaci slouží jako demonstrační model. Testování proto neřešilo napojení na reálnou platební bránu, webhooks, externí finanční infrastrukturu ani právní aspekty skutečných plateb.

Pro další vývoj by bylo vhodné doplnit automatizované testy frontendových scénářů, integrační testy nad reálnou databází, samostatné websocketové testy a detailnější testování chybových stavů finančních operací. U reálné platební integrace by bylo nutné řešit také idempotency klíče, ověřování webhooků a přesnější audit platebních událostí.

Uvedená omezení nebrání splnění cíle práce. Hlavním záměrem bylo navrhnout a implementovat marketplace platformu s obchodním workflow, escrow mechanismem, ledger záznamy a realtime komunikací. Tyto části byly v rámci dostupného rozsahu ověřeny automatizovanými i manuálními testy.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout a implementovat webovou marketplace platformu pro digitální produkty a služby. Tento cíl byl splněn. V práci byla nejprve provedena analýza existujících marketplace platforem a následně byly definovány požadavky na vlastní řešení. Na jejich základě vznikl návrh systému, který propojuje katalog nabídek, uživatelskou komunikaci, obchodní workflow, interní peněženku, escrow mechanismus, recenze, reportování obsahu a administrátorskou správu.

Výsledkem práce je funkční fullstack aplikace tvořená frontendovou částí v Reactu, backendem ve frameworku NestJS a databází PostgreSQL. Aplikace umožňuje registraci a autentizaci uživatelů, správu nabídek, vytváření obchodů, komunikaci mezi kupujícím a prodávajícím, práci s interní peněženkou, hodnocení dokončených obchodů a moderaci problematického obsahu. Realtime komunikace je využita zejména pro chat, inbox, systémové notifikace a aktualizace stavu obchodů.

Za hlavní přínos práce považuji návrh obchodního procesu založeného na escrow principu. Prostředky kupujícího nejsou předány prodávajícímu okamžitě, ale nejprve se zablokují v interní peněžence a uvolní se až po dokončení obchodu. Každá změna zůstatku je zároveň evidována pomocí ledger záznamu, což umožňuje zpětně dohledat historii finančních operací v rámci aplikace. Zvolená architektura modulárního monolitu se pro tento typ projektu ukázala jako vhodná, protože jednotlivé části systému zůstávají logicky oddělené, ale obchodní a finanční operace mohou probíhat nad jednou databází.

Vytvořená aplikace byla ověřena kombinací automatizovaných backendových testů, základních end-to-end testů API a manuálního testování uživatelských scénářů. Testování potvrdilo funkčnost hlavních částí systému, zejména obchodního workflow, oprávnění uživatelů, interní peněženky, chatu, administrace a realtime aktualizací. Současně se při testování ukázala omezení současné verze. Interní peněženka slouží jako demonstrační model a není napojena na reálnou platební bránu. Frontendová část byla testována převážně manuálně a websocketová komunikace není pokryta rozsáhlou sadou automatizovaných integračních testů.

V dalším vývoji by bylo vhodné doplnit skutečnou platební integraci, přesnější kontrolu finančních operací, automatizované frontendové testy, integrační testy nad reálnou databází a podrobnější monitoring provozu aplikace. Přínosné by bylo také rozšíření administrační části a doplnění dalších nástrojů pro řešení sporů mezi uživateli. Práce tak vytvořila základ marketplace platformy, která neslouží pouze jako katalog digitálních produktů, ale pokrývá celý

obchodní proces od publikování nabídky přes komunikaci až po dokončení transakce a případnou moderaci obsahu.

POUŽITÁ LITERATURA

- [1] FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. ISBN 978-0321127426.
- [2] KRUG, Steve. Don't Make Me Think: A Common Sense Approach to Web Usability. 3rd ed. New Riders, 2014. ISBN 978-0321965516.
- [3] NEWMAN, Sam. Building Microservices. O'Reilly, 2015. ISBN 978-1491950357.
- [4] FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Dissertation. University of California, 2000.
- [5] EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003. ISBN 978-0321125217.
- [6] KLEPPMANN, Martin. Designing Data-Intensive Applications. O'Reilly Media, 2017. ISBN 978-1449373321.
- [7] PostgreSQL Global Development Group. PostgreSQL Documentation. Dostupné z: <https://www.postgresql.org/docs/>
- [8] NestJS Documentation. NestJS - A progressive Node.js framework. Dostupné z: <https://docs.nestjs.com>
- [9] React Documentation. React - The library for web and native user interfaces. Dostupné z: <https://react.dev>
- [10] Prisma Documentation. Next-generation ORM for Node.js and TypeScript. Dostupné z: <https://www.prisma.io/docs>
- [11] Socket.IO Documentation. Socket.IO - Bidirectional and low-latency communication. Dostupné z: <https://socket.io/docs>
- [12] Docker Documentation. Docker Compose Documentation. Dostupné z: <https://docs.docker.com/compose/>
- [13] MDN Web Docs. HTTP. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- [14] MDN Web Docs. JSON. Dostupné z: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON>
- [15] MDN Web Docs. WebSocket API. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [16] TypeScript Documentation. TypeScript - JavaScript with syntax for types. Dostupné z: <https://www.typescriptlang.org/docs/>

- [17] Tailwind CSS Documentation. Tailwind CSS Documentation. Dostupné z: <https://tailwindcss.com/docs>
- [18] Spring Boot Documentation. Spring Boot Reference Guide. Dostupné z: <https://docs.spring.io/spring-boot/index.html>
- [19] Django Documentation. Django documentation. Dostupné z: <https://docs.djangoproject.com/>
- [20] Laravel Documentation. Laravel Documentation. Dostupné z: <https://laravel.com/docs>
- [21] Angular Documentation. Angular Documentation. Dostupné z: <https://angular.io/docs>
- [22] Vue.js Documentation. Vue.js Documentation. Dostupné z: <https://vuejs.org/guide/introduction.html>
- [23] MySQL Documentation. MySQL 8.4 Reference Manual. Dostupné z: <https://dev.mysql.com/doc/refman/8.4/en/>
- [24] MongoDB Documentation. MongoDB Manual. Dostupné z: <https://www.mongodb.com/docs/manual/>
- [25] TypeORM Documentation. TypeORM Documentation. Dostupné z: <https://typeorm.io>
- [26] FunPay. Dostupné z: <https://funpay.com/>
- [27] G2A. Dostupné z: <https://www.g2a.com/>
- [28] GGsel. Dostupné z: <https://ggssel.net/en>
- [29] Plati.Market. Dostupné z: <https://plati.market/>
- [30] OWASP Foundation. OWASP Top Ten. Dostupné z: <https://owasp.org/www-project-top-ten/>
- [31] JWT.IO. Introduction to JSON Web Tokens. Dostupné z: <https://jwt.io/introduction>
- [32] MDN Web Docs. HTTP cookies. Dostupné z: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

SEZNAM PŘÍLOH

Příloha A: Struktura projektu

Příloha B: Prisma schema databázového modelu

Příloha C: Konfigurace Docker Compose

PŘÍLOHA A: Struktura projektu

Tato příloha obsahuje zjednodušenou strukturu monorepozitáře se zaměřením na hlavní zdrojové soubory backendové a frontendové části aplikace.

```
BCproject/
├── .gitattributes
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── README.md
├── apps/
│   └── backend/
│       ├── .dockerignore
│       ├── .env.example
│       ├── .gitignore
│       ├── .npmrc
│       ├── .prettierrc
│       ├── Dockerfile
│       ├── README.md
│       ├── docker-entrypoint.sh
│       ├── eslint.config.mjs
│       ├── nest-cli.json
│       ├── package-lock.json
│       ├── package.json
│       └── prisma/
│           ├── migrations/
│           │   ├── ..
│           │   └── migration_lock.toml
│           ├── prisma.module.ts
│           ├── prisma.service.ts
│           └── schema.prisma
│       └── src/
│           ├── admin/
│           │   ├── admin.controller.ts
│           │   ├── admin.guard.ts
│           │   ├── admin.module.ts
│           │   └── admin.service.ts
│           ├── dto/
│           │   ├── assign-achievement.dto.ts
│           │   ├── ban-user.dto.ts
│           │   ├── broadcast-system-message.dto.ts
│           │   ├── create-achievement.dto.ts
│           │   ├── list-admin-query.dto.ts
│           │   ├── moderate-report.dto.ts
│           │   ├── update-achievement.dto.ts
│           │   ├── update-user-role.dto.ts
│           │   └── warn-user.dto.ts
│           ├── app.controller.ts
│           ├── app.module.ts
│           ├── app.service.spec.ts
│           ├── app.service.ts
│           ├── auth/
│           │   ├── auth.controller.ts
│           │   ├── auth.module.ts
│           │   ├── auth.service.spec.ts
│           │   ├── auth.service.ts
│           │   ├── decorators/
│           │   │   ├── current-user.decorator.ts
│           │   │   └── roles.decorator.ts
│           │   ├── dto/
│           │   │   ├── forgot-password.dto.ts
│           │   │   ├── login.dto.ts
│           │   │   ├── register.dto.ts
│           │   │   ├── resend-verification.dto.ts
│           │   │   ├── reset-password.dto.ts
│           │   │   └── verify-email.dto.ts
│           │   ├── guards/
│           │   │   ├── jwt-auth.guard.ts
│           │   │   └── roles.guard.ts
│           │   ├── jwt.strategy.ts
│           │   └── mail.service.ts
│           ├── chat/
│           │   ├── chat.gateway.ts
│           │   └── chat.module.ts
│           ├── conversations/
│           │   ├── conversations.controller.ts
│           │   ├── conversations.module.ts
│           │   ├── conversations.service.ts
│           │   └── dto/
│           │       ├── create-conversation.dto.ts
│           │       └── get-older-messages-query.dto.ts
│           └── deals/
```

- deal-timeout.service.ts
- deal-timeout.util.ts
- deals.controller.ts
- deals.module.ts
- deals.service.spec.ts
- deals.service.ts
- dto/
 - create-deal.dto.ts
- listings/
 - dto/
 - create-listing.dto.ts
 - listing-query.dto.ts
 - update-listing.dto.ts
 - listings.controller.ts
 - listings.module.ts
 - listings.service.spec.ts
 - listings.service.ts
- main.ts
- messages/
 - dto/
 - send-message.dto.ts
 - messages.controller.ts
 - messages.module.ts
 - messages.service.ts
- reports/
 - dto/
 - create-report.dto.ts
 - reports.controller.ts
 - reports.module.ts
 - reports.service.ts
- reviews/
 - dto/
 - create-review.dto.ts
 - reviews.controller.ts
 - reviews.module.ts
 - reviews.service.spec.ts
 - reviews.service.ts
- system-notifications/
 - dto/
 - list-system-notifications.dto.ts
 - system-notifications.controller.ts
 - system-notifications.module.ts
 - system-notifications.service.ts
- users/
 - dto/
 - change-password.dto.ts
 - update-active-badge.dto.ts
 - update-me.dto.ts
 - update-payment-card.dto.ts
 - update-profile-badges.dto.ts
 - users.controller.ts
 - users.module.ts
 - users.service.ts
- wallet/
 - dto/
 - top-up.dto.ts
 - withdraw.dto.ts
 - wallet.controller.ts
 - wallet.module.ts
 - wallet.service.spec.ts
 - wallet.service.ts
- test/
 - app.e2e-spec.ts
 - jest-e2e.json
- tsconfig.build.json
- tsconfig.json
- frontend/
 - .dockerignore
 - .env.example
 - .gitignore
 - Dockerfile
 - README.md
 - components.json
 - eslint.config.js
 - index.html
 - nginx.conf
 - package-lock.json
 - package.json
 - postcss.config.js
 - public/
 - vite.svg
 - src/
 - App.css
 - App.tsx
 - api/
 - http.ts
 - socket.ts

- app/
 - router.tsx
- auth/
 - AuthContext.tsx
 - RequireAdmin.tsx
 - RequireAuth.tsx
- components/
 - chat/
 - ChatPanel.tsx
 - ConversationView.tsx
 - SystemConversationView.tsx
 - layout/
 - ErrorBoundary.tsx
 - Navbar.tsx
 - listing/
 - ListingDetails.tsx
 - ListingForm.tsx
 - MarketplaceListingCard.tsx
 - PriceHistoryChart.tsx
 - profile/
 - DealStatusBadge.tsx
 - ProfileHeaderCard.tsx
 - SellerRankCard.tsx
 - report/
 - ReportDialog.tsx
 - review/
 - RatingStars.tsx
 - ReviewForm.tsx
 - ReviewSection.tsx
 - SellerReviewsList.tsx
 - ui/
 - Avatar.tsx
 - Badge.tsx
 - Button.tsx
 - Card.tsx
 - EmptyState.tsx
 - Input.tsx
 - PageLayout.tsx
 - PageStates.tsx
 - Textarea.tsx
- index.css
- lib/
 - cn.ts
 - currency.ts
 - theme.ts
 - utils.ts
- main.tsx
- pages/
 - AdminPage.tsx
 - ConversationPage.tsx
 - CreateListingPage.tsx
 - DealRoomPage.tsx
 - DealsPage.tsx
 - EditListingPage.tsx
 - ForgotPasswordPage.tsx
 - InboxPage.tsx
 - ListingPage.tsx
 - ListingsPage.tsx
 - LoginPage.tsx
 - MyListingsPage.tsx
 - NotFoundPage.tsx
 - ProfilePage.tsx
 - PublicSellerProfilePage.tsx
 - RegisterPage.tsx
 - ResetPasswordPage.tsx
 - SettingsPage.tsx
 - TopSellersPage.tsx
 - VerifyEmailPage.tsx
 - WalletPage.tsx
- types/
 - chat.ts
 - listing.ts
- utils/
 - httpError.ts
- tailwind.config.js
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts
- docker-compose.yml
- docs/
 - BCdoc.docx
- scripts/
 - smoke.ps1

PŘÍLOHA B: Prisma schema databázového modelu

Tato příloha obsahuje Prisma schema, které definuje hlavní databázové entity, jejich atributy, vztahy, enumy a indexy.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum Role {
  BUYER
  SELLER
  ADMIN
}

enum ListingType {
  GOOD
  SERVICE
}

enum ListingCategory {
  GAMES
  ACCOUNTS
  BOOSTING
  MENTORING
  GAME_CURRENCY
  OTHER
}

enum ListingStatus {
  ACTIVE
  ARCHIVED
}

enum DealStatus {
  INITIATED
  FUNDED
  DELIVERED
  COMPLETED
  CANCELED
}

enum DealCancellationActor {
  BUYER
  SELLER
  SYSTEM
}

enum WalletTxType {
  TOPUP
  WITHDRAW
  ESCROW_LOCK
  ESCROW_RELEASE
  REFUND
  WEEKLY_REWARD
}

enum WeeklyCompetitionStatus {
  PENDING
  FINALIZED
  CANCELED
}

enum ReportTargetType {
  LISTING
  USER
  REVIEW
  DEAL
  MESSAGE
}

enum ReportStatus {
  OPEN
  UNDER_REVIEW
  RESOLVED
  REJECTED
}

model User {
```

```

id          String    @id @default(cuid())
email       String    @unique
passwordHash String
emailVerifiedAt DateTime?
emailVerificationTokenHash String?
emailVerificationExpiresAt DateTime?
passwordResetTokenHash String?
passwordResetExpiresAt DateTime?
displayName String
avatarUrl   String?
activeBadgeDefinitionId String?
paymentCardLast4 String?
paymentCardBrand String?
paymentCardLinkedAt DateTime?
role        Role      @default(BUYER)
isBannedPermanent Boolean @default(false)
bannedUntil DateTime?
banReason String?
warningStage Int @default(1)
createdAt   DateTime @default(now())

ratingAvg   Float @default(0)
ratingCount Int  @default(0)

listings Listing[]
wallet       Wallet?

buyerConversations Conversation[] @relation("BuyerConversations")
sellerConversations Conversation[] @relation("SellerConversations")
buyerDeals     Deal[]             @relation("BuyerDeals")
sellerDeals    Deal[]             @relation("SellerDeals")

messages      Message[]
walletTransactions WalletTransaction[]

reviewsGiven  Review[] @relation("BuyerReviews")
reviewsReceived Review[] @relation("SellerReviews")
achievements  UserAchievement[]
profileBadges UserProfileBadge[]
activeBadgeDefinition AchievementDefinition? @relation("ActiveBadgeDefinition", fields:
[activeBadgeDefinitionId], references: [id])
weeklyStats UserWeeklyStats?
wonCompetitions WeeklyCompetition[] @relation("CompetitionWinner")
weeklyWinnerSnapshots WeeklyWinner[]
createdAchievementAssignments AchievementAssignment[]
@relation("AdminAchievementAssignments")
assignedAchievements AchievementAssignment[]
@relation("UserAchievementAssignments")
systemNotifications SystemNotification[] @relation("UserSystemNotifications")
createdSystemNotifications SystemNotification[] @relation("AdminSystemNotifications")
reportsSubmitted Report[] @relation("ReportsSubmitted")
reportsReviewed Report[] @relation("ReportsReviewed")
adminAuditLogs AuditLog[] @relation("AdminAuditLogs")
warningsReceived UserWarning[] @relation("WarningsReceived")
warningsIssued UserWarning[] @relation("WarningsIssued")
warningsRevoked UserWarning[] @relation("WarningsRevoked")

@@index([activeBadgeDefinitionId])
}

model UserWarning {
  id String @id @default(cuid())
  userId String
  issuedByAdminId String
  reason String
  expiresAt DateTime
  createdAt DateTime @default(now())
  revokedAt DateTime?
  revokedByAdminId String?

  user User @relation("WarningsReceived", fields: [userId], references: [id])
  issuedByAdmin User @relation("WarningsIssued", fields: [issuedByAdminId], references: [id])
  revokedByAdmin User? @relation("WarningsRevoked", fields: [revokedByAdminId], references:
[id])

  @@index([userId, createdAt])
  @@index([userId, expiresAt, revokedAt])
  @@index([issuedByAdminId, createdAt])
}

model AchievementDefinition {
  id          String    @id @default(cuid())
  code        String    @unique
  title       String
  description String
  createdAt   DateTime @default(now())

```

```

users UserAchievement[]
profileBadgeUsers UserProfileBadge[]
activeBadgeUsers User[] @relation("ActiveBadgeDefinition")
manualAssignments AchievementAssignment[]
}

model UserProfileBadge {
  id String @id @default(cuid())
  userId String
  definitionId String
  sortOrder Int
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id])
  definition AchievementDefinition @relation(fields: [definitionId], references: [id])

  @@unique([userId, definitionId])
  @@unique([userId, sortOrder])
  @@index([userId, sortOrder])
}

model UserAchievement {
  id String @id @default(cuid())
  userId String
  definitionId String
  unlockedAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id])
  definition AchievementDefinition @relation(fields: [definitionId], references: [id])

  @@unique([userId, definitionId])
  @@index([userId, unlockedAt])
}

model AchievementAssignment {
  id String @id @default(cuid())
  adminId String
  userId String
  definitionId String
  createdAt DateTime @default(now())

  admin User @relation("AdminAchievementAssignments", fields: [adminId],
references: [id])
  user User @relation("UserAchievementAssignments", fields: [userId],
references: [id])
  definition AchievementDefinition @relation(fields: [definitionId], references: [id])

  @@index([createdAt])
  @@index([adminId, createdAt])
  @@index([userId, createdAt])
}

model WeeklyCompetition {
  id String @id @default(cuid())
  weekStart DateTime @unique
  weekEnd DateTime
  status WeeklyCompetitionStatus @default(PENDING)
  winnerUserId String?
  rewardAmount Int @default(0)
  finalizedAt DateTime?
  createdAt DateTime @default(now())

  winner User? @relation("CompetitionWinner", fields: [winnerUserId], references:
[id])
  summary WeeklyWinner?

  @@index([status, weekStart])
  @@index([winnerUserId, weekStart])
}

model WeeklyWinner {
  id String @id @default(cuid())
  competitionId String @unique
  userId String
  rank Int
  score Float
  completedBeats Int
  ratingAvgSnapshot Float
  ratingCountSnapshot Int
  activeListings Int
  streakAfterWin Int
  rewardAmount Int
  createdAt DateTime @default(now())

  competition WeeklyCompetition @relation(fields: [competitionId], references: [id])
  user User @relation(fields: [userId], references: [id])
}

```

```

    @@index([userId, createdAt])
    @@index([createdAt])
}

model UserWeeklyStats {
  userId      String    @id
  totalWins   Int        @default(0)
  currentStreak Int      @default(0)
  bestStreak  Int        @default(0)
  lastWinWeekStart DateTime?
  updatedAt   DateTime @updatedAt

  user User @relation(fields: [userId], references: [id])
}

model SystemNotification {
  id          String    @id @default(cuid())
  userId      String
  senderAdminId String?
  title       String
  text        String
  createdAt   DateTime @default(now())
  readAt      DateTime?

  user User @relation("UserSystemNotifications", fields: [userId], references: [id])
  senderAdmin User? @relation("AdminSystemNotifications", fields: [senderAdminId], references: [id])

  @@index([userId, createdAt])
  @@index([userId, readAt])
  @@index([senderAdminId, createdAt])
}

model Report {
  id          String    @id @default(cuid())
  reporterId  String
  targetType  ReportTargetType
  targetId    String
  reason      String
  details     String?
  status      ReportStatus @default(OPEN)
  adminNote   String?
  reviewedByAdminId String?
  reviewedAt  DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  reporter User @relation("ReportsSubmitted", fields: [reporterId], references: [id])
  reviewedByAdmin User? @relation("ReportsReviewed", fields: [reviewedByAdminId], references: [id])

  @@index([reporterId, createdAt])
  @@index([targetType, targetId])
  @@index([status, createdAt])
  @@index([reviewedByAdminId, reviewedAt])
}

model AuditLog {
  id          String    @id @default(cuid())
  actorAdminId String
  action      String
  entityType  String
  entityId    String?
  requestId  String?
  summary     String?
  before     Json?
  after      Json?
  metadata   Json?
  createdAt  DateTime @default(now())

  actorAdmin User @relation("AdminAuditLogs", fields: [actorAdminId], references: [id])

  @@index([actorAdminId, createdAt])
  @@index([action, createdAt])
  @@index([entityType, entityId])
  @@index([createdAt])
}

model Listing {
  id          String @id @default(cuid())
  sellerId   String
  seller     User    @relation(fields: [sellerId], references: [id])

  title       String
  description String
  imageUrl    String?
  stockQuantity Int?

```

```

price          Int // cents
category       ListingCategory @default(OTHER)
tags           String[]         @default([])
salePercent    Int?
saleStartsAt   DateTime?
saleEndsAt     DateTime?
type           ListingType
status         ListingStatus @default(ACTIVE)
createdAt      DateTime         @default(now())

conversations  Conversation[]
Deal           Deal[]
priceHistory   ListingPriceHistory[]

@@index([sellerId])
@@index([category])
}

model ListingPriceHistory {
  id          String @id @default(cuid())
  listingId   String
  price       Int
  isSale      Boolean @default(false)
  salePercent Int?
  createdAt   DateTime @default(now())

  listing Listing @relation(fields: [listingId], references: [id])

  @@index([listingId, createdAt])
  @@index([listingId, isSale, createdAt])
}

model wallet {
  userId      String @id
  user        User   @relation(fields: [userId], references: [id])
  balance     Int    @default(0)
  walletTransaction WalletTransaction[]
}

model walletTransaction {
  id          String @id @default(cuid())
  walletId    String
  type        WalletTxType
  amount      Int // + / - в центрах
  dealId      String?
  userId      String
  createdAt   DateTime @default(now())

  wallet wallet @relation(fields: [walletId], references: [userId])
  User    User  @relation(fields: [userId], references: [id])

  @@index([walletId])
  @@index([dealId])
  @@index([userId, createdAt])
}

model Conversation {
  id          String @id @default(cuid())
  listingId   String
  buyerId    String
  sellerId    String
  buyerLastReadAt DateTime?
  sellerLastReadAt DateTime?
  createdAt   DateTime @default(now())

  listing Listing @relation(fields: [listingId], references: [id])
  buyer  User    @relation("BuyerConversations", fields: [buyerId], references: [id])
  seller  User    @relation("SellerConversations", fields: [sellerId], references: [id])

  messages Message[]

  @@unique([listingId, buyerId])
  @@index([sellerId])
  @@index([buyerId])
  @@index([buyerId, createdAt])
  @@index([sellerId, createdAt])
  @@index([buyerId, buyerLastReadAt])
  @@index([sellerId, sellerLastReadAt])
}

model Message {
  id          String @id @default(cuid())
  conversationId String
  senderId    String
  text        String
  mediaUrl    String?
  mediaType   String?

```



```

mediaItems      Json?
createdAt       DateTime @default(now())

conversation Conversation @relation(fields: [conversationId], references: [id])
sender          User       @relation(fields: [senderId], references: [id])

@@index([conversationId, createdAt])
@@index([conversationId, senderId, createdAt])
}

model Deal {
  id          String      @id @default(cuid())
  listingId   String
  buyerId    String
  sellerId    String
  quantity    Int         @default(1)
  unitPriceSnapshot Int
  totalAmountSnapshot Int
  expiresAt   DateTime?
  canceledByActor DealCancellationActor?
  status      DealStatus @default(INITIATED)
  createdAt   DateTime   @default(now())

  listing Listing @relation(fields: [listingId], references: [id])
  buyer  User    @relation("BuyerDeals", fields: [buyerId], references: [id])
  seller  User    @relation("SellerDeals", fields: [sellerId], references: [id])
  Review  Review?

  @@index([buyerId])
  @@index([sellerId])
  @@index([buyerId, createdAt])
  @@index([sellerId, createdAt])
  @@index([listingId, buyerId, status, createdAt])
  @@index([status, expiresAt])
}

model Review {
  id          String      @id @default(cuid())
  rating      Int
  comment     String?
  createdAt   DateTime   @default(now())

  dealId      String @unique
  buyerId    String
  sellerId    String

  deal Deal @relation(fields: [dealId], references: [id])
  buyer User @relation("BuyerReviews", fields: [buyerId], references: [id])
  seller User @relation("SellerReviews", fields: [sellerId], references: [id])

  @@index([sellerId])
}

```

PŘÍLOHA C: Konfigurace Docker Compose

Tato příloha obsahuje soubor docker-compose.yml použitý pro lokální spuštění databáze, backendu a frontendové části aplikace.

```
services:
  db:
    image: postgres:16
    container_name: marketplace_db
    environment:
      POSTGRES_USER: marketplace
      POSTGRES_PASSWORD: marketplace
      POSTGRES_DB: marketplace
    ports:
      - "5432:5432"
    volumes:
      - marketplace_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U marketplace -d marketplace"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build:
      context: ./apps/backend
      dockerfile: Dockerfile
    container_name: marketplace_backend
    env_file:
      - ./apps/backend/.env
    environment:
      DATABASE_URL: postgresql://marketplace:marketplace@db:5432/marketplace?schema=public
      PORT: "3000"
      NODE_ENV: production
      CORS_ORIGIN: http://localhost:8080,http://localhost:5173
    ports:
      - "3000:3000"
    healthcheck:
      test: ["CMD-SHELL", "wget -q -O - http://localhost:3000/health >/dev/null 2>&1 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s
    volumes:
      - marketplace_uploads:/app/uploads
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build:
      context: ./apps/frontend
      dockerfile: Dockerfile
    args:
      VITE_API_URL: http://localhost:3000
      VITE_WS_URL: http://localhost:3000
    container_name: marketplace_frontend
    ports:
      - "8080:80"
    healthcheck:
      test: ["CMD-SHELL", "wget -q -O - http://localhost/ >/dev/null 2>&1 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 10s
    depends_on:
      backend:
        condition: service_healthy

volumes:
  marketplace_pgdata:
  marketplace_uploads:
```